

“本体论”的那些事儿

—从Web语义本体、知识图谱本体到Palantir Ontology



极客时间

2026-05-11

为什么现在大家都在谈“本体论”？

背景：大模型的“失控”困境



幻觉问题：不可信

大模型容易产生不符合事实的“幻觉”，无法应用于法律、医疗等严肃高价值场景。



不可控性：无逻辑

生成内容发散且难以预测，无法保证逻辑推理的严密性，导致决策失误风险高。



不可审计：黑箱化

模型推理过程不透明，决策依据无法追溯，企业难以承担由此产生的责任与后果。

工程版“本体论”：为AI建立“语义笼子”

定义：领域的“语义Schema”

对特定领域的**概念**（如用户、风险）、**关系**（如用户→下单→订单）、**规则**（如金额>10万需风控）进行结构化的精确描述，不再是抽象的哲学追问。

核心价值：结构化与可解释性

为大模型构建一个可解释的“**语义笼子**”，强制其推理和输出严格遵循预设的领域框架，从根本上解决“失控”难题，提升AI的商业可信度。



核心困境-盲人摸象的四重盲视

01 / 概念混淆：“本体论”到底是什么？

各方对“本体论”的理解和指代完全不同，导致讨论基础错位，形成五大流派：



结论：同一个词，每个人摸的是完全不同的部位，却都在说“我摸到了本体论”。

02 / 局部宣称全局：万能解药的迷思

各个技术流派都将自己的局部解决方案，夸大为解决AI所有问题的“万能解药”：



结论：都是局部有效，但没有一个能单独解决AI的全部问题。



核心困境-盲人摸象的四重盲视

03 落地碎片化：没有统一的“大象全貌图”

工程落地层面极度分散，缺乏统一标准和互操作性，导致各方“各说各话”。

- **学术界：** 专注OWL、描述逻辑等理论研究，门槛高且落地路径模糊。
- **科技大厂：** 阿里、华为、百度等各自为政，内部体系互不兼容，形成“技术孤岛”。
- **中小企业：** 多为跟风炒作，常将ER图或简单数据模型直接“改名”为本体论。

结论： 行业缺乏统一的标准和顶层蓝图，每个人都在自己的小圈子里“盲人摸象”。

04 刻意淡化成本：“重、慢、贵”的现实

在营销宣传中避而不谈本体论构建和维护的高昂成本，存在明显的误导性。

- **构建成本极高：** 需行业专家与技术专家深度协作数月，时间与人力投入巨大。
- **维护更新困难：** 业务逻辑的微小变动，可能导致整个本体体系的大规模重构。
- **存在过度设计：** 大量场景下，轻量级Schema结合提示词工程，可实现更优的“性价比”。

结论： 只谈“智能”不谈代价，是当下AI行业典型的“盲人摸象式营销”。



热潮之下的冷思考



本质：行业焦虑下的新“神话”

这波本体论热，本质上是**大模型泡沫破裂后，行业在寻找新“神话”的集体焦虑体现。**

它被包装成了那个“看起来很高级、能解决所有问题”的新故事。



理性：承认价值，避免神化

- **承认价值：**在知识推理、数据治理等特定场景有不可替代的作用。
- **避免神化：**它不是解决AI问题的“银弹”，而是众多工具中的一种。
- **务实落地：**推动轻量级、可演进的构建方法，避免“过度设计”。



最终结论

“最近的本体论热，是AI圈集体‘盲人摸象’的生动写照：每个人都摸到了真实的局部，却都误以为摸到了全部。”

未来行业更需要的是**开放标准的建立与务实的落地方法**，而非追逐下一个“神话”。

主要内容

- 01 Semantic Web Ontology
- 02 Knowledge Graph Schema
- 03 Palantir & Palantir Foundry
- 04 Palantir Ontology的三层架构
- 05 Palantir Ontology的五大要素
- 06 Palantir Ontology的本体构建及案例
- 07 Ontology驱动数据治理及决策
- 08 Ontology驱动大模型Agent执行



本体(Ontology)

广义：对某个领域知识的一种形式化、规范化的描述。

狭义：在不同技术体系中有不同的具体实现，如OWL、Schema、PalantirOntology。



语义(Semantic)

定义：数据所表达的业务含义，是连接底层数据与上层业务价值的桥梁。

目标：消除歧义，实现数据在不同系统间的统一理解和互操作。



推理(Reasoning)

定义：基于已有的知识库与规则，自动推导出隐含的、新的业务知识的过程。

类型：主要分为基于逻辑规则的逻辑推理和基于数据挖掘的统计推理。



最基本的三大ontology



Semantic Web Ontology(OWL)

逻辑严密的知识表示

- 源于AI知识表示领域，核心追求逻辑完备性与无歧义。
- 采用OWL、RDFS等标准语言，支持基于描述逻辑的强推理能力。
- 典型应用：学术研究、医疗/制造等领域的行业标准建立。



知识图谱Schema

实用主义的类型标签系统

- 工业界主流选择，核心侧重海量数据的有效组织与检索应用。
- 轻量级模式定义，易于扩展，支持大规模自动化构建与迭代。
- 典型应用：搜索引擎优化、智能推荐系统、通用问答助手。



Palantir语义层

以业务为中心的可执行数据模型

- 融合语义统一与业务操作能力，打通数据到业务行动的链路。
- 支持业务人员通过可视化方式参与建模，降低技术门槛。
- 典型应用：企业级复杂数据应用构建、业务流程自动化。



最基本的三大ontology

Semantic Web Ontology(OWL)

建模逻辑: 严格形式化逻辑、公理约束, 强调逻辑严密性。

构建方式: 主要由领域专家手工自上而下定义, 门槛较高。

推理能力: 具备强逻辑推理能力, 支持基于公理的复杂推理。

规模风格: 小而精, 追求严谨性与标准化, 不追求海量数据。

典型用途: 制定行业标准、医学/生物术语库、知识体系标准化。

普通知识图谱(KG)

建模逻辑: 以图论为基础, 强调图节点与边的存储与关联。

构建方式: 多采用半自动化技术抽取, 辅以人工校对。

推理能力: 推理能力相对较弱, 更多用于图路径查询与检索。

规模风格: 大而全, 重点在于覆盖海量实体与复杂关联网络。

典型用途: 智能搜索、问答系统、关联检索、金融反欺诈。

PalantirOntology

建模逻辑: 以业务为导向的语义对象建模, 贴合业务视角。

构建方式: 通过业务建模器拖拽配置, 降低技术门槛, 易于迭代。

推理能力: 基于函数计算与事件触发Action, 实现业务逻辑自动化。

规模风格: 适配业务系统规模, 注重多源异构数据的融合。

典型用途: 企业级数据语义层、业务流程自动化、情报分析。

01 Semantic Web Ontology

逻辑严密的知识表示



起源背景：从AI知识工程到语义网

Web Ontology的思想深深植根于**人工智能(AI)**领域的**知识表示与知识工程**。

它的核心目标，是为互联网上的海量数据赋予结构化的“意义”，构建一个让机器能够理解、推理和处理复杂知识的统一框架，以此解决传统Web“机器可读但不可理解”的局限，从而实现真正的“语义互联”。



经典落地领域与标杆应用



生物医学：标准化术语体系的基石

- **GO(GeneOntology)**: 描述基因功能的权威标准本体。
- **SNOMEDCT**: 全球最全面的临床医疗术语本体，广泛用于电子病历。



学术知识库：链接数据的引擎

- **DBpedia**: 从维基百科提取的多语言知识库，被誉为“语义网的核心”。
- 利用本体技术，将非结构化百科信息转化为机器可推理的结构化知识图谱。



行业标准：跨机构协作的通用语言

在金融、政务、智能制造等领域，用于定义跨组织的数据交换标准和业务规则。



01/核心定义

领域本体是对某个专业领域内**概念(类)**、**属性**、**关系**、**约束**、**逻辑公理**、**推理规则**所做的**显式**、**形式化**、**可机器理解**的严格规范定义。它是连接现实世界语义与机器可处理数据的桥梁，为知识的结构化表达提供了基础框架。

02/核心目标



知识共享与互操作

实现跨系统、跨平台的知识共享，消除“信息孤岛”，让异构系统能够理解并交换基于同一套概念模型的数据。



自动化逻辑推理

基于本体中定义的逻辑公理和推理规则，从已有的显性知识库中自动推导出隐含的新事实，从而实现知识的增值。



构建符号系统

将复杂多变的现实世界业务逻辑，抽象为计算机能够理解、处理并进行逻辑推演的符号化体系，实现业务知识的计算化。



Semantic Web Ontology-四大核心构成要素

01.类(Class)

代表领域中的核心概念和分类，是构建本体的基础骨架。

示例：
Person(人),Disease(疾病),Drug(药物)

02.属性/关系

用于描述类的内在特征（数据属性），或连接不同类之间的逻辑关联（对象属性）。

示例：
hasAge(有年龄),hasParent(有父母),treats(治疗)

03.约束(Constraints)

对属性和关系的取值范围、基数（数量）等施加逻辑限制，如基数约束、值域约束、不相交约束等，确保数据的一致性。

示例：
一个人(Person)恰好有两个亲生父母；年龄必须大于0。

04.推理规则

基于现有本体知识和逻辑公理，推导出隐含的新知识，赋予本体“智能”。

示例：
 $\text{hasParent}(x,y) \wedge \text{hasParent}(y,z) \rightarrow \text{hasGrandparent}(x,z)$



Semantic Web Ontology-要素1-Class领域类/概念

01/定义Definition

类(Class)是对领域中具有共同特征的一组实体的抽象，是构建本体的基础。它的逻辑地位与面向对象编程(OOP)中的“类”，或者关系型数据库中的“表”非常相似，是对客观事物的结构化描述。

示例：Person(人),Vehicle(交通工具),Organization(组织)

02/关键特性KeyFeatures

- 层级关系**：通过rdfs: subClassOf构建继承体系，如`CardiovascularDisease`是`Disease`的子类。
- 等价类**：使用owl: equivalentClass声明两个类指代完全相同的概念集合。
- 互斥类**：使用owl: disjointWith声明两个类在实例上无交集，确保分类严谨性。

03/跨领域示例Examples



生物医学领域Biomedical

核心类：`Gene` • `Protein` • `Disease` • `Symptom`



金融领域Finance

核心类：
`Customer` • `Account` • `Transaction` • `Product`



制造业领域Manufacturing

核心类：
`Equipment` • `Product` • `Supplier` • `ProductionLine`



01/属性类型PropertyTypes



数据属性(DatatypeProperty)

描述类自身的特征，连接个体与具体的字面量数据值（如数字、字符串）。

示例: `hasAge(Person→xsd: integer)`, `hasName(Person→xsd: string)`



对象属性(ObjectProperty)

描述不同类个体之间的关联，连接的两端均为实体，用于表达复杂关系。

示例: `hasParent(Person→Person)`, `treats(Doctor→Disease)`

02/关键特性KeyCharacteristics



定义域(rdfs: domain)

定义属性适用的主体类。即属性“属于”哪个类。
例: `hasAge`的domain是Person



值域(rdfs: range)

定义属性值的类型或目标类。限制属性取值的范围。
例: `hasAge`的range是xsd: integer(整数)



逆属性(owl: inverseOf)

定义属性的反向逻辑关系，便于从不同方向查询关联。
例: `hasChild`与`hasParent`互为逆属性



约束是保证本体逻辑严谨性的关键，OWL提供了丰富的约束机制以构建精确的语义规则。



1.数据类型约束(Datatype)

用于严格限制属性值的数据类型，确保数据的规范性。例如：声明**hasAge**属性的值必须是整数类型(**xsd: integer**)，而非字符串或其他类型。



2.基数约束(Cardinality)

定义属性值的数量范围，控制实例间的关联强度。例如：人类的**hasBiologicalParent**属性的基数可设定为**exactly2**，确保每个实例必须且仅有两个父母。



3.量词约束(Quantifiers)

- 全称(allValuesFrom)**: 属性的所有值必须属于指定类。如：人(Person)的子女(hasOffspring)必须是人。
- 存在(someValuesFrom)**: 至少存在一个值属于指定类。如：学生(Student)必须至少选修一门课程(Course)。



4.不相交约束(Disjointness)

用于声明两个或多个类之间互斥，没有共同的实例。这在分类体系构建中至关重要。

示例：**Manowl: disjointWithWoman**



推理规则是本体能够自动发现新知识、实现知识图谱智能化的核心引擎

核心表达形式

OWL公理(OWLAxioms)

通过类、属性和约束的组合来隐式表达逻辑。例如，将属性 **hasParent** 定义为“传递性”，推理机即可自动识别并推断出所有隐含的祖孙关系，无需显式编程。

SWRL规则(SemanticWebRuleLanguage)

提供更灵活、显式的“IF...THEN...”逻辑定义。对于复杂业务规则，其表达能力远超基础的OWL公理，是实现复杂业务推理的首选工具。

祖孙关系推理示例

业务逻辑：如果实体X是Y的父母，且实体Y是Z的父母，那么逻辑上X必然是Z的祖父母。

```
hasParent(?x,?y) ^ hasParent(?y,?z) → hasGrandparent(?x,?  
z)
```

//SWRL语法实现上述逻辑

自动执行：推理引擎（如HermiT、Pellet）扫描全库数据，自动为所有满足条件的三元组推导出新关系并补全图谱。



RDFS(RDFSchema)

定位：基础的本体语言，是构建Web Ontology的基石。

核心能力：提供定义类、属性的基础能力，支持类与类、属性与属性之间的继承关系（subClassOf/subPropertyOf）。

局限性：表达能力相对较弱，无法定义复杂的逻辑约束或关系特征。



OWL(Web Ontology)

定位：建立在RDFS之上的描述逻辑语言，是本体构建的核心标准。

核心能力：支持等价类、互斥类、基数约束、传递性、对称性等复杂逻辑定义，满足复杂知识表达需求。

主要版本：OWL1和OWL2(包含EL、QL、RL等不同表达能力的子语言，以适应不同场景)。



SWRL(RuleLanguage)

定位：语义网规则语言扩展，专为解决本体语言无法覆盖的业务逻辑推理而生。

核心能力：允许在OWL本体之上添加类似“IF-THEN”的规则，极大增强了对复杂业务规则的表达与推理能力。

关键特点：通常需与OWL本体结合使用，并依赖支持SWRL规则的专用推理机。

主要构建工具

Protégé: 由斯坦福大学开发的开源本体编辑器，功能强大且社区活跃，是当前学术界和工业界本体构建的首选工具。

TopBraidComposer: 一款商业化的综合本体开发环境，集成了多种本体管理与开发功能，适合企业级应用场景。

主要推理机

HermiT: 一款高效的开源OWL本体推理机，支持OWL2标准，以优异的性能和对复杂本体的处理能力著称。

Pellet: 开源的基于Java的OWL DL推理机，提供了全面的推理支持，广泛应用于科研与项目开发中。

主要查询语言 - SPARQL

RDF数据的标准查询语言

SPARQL (SPARQL Protocol and RDF Query Language) 是 W3C 推荐的语义网标准查询语言，用于从 RDF 知识库中检索和操作数据。

它在语义网中的地位类似于 SQL 在关系型数据库中的地位，支持对本体知识库进行复杂的模式匹配、聚合和数据提取，是连接本体数据与上层应用的关键接口。



核心思想Core Idea

Web Ontology的构建遵循**自上而下(Top-Down)**的路径：从定义领域内最顶层的抽象概念与关系入手，确立宏观框架，随后层层向下细化，直至形成包含具体子类、属性和实例的完整知识体系。



“小而精”

规模未必庞大，但概念定义精准，逻辑推理严密，语义高度严谨。



可标准化

旨在成为行业通用的知识标准，便于跨系统集成与共享。

专家主导模式Expert-Led

- 领域专家**：输出核心概念、业务规则与术语体系，奠定知识基石。
- 知识工程师**：将隐性知识转化为标准化的OWL语言，实现逻辑落地。
- 核心目标**：确保本体具备**逻辑完备性**、**语义一致性**和**严格约束**。

构建流程逻辑示意

顶层定义
抽象概念与关系



逐步细化
子类/属性/实例



形式化本体
OWL/RDF语言



Semantic Web Ontology-构建方法论-自上而下专家建模



01 范围界定

明确本体的应用领域和边界，界定其覆盖的知识范围，确保构建方向清晰准确。



02 概念化

识别领域内的核心概念（类）、属性以及它们之间的语义关系，建立知识的抽象模型。



03 形式化

将概念化模型转换为机器可读的形式语言，例如使用OWL、RDFS等本体描述语言进行逻辑编码。



04 实现与编码

使用专业的本体编辑工具（如Protégé）将形式化的逻辑模型构建为可运行的本体知识库。



05 评估与迭代

利用推理机进行一致性和完整性检查，并收集领域专家反馈，不断修正和完善本体模型。



STEP01 领域范围界定&术语梳理

核心目标

明确本体的知识边界，确定要解决的核心问题，并形成统一的术语共识，为后续建模奠定坚实基础。

关键活动

- 确定本体应用场景与解决的核心问题，明确建模范围
- 广泛收集、整理并规范领域内的核心术语与定义
- 输出一份结构化的领域术语表(Glossary)及知识清单



STEP02 定义层级Class类体系

核心目标

构建本体的“骨架”——即分类结构(Taxonomy)，确立领域概念之间的继承与分类逻辑，实现知识的系统化组织。

关键活动

- 识别并抽象领域中的核心概念作为“类(Classes)”
- 利用rdfs: **subClassOf**建立类间的层级继承关系
- 定义必要的owl: **equivalentClass**(等价)与owl: **disjointWith**(互斥)关系约束



STEP03 定义数据属性&对象关系

核心目标： 丰富类的语义描述，明确建立不同类之间的关联关系，赋予本体基本的知识骨架。

关键活动：

- 为每个类定义必要的**数据属性(DataTypeProperties)**，描述类实例的特征。
- 定义连接不同类的**对象属性(ObjectProperties)**，构建类间的逻辑桥梁。
- 严格定义属性的**定义域(Domain)**与**值域(Range)**，确保属性应用的准确性。



STEP04 配置各类逻辑约束

核心目标： 通过严谨的逻辑规则限制本体中元素的行为和关系，确保知识表达的一致性和完整性。

关键活动：

- 为属性配置**基数约束(Cardinality)**，明确关联数量限制（如“一个人有一个身份证号”）。
- 定义属性的逻辑特征，如**传递性**、**对称性**、**功能性**、**反身性**等。
- 应用**存在/全称量词(Existential/Universal)**约束定义复杂类条件。
- 声明类之间的**不相交约束(Disjointness)**，避免分类重叠。



STEP05编写OWL公理&SWRL推理规则

核心目标: 赋予本体“思考”的能力，让知识库能自动推导隐含的知识。

关键活动:

- 使用OWL2DL公理，精准表达类、属性间的层级、互斥等逻辑约束。
- 针对复杂的业务逻辑，编写SWRL规则实现自定义推理逻辑。

规则示例:

```
Patient(?p) ^ hasSymptom(?p, :Fever) → suggestsDisease(?p, :Infection)
```



STEP06数据向OWL本体映射

核心目标: 将分散的、异构的现实数据，填充到定义好的本体骨架中，形成实例化知识图谱。

关键活动:

- 结构化数据:** 使用R2RML等映射语言，将关系型数据库(MySQL等)转化为RDF实例。
- 非结构化数据:** 利用NLP信息抽取技术，从文档、文本中提取实体和关系，转化为本体实例。
- 融合:** 将所有实例数据与本体模型结合，生成可推理、可查询的完整图谱。



Semantic Web Ontology-构建步骤7-推理机校验与推理

01.加载本体与数据将OWL本体模型文件和已标注的实例数据批量加载到推理机运行环境中。

02.逻辑一致性检查推理机自动扫描并识别本体定义及数据中潜在的逻辑冲突（如互斥类的重叠）。

03.自动分类与实例化计算类的层级关系，并将所有实例自动归类到最精确的类定义下，完善数据结构。

04.规则推理

基于SWRL规则库，从已知事实推导出新的实体关系与属性知识。

05.查询与应用

使用SPARQL语言对融合了隐式知识的图谱进行复杂查询与应用。



核心目标

消除本体与数据中的逻辑错误，保证知识库的严谨性；并基于既定规则自动挖掘隐含的深层知识，赋予图谱“推理”能力。



最终产出

一个逻辑严密、融合了显式定义与隐式推理结果的完整知识图谱，可直接支撑业务系统进行智能问答、决策支持等应用场景。



场景与本体定义

场景设定

构建包含家庭成员基础关系的本体模型，利用逻辑规则实现“祖父母”等衍生亲属关系的自动计算与推理。

核心逻辑定义

- 核心类: Person(定义所有人)
- 对象属性: hasParent(标注为传递属性TransitiveProperty)
- SWRL推理规则:

```
hasParent(?x,?y) ^ hasParent(?y,?z) → hasGrandparent(?x,?z)
```

数据输入与推理结果

已知事实(实例数据)

- 1.: Alice → hasParent →: Bob
- 2.: Bob → hasParent →: Charlie

推理机自动生成的新知识

- 规则推理: 自动推导出: **Alice: hasGrandparent: Charlie**
- 传递闭包: 利用属性传递性, 推断出: **Alice: hasParent: Charlie**



什么是推理机?

推理机 (Reasoner/InferenceEngine) 是一种专门的软件, 能够加载OWL本体和实例数据, 并根据本体中定义的逻辑公理和规则, 自动执行推理, 发现隐含知识并检查逻辑一致性。

核心功能



分类(Classification)

计算类之间的隐含层次关系



实例化(Realization)

确定实例最具体的所属类别



一致性检查

检测本体逻辑定义是否存在矛盾



查询回答

回答包含推理能力的SPARQL查询

主流推理机

HermiT

一个高效的开源OWL2推理机, 支持所有OWL2标准, 在处理大型本体时表现优异。

Pellet

流行的开源Java推理机, 以其优秀的性能和对SWRL规则的支持而被广泛使用。

FaCT++

快速的开源C++描述逻辑(DL)推理机, 主要用于分类和一致性检查任务。

Stardog

商业图数据库产品, 内置了强大的推理引擎, 支持多种推理规则和查询优化。



Semantic Web Ontology-推理案例2-医疗诊断辅助

场景设定: 构建一个包含疾病、症状与药物关系的医学知识本体，辅助医生进行初步诊断建议生成和临床处方的用药安全实时检查。



核心本体模型定义

本体类(Classes)

Patient,Symptom,Disease,Drug

对象属性(Properties)

hasSymptom,isAssociatedWith
treats,isContraindicatedFor

SWRL推理规则示例:

```
Patient(?p)^hasSymptom(?p, Cough)^hasSymptom(?p, Fever)→suggestsDisease(?p, RespiratoryInfection)
```



智能推理应用场景



01.诊断推荐辅助

当系统识别到患者同时具有“咳嗽”和“发烧”症状时，根据预定义的规则，自动触发推理并向医生推荐“呼吸道感染”作为初步诊断方向，减少漏诊。



02.临床用药安全预警

当医生为“胃溃疡”患者开具“阿司匹林”处方时，系统自动比对本体中的“禁忌症”关系，实时触发红色警告，有效规避医疗差错。



结构化数据映射

数据源

主要包含各类关系型数据库表、CSV文件、Excel表格以及JSON数据等具有明确行列结构的数据形式。

映射过程

直接建立数据元素与本体概念的对应关系：将数据表映射为本体中的“类(Class)”，字段映射为“数据属性(DataProperty)”，表间外键关联映射为“对象属性(ObjectProperty)”。



非结构化数据映射

数据源

包含纯文本文件、PDF文档、网页HTML内容、社交媒体评论等缺乏统一格式和预定义数据模型的海量信息。

映射过程

先利用自然语言处理(NLP)技术，如命名实体识别(NER)提取关键实体，通过关系抽取算法识别实体间联系，再将提取出的知识要素规范化地映射到本体对应的类与属性中。



01 从小规模开始

先构建一个核心的、小范围的本体，验证其价值后再逐步扩展，避免因规模过大而导致的资源浪费和系统复杂度失控。



02 领域专家深度参与

本体的质量高度依赖于领域专家的知识 and 经验。让他们参与本体的定义和验证，能确保概念体系准确反映真实业务逻辑。



03 持续迭代优化

本体构建不是“一劳永逸”的工程，而是一个持续迭代的过程。需根据业务发展和用户反馈，不断完善分类、属性和关系。



04 重视一致性评估

定期使用专业的推理机对本体进行逻辑一致性检查，确保没有逻辑冲突，并评估其在检索、推理任务中的实际表现。



核心概念

- 四大核心要素：**类、属性、约束、规则**构成本体的基石。
- 描述语言：**OWL**是目前最主流的本体标准描述语言。
- 关键技术：**推理机**是赋予本体逻辑推理与智能的核心引擎。



构建流程

- 主流模式：采用“自上而下”的方式，由领域专家主导，确保知识体系的专业性与严谨性。
- 关键步骤：**数据映射**是连接异构现实数据与本体知识库的关键桥梁。

应用价值



知识共享与复用

打破信息孤岛，为不同系统间实现统一的知识交换与共享提供标准化的“通用语言”。



深度逻辑推理

利用推理规则挖掘数据中潜在的隐含知识，实现从“已知”推导出“未知”的智能计算能力。



辅助智能决策

整合并结构化海量信息，为复杂场景下的业务问题提供基于逻辑的事实依据和决策支持。

核心优点



逻辑严密

基于严格的逻辑基础，能够保证知识表达的一致性与准确性，有效规避数据冲突。



强大推理

支持基于规则的复杂逻辑推理，能够从现有的显性知识库中自动推导出隐含知识。



知识共享

提供标准化的知识描述框架，极大地降低了沟通成本，便于跨系统、跨组织的互操作。

面临挑战



构建成本高

本体工程化需要深厚的领域专业知识和知识工程技能，人力与时间成本投入巨大。



维护困难

随着业务场景和数据的演变，本体的持续更新、重构和质量控制是复杂且耗时的工程。



可扩展性差

面对海量、高动态的数据场景，传统本体架构在处理大规模并发和灵活扩展方面存在瓶颈。



常见误区

COMMON MISCONCEPTIONS

打破对Web Ontology的
认知偏差，建立正确理解



误区一：本体就是分类体系

纠正：本体远不止分类这么简单，它还包括属性定义、复杂的实体关系描述、数据约束条件以及业务规则的表达。



误区二：推理机可以解决所有问题

纠正：推理机的推理能力高度依赖于本体模型构建的质量以及推理规则的完备性，它无法凭空产生知识或解决超出逻辑定义范围的问题。



误区三：本体可以完全自动化构建

纠正：目前技术手段下，完全自动化构建高质量、可复用的本体依然非常困难，领域专家的深度参与与知识沉淀依然不可或缺。



知识 · 推理 · 互联

Knowledge · Reasoning · Interconnection

01

本体和数据库有什么区别？

数据库侧重于结构化数据的高效存储与读写查询，解决“数据管理”问题；而本体侧重于对领域知识的概念化、关系化与形式化表达，解决“机器理解”与逻辑推理问题，是语义网的核心。

02

如何选择合适的本体构建工具？

首选**Protégé**（斯坦福大学开源工具），它是本体工程的事实标准，支持多种描述逻辑语言且社区资源丰富；商业级场景可选择**TopBraidComposer**，其优势在于强大的团队协作能力与企业级应用集成支持。

03

本体构建的核心难点在哪里？

主要挑战在于两点：一是领域知识的获取与形式化，即如何将行业专家的隐性经验转化为计算机可理解的显性本体模型；二是本体的长期维护与演化，确保其在知识更新中保持逻辑一致性。

02 Knowledge Graph Schema

实用主义的类型标签系统

知识图谱(Knowledge Graph)是一种用图结构来表示现实世界中实体、概念及其之间关系的语义网络。它通过将信息转化为机器可理解的形式，实现了数据的深度关联和智能推理，是构建人工智能应用的重要基石。



打破数据孤岛

将分散在不同系统、格式中的孤立信息有效连接，消除壁垒，形成统一、全局的知识视图。



实现智能搜索

深度理解用户查询背后的真实意图与上下文，打破关键词匹配限制，提供精准且丰富的搜索结果。



支持智能问答

基于图结构的强大推理能力，能够准确理解复杂逻辑，并回答需要多步关联推理的专业问题。



赋能决策分析

揭示海量数据背后隐藏的深层模式、潜在关联和发展趋势，辅助企业进行科学的战略与业务决策。

本章节将深入探讨知识图谱构建的基石——本体(Ontology)，以及如何从零开始构建一个知识图谱

知识图谱定义

■ 定义

一种以实体、属性、关系为核心的结构化语义网络，用于表示现实世界中的事物及其相互关联。它通过图结构直观展示了事物间的逻辑关系，解决了传统数据库难以处理的非结构化数据关联问题。

■ 核心构成

本质是图结构，由两类基本元素构成：

- **节点(Node)**：代表现实世界中的实体或概念（如人物、公司、地点）
- **边(Edge)**：代表实体之间的特定关系（如“创始人”、“位于”、“关联交易”）

行业背景与价值

■ 起源与背景

知识图谱的概念最早由**Google(谷歌)**于2012年提出并应用，其初衷是为了解决传统搜索引擎“关键词匹配”的局限性，赋予机器理解真实世界事物关联的能力，从而大幅提升搜索结果的准确性和智能化水平。

■ 核心技术价值

它已成为人工智能、大数据分析、智能检索、个性化推荐系统等领域不可或缺的核心底层技术，特别擅长在海量信息中表达复杂的逻辑关联，挖掘深层语义价值，支撑企业实现数据驱动的智能决策。

01/核心定位

定位：为大规模、不完美、且持续动态变化的复杂数据，提供一个逻辑清晰且**可伸缩的“骨架”**，是实现数据关联与价值挖掘的基石。

02/三大设计目标



高效存储与查询

支撑海量数据的稳定存储，支持毫秒级复杂关联查询



赋能业务场景

直接服务于搜索引擎、推荐系统、问答等核心应用



高可扩展性与灵活性：能够快速适应业务需求与底层数据结构的动态变化。

03/典型应用场景



通用搜索引擎

如Google Knowledge Graph，整合海量异构信息，增强搜索结果理解与呈现，提升检索体验。



个性化推荐系统

应用于主流电商与内容平台，基于用户画像与物品属性图谱，实现“人-货-场”的精准匹配。



企业知识管理(KM)

构建统一的企业级知识库，打破信息孤岛，赋能内部智能问答与辅助决策。



属性图(Property Graph)

定义

目前工业界最主流的存储形态，其核心特点是节点和边均可灵活定义属性，能够自然地映射现实世界中的复杂关系。

代表技术

Neo4j, JanusGraph, NebulaGraph, TigerGraph

核心优势

数据表达能力强，图结构与业务逻辑契合度高；原生图存储带来极高的图遍历与查询性能。

RDF图(Resource Description Framework)

定义

W3C推荐的语义网数据模型标准，严格遵循“主语-谓语-宾语”三元组结构来描述实体与关系，强调数据的语义性。

代表技术

Virtuoso, Blazegraph, GraphDB, Stardog

核心优势

标准化程度极高，支持丰富的推理能力，适合跨机构、跨平台的大规模知识融合与共享。



实体(Entity)

■ **定义：** 现实世界中可独立标识的事物，是知识图谱中最基本的构成单元。

■ **示例：** 一个具体的人（如“张艺谋”）、一部电影（如“满江红”）、一家公司（如“字节跳动”）等具体对象。



关系(Relation)

■ **定义：** 实体与实体之间客观存在的关联，将孤立的实体连接成网状的整体。

■ **示例：** “导演”关系连接“张艺谋”与“满江红”；“创始人”关系连接“张一鸣”与“字节跳动”。



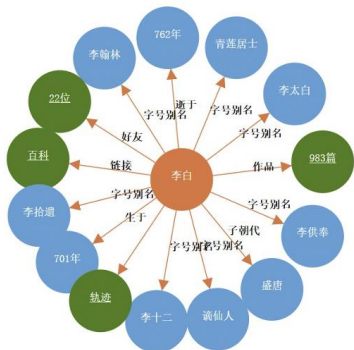
属性(Property)

■ **定义：** 用于描述实体本身的特征、性质或状态的字段，补充实体的具体信息。

■ **示例：** “满江红”的“上映时间”是“2023年1月22日”；“字节跳动”的“成立时间”是“2012年3月”。

极客时间

三元组查询：李白的作品有哪些？



实体识别

李白 的诗歌家喻户晓	
实体标注	
身份证号	68180084774179171
名称	李白
类型	李白
Popular	900
别名	太白、李太白、李供奉、李十二、李太白、李谪仙、李白、李白701~762、李白年谱、李翰林、李青莲、诗仙、诗仙太白、诗仙狂、谪仙人、酒仙、酒仙翁、青莲居士
简介	李白（701年—762年），字太白，号青莲居士，又号“谪仙人”，是唐代伟大的浪漫主义诗人，被后人誉为“诗仙”，与杜甫并称为“李杜”，为了与另两位唐朝诗人（王维与杜甫）区别，姓与李白合称“李杜”，唐《新唐书》记载：李白字太白，自号青莲居士。李白是唐朝诗坛上的一朵奇葩，与杜甫齐名，世称“李杜”。李白是继屈原之后最伟大的浪漫主义诗人之一，有“诗仙”之称。李白诗歌以想象奇特、构思新颖、意境瑰丽著称。李白是继屈原之后最伟大的浪漫主义诗人之一，有“诗仙”之称。李白诗歌以想象奇特、构思新颖、意境瑰丽著称。李白是继屈原之后最伟大的浪漫主义诗人之一，有“诗仙”之称。李白诗歌以想象奇特、构思新颖、意境瑰丽著称。
诗词	将进酒 蜀道难行难于上青天

实体标注

[illegible]

百度一下

更多

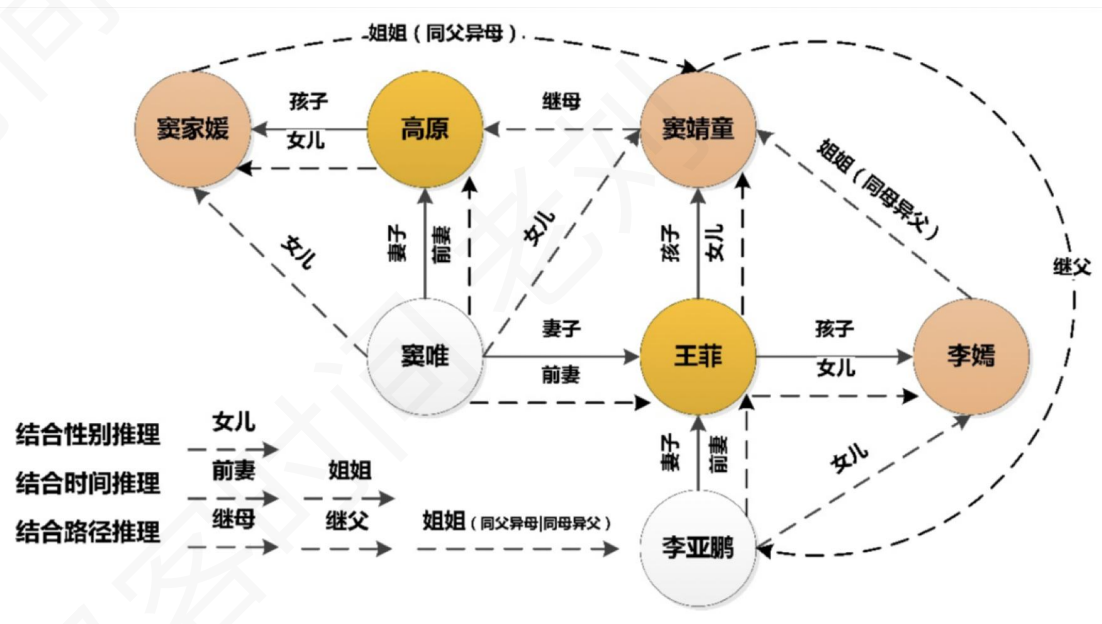
▼ 搜索工具

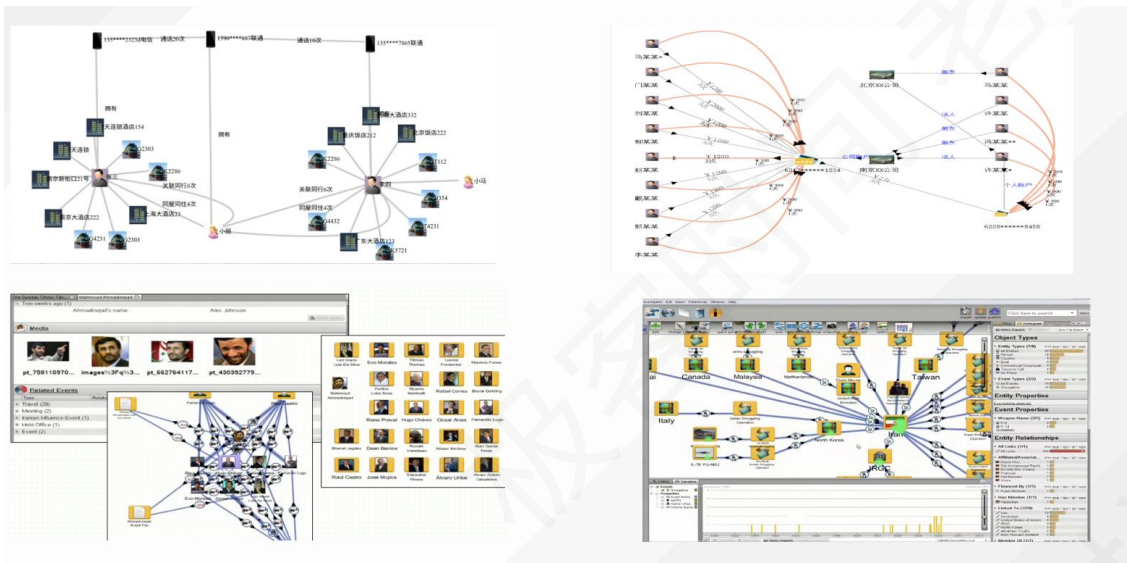


李亚鹏



突唯

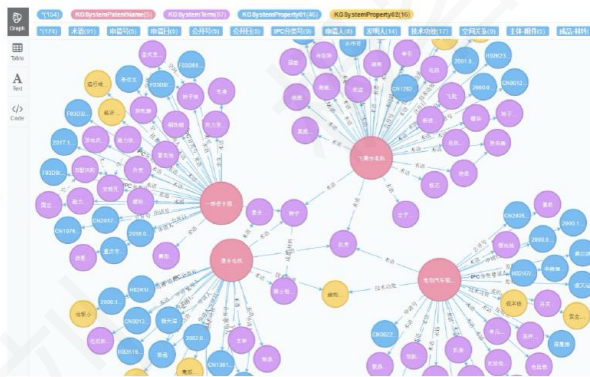




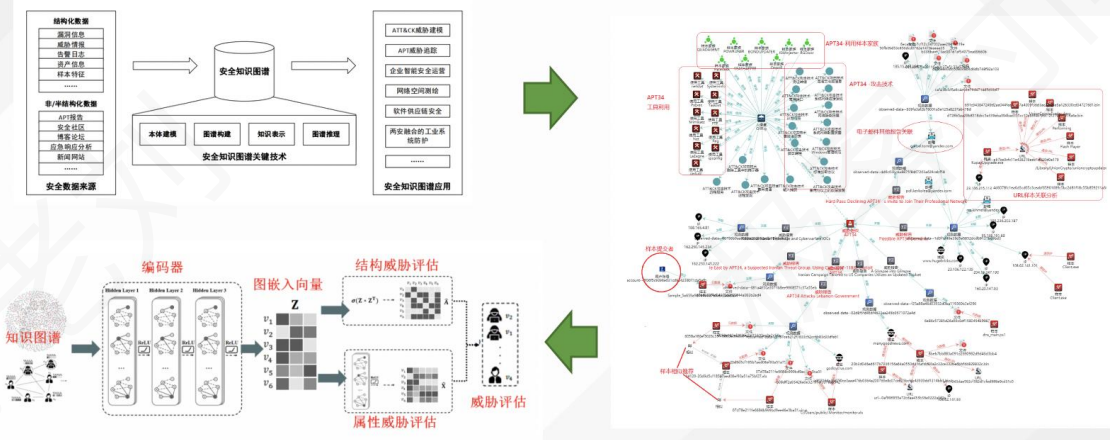
公安知识图谱



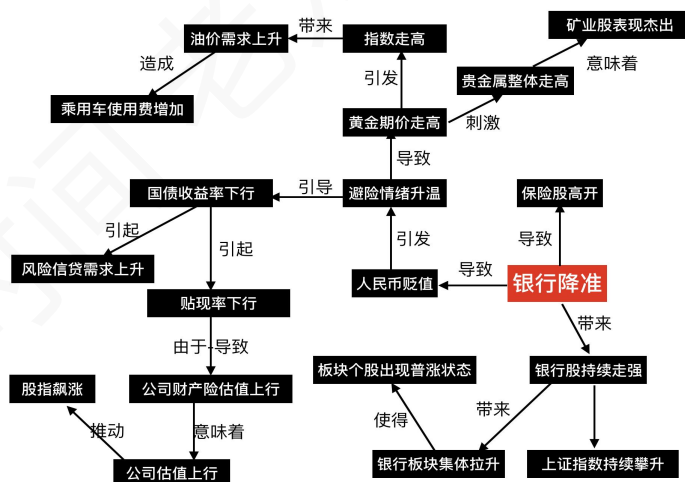
专利知识图谱



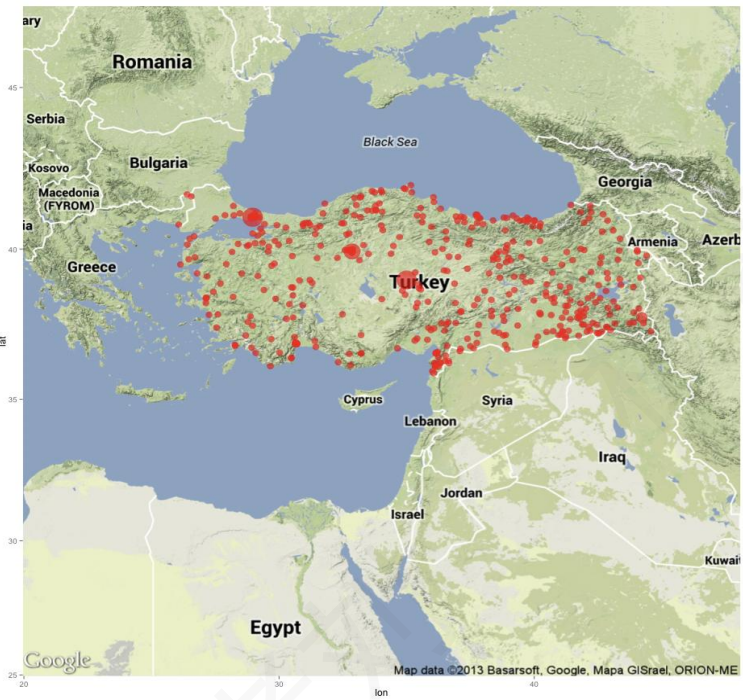
安全领域知识图谱：一个典型的安全知识图谱基于网络和安全知识库、情报库、资产库、行为日志中关键实体（概念）及关系构建的大规模语义网络。



安全知识图谱



因果事件图谱



事件检索与分析



事件演化追踪与监测



知识交互问答



工业界定义

定义：一个实用的、可伸缩的类型标签系统，用于组织大规模、不完美、且处于动态变化中的数据。

特点：轻量级架构·灵活适配业务需求·不追求绝对的逻辑完备性

学术界定义(对比Web Ontology)

定义：强调知识表达的逻辑严密性与完备性，旨在建立精确的概念模型与语义关系，以支持深度推理。

特点：重逻辑体系·强约束条件·专业门槛高且构建成本昂贵

工业界：实用主义导向

侧重于海量数据的有效组织与业务场景的快速落地应用。

学术界：理想主义导向

侧重于理论模型的严谨性、知识推理的准确性与逻辑真理。



知识图谱Schema约束特点：弱约束、非强逻辑完备



弱约束

WeakConstraints

定义：不强制全局的、严格的约束规则，在保证数据可用性的前提下，给予实体属性极大的灵活性。

示例：不强制要求所有“人物”实体都必须填写“出生日期”这一属性，避免因数据不全导致实体无法入库。



允许缺失

MissingValues

定义：实体的属性值可以为“空”，不强求每个字段都必须有数据，适应真实世界中信息的不完备性。

示例：对于一部刚刚上映的“电影”实体，系统允许其暂时不包含“全球总票房”的具体数值。



容忍矛盾

Inconsistencies

定义：允许同一个实体在不同数据源中存在相互冲突的信息，支持后续进行多源数据的融合与验证。

示例：同一历史人物，百科A记录生于“1900年”，百科B记录生于“1901年”，图谱可以同时容纳此类信息。

01 轻量级模式定义

核心定义： Schema由实体类型、关系类型和属性类型三类基础元素构成，形式简单直观，易于理解和维护。



实体类型

人物、电影、公司
(现实世界的事物)



关系类型

导演、出演、创始人
(事物间的关联)



属性类型

出生日期、上映时间
(事物的描述特征)

02 不强制全局约束



允许缺失属性

不强求所有实体都必须具备完整属性。
如：某“人物”实体可以没有“出生日期”信息。



容忍数据矛盾

现实世界信息复杂多变，允许同一实体存在相互冲突的数据，
如：同一人物有不同的出生日期记录。



灵活存储形式

支持多种底层存储与数据模型，既可以是属性图（如Neo4j），也可以是RDF图，适配不同技术栈。



实体类型定义

定义：对具有相同特征的实体的分类，用于区分不同性质的数据对象。

示例：Person, Movie, Company, Location



类型层级

定义：实体类型之间支持结构化的层级继承关系，形成逻辑分类树。

示例：Student(子类型)→Person(父类型)



类型标签

定义：灵活地为每个实体打上一个或多个自定义的类型标签，补充描述实体属性。

核心价值：打破层级限制，极大提升实体数据在海量场景下的组织、查询与分析效率。



01/关系类型定义

定义：对实体之间关联的分类，是构建图数据库模型时，用于明确不同实体在业务逻辑中如何相互作用的核心标签。

典型示例：

directed_by(导演)|acted_in(出演)|founded_by(创立)|located_in(位于)



02/关系方向

定义：关系可分为有向 (Directed) 和无向 (Undirected)，决定了实体间的作用流向。

示例：“导演”关系是明确的有向关系，方向为：导演→电影。



03/关系基数

定义：描述关系两端实体的数量对应关系，用于量化实体间的连接强度。

示例：“导演-电影”是典型的一对多(1: N)关系；“演员-电影”通常为多对多(M: N)关系。

01/属性类型定义

定义：对实体特征的分类，用于描述实体具备的某一项特性或维度，是构建数据模型的基础单元。

常见属性示例

birth_date · release_date · address
price · is_active · user_status

02/数据类型规范

定义：属性值的具体类型约束，用于规范数据的存储格式、验证规则及运算逻辑，确保数据的一致性与可用性。

文本(Text)

名称、描述、地址等
字符信息

数值(Number)

价格、年龄、数量等
可计算数值

日期(Date)

生日、发布时间、
截止日期等

布尔(Boolean)

逻辑判断（是/否）：如是否活跃、
是否上架、是否删除

枚举(Enum)

固定值列表：如状态（活跃/禁用）、
性别（男/女/未知）、订单类型等



知识图谱Schema构建-自底向上自动抽取建模



核心定义

从大规模非结构化数据中自动抽取实体、关系与属性等关键知识，并基于抽取结果逐步归纳、构建知识图谱Schema的主流构建方法。



01 信息抽取

利用NLP技术，从海量文本中精准抽取实体、实体间关系及实体属性等基础事实。



02 实体链接

将抽取出来的实体，与已有的知识库中的标准实体进行对齐与关联，实现跨数据源的统一。



03 Schema归纳

从已链接的海量知识事实中，自动归纳出实体类型体系和关系类型体系，完成Schema的构建。

核心优势



自动化程度高

显著降低人工成本，能快速构建并扩展大规模的知识图谱，适合处理海量数据源。



适应新领域

无需预设Schema，能够从数据中自动发现新的实体类型与关系模式，适应快速变化的新业务场景。



01/核心定义

结合人工定义的高层类别，对自动抽取的结果进行整理和优化。

它是连接底层数据与上层知识体系的关键桥梁，旨在弥补纯自动化处理的局限性，确保结构化数据不仅准确，而且具备领域内的业务价值和逻辑关联。

02/关键作用



提升Schema质量

解决歧义性与不一致性，提升数据准确性



融入领域知识

引入专家经验，增强Schema的业务相关性



构建层级体系

建立清晰的实体层级关系，优化结构

03/实施方法



定义顶层类别

由专家界定核心的顶层实体，如“人物”、“组织”、“地点”等基础类别。



归类抽取结果

将自动识别出的“演员”、“导演”等细粒度实体映射到顶层类别中。



第一步：明确目标，划定范围



01/构建目标是什么？

是实现智能搜索、智能问答，还是辅助决策分析？
明确目标是决定本体知识覆盖深度与广度的基石。



02/应用领域是什么？

是面向通用百科，还是垂直领域（如金融、医疗、影视）？
领域决定了本体中核心概念的选取和专业术语的定义。



03/目标用户是谁？

面向普通大众用户，还是专业领域分析师？
用户画像直接决定了知识表达的复杂度与推理能力需求。



04/范围边界在哪里？

如影视领域：是否需包含演员私生活、电影票房数据？
清晰界定边界，避免范围无限扩大，保障项目可行性。



产出物：详细的需求文档|明确知识图谱的核心目标、应用领域、目标用户群体及范围边界。



01.识别核心类(Classes)

列出领域内所有重要的概念实体，构成图谱的基础骨架。

示例：电影、导演、演员、编剧、制片人、类型、奖项、国家



02.构建分类体系(Taxonomy)

将识别出的类组织成逻辑层级结构，体现概念间的上下位关系。

示例：“导演/演员/编剧”→父类“人物”；“电影/音乐奖项”→父类“奖项”



03.定义核心关系(Relations)

识别类与类之间的语义关联，建立图谱的连接网络。

示例：电影-[执导]->导演|电影-[出演]->演员|演员-[出生于]->国家|电影-[获得]->奖项



04.定义关键属性(Properties)

为每个类定义描述其特征的具体数据属性，丰富实体信息。

示例：电影（上映年份、时长、评分、语言）|人物（出生日期、国籍、性别）



本阶段产出：一份包含领域核心类、层级分类体系、核心关系类型和关键属性定义的本体初稿。



■ 常用本体语言

RDF(资源描述框架): 描述网络资源的标准, 使用“主语-谓语-宾语”三元组表示知识。例如: `<泰坦尼克号><上映年份>"1997"`

OWL(网络本体语言): RDF的扩展, 提供丰富表达力, 用于定义类、属性及复杂约束关系。例如: 定义“导演”是“人物”的子类, “directedBy”是功能性属性。

■ 主流本体构建工具

- **Protégé:** 斯坦福大学开源工具, 功能强大, 是本体工程领域的事实标准。
- **TopBraidComposer:** 企业级的商业化本体开发与知识图谱全生命周期管理平台。
- **Neo4jBloom/GraphStudio:** 支持本体定义与图数据导入的可视化管理工具, 便于业务人员理解。



关键产出物: 机器可读的本体文件

将概念模型转化为标准化的代码文件 (如.owl或.rdf格式), 这是连接业务逻辑与机器处理的桥梁, 为后续知识图谱的数据映射与推理奠定基础。



第四步：验证与迭代

本体构建不是一蹴而就的，它是一个持续优化的闭环过程，需要经过反复的评估、验证与迭代，以确保最终模型的准确性与可用性。



一致性检查

使用推理机(Reasoner)检测逻辑矛盾，例如实例是否同时属于不相交类、是否违反基数约束等。



完整性评估

验证本体是否完整覆盖了需求分析中定义的所有核心概念、关系和属性，避免关键信息的遗漏。



可扩展性评估

评估架构设计是否灵活，能否方便地扩展新的类、属性或关系，以适应业务和知识范围的增长。



领域专家评审

邀请行业资深专家进行专业评审，确保本体模型准确反映领域内的共识知识与业务逻辑规则。



真实场景反馈

将本体模型投入实际应用场景中，收集用户在数据加载、查询推理中的反馈，进行针对性的优化。

迭代循环：评估→发现问题→修改本体→重新评估|最终产出：稳定、高质量的本体模型(KG应用基石)



01 数据准备

收集和清洗结构化、半结构化、非结构化数据，为后续处理打好基础。



02 信息抽取

利用自然语言处理（NLP）技术，从海量文本中精准抽取实体、关系与属性。



03 实体链接

将抽取的实体对象，准确匹配并链接到现有的知识库体系中。



04 Schema归纳

从已抽取的碎片化知识中，归纳并定义标准化的实体类型与关系类型。



05 数据融合

整合多源异构数据，解决不同来源实体间的冲突与属性冗余问题。



06 存储

将融合后的结构化知识数据，统一持久化存储到图数据库中，支撑应用查询。



知识图谱构建-结构化数据接入-数据映射

01

数据源选择

根据业务需求，选定结构化数据源，如关系型数据库表、CSV/Excel文件等。

02

实体抽取

定义业务实体类型，将源表中的每一行记录转换为图谱中的一个节点。

03

属性映射

将源表的字段映射为实体节点的属性，并定义数据类型，保留关键业务信息。

04

关系抽取

识别源表中的外键关联或业务逻辑，将其转化为实体间的有向边关系。

05

数据加载

将完成映射的实体与关系数据，批量导入到图数据库中，完成图谱构建。

映射示例Example

源数据表(SourceTable)
``employees` (id,name,department_id)`

知识图谱实体(KGEntity)
Type: **Employee**(id,name)

图谱关系(KGRelation)
`Employee→works_in→Department`



知识图谱构建-非结构化数据抽取与图谱关联

01

文本预处理清洗、分词、标注，为后续NLP处理准备高质量文本。

02

实体抽取(NER)运用命名实体识别技术，精准定位并抽取文本中的关键实体。

03

关系抽取(RE)识别并抽取实体之间存在的语义关系，构建基础的三元组。

04实体链接

将抽取的实体映射到通用或行业知识库中的标准实体，实现归一化。

05数据加载

将抽取和链接后的结构化知识批量加载至图数据库，形成可视化图谱。

核心技术支撑体系



NLP自然语言处理技术

包含命名实体识别(NER)、关系抽取(RE)及实体链接三大核心模块，是实现非结构化文本向结构化知识转化的基石。



前沿深度学习模型

引入BERT、GPT等预训练大模型，显著提升实体抽取的准确率与召回率，确保知识图谱的构建质量与效率。

结构化数据

- **关系型数据库**: MySQL、PostgreSQL 等表数据
- **文件格式**: CSV/Excel表格、JSON/XML数据

特点: 数据结构清晰、标准, 易于直接解析与映射。

半结构化数据

- **Web资源**: HTML网页 (包含标签与自由文本)
- **链接数据**: JSON-LD (含上下文信息的JSON)

特点: 具备一定结构特征, 但缺乏统一标准, 需额外清洗与解析。

非结构化数据

- **文本内容**: 新闻文章、百科词条、PDF文档等
- **多媒体**: 图片、音频、视频中的隐含信息

特点: 无固定格式, 信息密度高但隐蔽, 依赖NLP/AI技术提取知识。



核心挑战: 如何将上述不同来源、不同格式的数据, 依据定义好的本体规范, 准确、高效地清洗并映射到知识图谱中。



处理结构化数据

01.数据提取

利用ETL工具（如Talend,NiFi）或Python脚本，从关系型数据库、CSV/Excel文件中提取源数据。

02.数据清洗

处理缺失值、去除重复记录、统一数据格式。

例：将“上映年份”字段统一清洗为标准的“YYYY”四位数字格式。

03.数据对齐

对多源异构数据中的同名字段进行合并和标准化。

例：将不同表中的“导演”和“Director”字段统一为“导演”。



映射到本体模型

实例映射(InstanceMapping)

数据表中的每一行记录对应知识图谱中的一个实体实例。

例：电影表的一行记录→KG中的一个“电影”类实例。

属性映射(PropertyMapping)

数据表中的每一列对应实例的一个数据属性或对象属性。

例：“上映年份”列的值“1997”→实例的`hasReleaseYear`属性值。

关系映射(RelationMapping)

通过外键或关联字段建立不同实体间的关系。

例：电影表“导演ID”→关联到“导演”实例→建立`directedBy`关系。



核心处理流程：NLP驱动的信息挖掘

1. 文本预处理(TextPreprocessing)

包含分词、词性标注及关键的命名实体识别(NER)。例：识别“詹姆斯·卡梅隆(人物)执导了1997年(时间)上映的电影《泰坦尼克号》(作品)”中的实体并分类。

2. 关系抽取(RE)

识别实体间的语义关联。
例：建立“詹姆斯·卡梅隆”与“泰坦尼克号”之间的“执导”关系。这是构建图谱边的关键。

3. 事件抽取(EE)

识别文本中的动态事件及核心要素。
例如：提取“电影上映”这一事件，确定时间地点和参与者等多维属性。

常用工具与技术栈

成熟NLP开发库



NLTK, spaCy, HanLP等基础库，提供开箱即用的分词、NER接口，快速实现原型开发。

预训练深度学习模型



BERT、GPT等Transformer架构模型，通过微调在复杂场景的实体与关系抽取任务中达到SOTA效果。

自动化抽取工具



OpenIE, ReVerb等开放信息抽取工具，无需预定义关系类型，适合从海量文本中发现新的关系。



实体链接(Entity Linking)

将文本中识别出的实体提及 (Mention) 链接到知识图谱中对应的实例，建立文本与知识库的关联。

核心挑战： 同一个实体可能有多种表述 (同义)，同一表述也可能指向多个实体 (歧义)。

●例1(同义): “卡梅隆”、“JamesCameron”均指导演。●例2(歧义): “苹果”可指水果或科技公司。

实体消歧(Entity Disambiguation)

解决同名歧义问题，确定实体在特定上下文的确切所指。其核心流程包含三步：

- 1.候选实体生成：检索所有可能的实体 (如“苹果”→水果、公司)。
- 2.相似度计算：将上下文与候选实体的描述、属性进行比对。
- 3.决策：选择相似度最高的候选实体作为最终结果。



核心价值： 消除非结构化数据的模糊性，确保提取的信息能准确融入知识图谱，避免产生错误的关联，是构建高质量知识图谱不可或缺的“纠错与校准”环节。

数据融合(DataFusion)

将来自不同异构数据源的信息整合为一个统一视图，解决实体与属性层面的冲突和不一致性。

01.实例融合(InstanceFusion)

识别并合并指向同一现实实体的不同数据记录。例如，将“<泰坦尼克号_豆瓣>”与“<Titanic_维基>”合并为一个统一电影实体。
方法：基于名称/上映年份的属性相似度匹配、实体链接技术。

02.属性融合(AttributeFusion)

解决同一实体在不同来源中属性值的冲突（如评分9.4vs9.5）。
策略：信任度加权平均、多数投票算法、优先选择最新数据。

知识图谱存储(KGStorage)

将清洗、融合与映射后的数据持久化存储，以支持高效的查询、推理与图分析能力。

主流图数据库选型：

- Neo4j: 最流行的原生图数据库，社区生态活跃，工具链丰富，适合中小规模图谱。
- JanusGraph: 开源分布式图数据库，支持PB级数据存储，适合构建超大规模企业级知识图谱。
- Amazon Neptune: AWS提供的全托管式图数据库服务，高可用且免维护。

核心动作：批量导入本体定义与三元组数据(Subject-Predicate-Object)

01/知识图谱质量评估



完整性(Completeness)

是否覆盖了预期的知识范围？是否存在大量缺失的实体或关系？



准确性(Accuracy)

实体和关系的描述是否正确？是否存在错误的事实？

一致性(Consistency)

是否存在逻辑矛盾？如一个电影同时上映于不同年份。

时效性(Timeliness)

知识是否最新？如演员的最新作品、电影的最新评分。

02/持续迭代与优化



数据补全与扩充

针对缺失的关键信息，利用众包协作、全网爬虫或引入新的权威数据源进行补充。



错误修正与清洗

通过人工审核与自动化规则相结合的方式，识别并修正图谱中的错误事实与冲突数据。



模型与算法优化

若NLP提取准确率未达标，需优化NER/RE模型参数，或调整抽取与融合规则。

知识图谱的构建是一个持续迭代的过程，需要不断地维护和更新，以保证知识的质量和价值。



场景设定 我们将构建一个小型的电影知识图谱，打通电影、人物、类型三者之间的数据孤岛，最终实现对“导演是谁”、“演员演过什么”、“电影评分”等复杂问题的智能语义查询。

STEP01 - 需求分析与本体设计

核心查询目标：支持查询“某部电影的导演是谁？”、“某个演员出演了哪些电影？”、“某部电影的评分是多少？”等问答。

定义核心类(Classes)：电影(Movie)、人物(Person)、类型(Genre)。

构建核心关系(Relationships)：directedBy(电影→人物),actedIn(人物→电影),hasGenre(电影→类型)。

补充核心属性(Properties)：电影(上映年份、评分、标题)，人物(姓名、出生年份)。

STEP02 - 数据准备(DataPreparation)

结构化数据(CSV)：使用标准的`movies.csv`文件作为主要结构化数据源。

```
id,title,release_year,rating,director,actors,genres
1,Titanic,1997,9.4,"JamesCameron","Leo;Kate","Drama;Romance"
2,Inception,2010,9.4,"C.Nolan","Leo;Joseph","Sci-Fi;Action"
```

非结构化数据：引入维基百科关于《泰坦尼克号》的摘要文本，用于丰富图谱内容。



处理结构化数据(movies.csv)

01.解析与实例化: 读取每行数据, 为电影、导演/演员、类型分别创建`Movie`,`Person`,`Genre`类型的实例ID, 例如:

<http://example.org/movie/1>

02.属性填充: 为每个电影实体添加基础属性信息, 建立三元组:

: movie1 <title> "Titanic" | <releaseYear> "1997"

03.关系构建: 连接不同类型的实体, 定义图谱中的关系边:

: movie1 <directedBy>: James_Cameron
: Leonardo_DiCaprio <actedIn>: movie1
: movie1 <hasGenre>: Drama



处理非结构化数据(维基百科文本)

信息提取(NER): 利用命名实体识别技术, 从纯文本中识别并提取出关键实体, 如“泰坦尼克号”、“詹姆斯·卡梅隆”等。

实体链接与关系抽取: 将文本中识别出的实体映射到图谱中已有的标准化实例上; 同时识别“执导”、“出演”等隐含的实体关系。

知识融合与补充: 提取并整合结构化数据中缺失的信息 (如“荣获11项奥斯卡奖”、“票房纪录”等), 极大地丰富知识图谱的属性维度。



知识图谱构建案例-构建一个电影知识图谱(3/3)

STEP04 - 存储与查询：从数据到洞察



将清洗与映射后的结构化三元组数据批量导入**Neo4j**图数据库中，以图结构存储实体与关系，为后续的复杂关联查询打下基础。

谁导演了《泰坦尼克号》？

```
MATCH(m: Movie{title: 'Titanic'})<-[: directedBy]-(p: Person)
RETURN p.name
```

结果: **JamesCameron**(詹姆斯·卡梅隆)

莱昂纳多出演了哪些电影？

```
MATCH(p: Person{name: 'LeonardoDiCaprio'})-[: actedIn]->(m: Movie)
RETURN m.title,m.releaseYear
```

结果: **Titanic(1997),Inception(2010)**等

所有科幻电影的平均评分？

```
MATCH(m: Movie)-[: hasGenre]->(g: Genre{name: 'Sci-Fi'})
RETURN avg(m.rating) AS average_rating
```

结果: **9.4/10.0**(基于样本数据计算)



总结：通过清晰的本体构建与精准的数据映射，我们成功将非结构化文本转化为可推理、可关联查询的电影知识图谱，具备了回答复杂跨实体问题的能力。



01/推理定义

基于已有的知识推导出新的知识

与传统Web Ontology推理不同，知识图谱推理更侧重通过数据驱动的方式，利用图结构和统计规律，从海量数据中自动挖掘实体间潜在的逻辑关系，从而不断丰富图谱内容，提升图谱的完整性和应用价值。

02/核心推理方式

统计推理

基于统计规律进行归纳，不保证逻辑上的绝对必然性。

近似推理

采用近似算法，在保证一定准确度的前提下追求推理效率。

03/常用技术方法

路径排序算法
(PRA)

知识图谱嵌入
(KGE)

简单传递规则
(Transitivity)

核心原理 · Principle

定义：一种基于图结构的推理方法，通过统计实体间的连通路径来预测它们之间潜在的语义或逻辑关系。

示例：若实体A与实体C存在“合作”关系，且实体B与实体C也存在“合作”关系，则可推断A与B之间也可能存在“合作”关系。

显著优势 · Advantages

●**直观易懂：**逻辑符合人类直觉，易于理解。●**可解释性强：**推理路径清晰，结果可追溯。

核心应用 · Applications



关系预测(RelationPrediction)

针对已知的两个实体，预测它们之间是否存在某种特定关系，常用于知识图谱补全任务。



链接预测(LinkPrediction)

在大规模图数据中，预测实体对之间可能缺失的连接边，以完善网络结构并发现潜在关联。



知识图谱推理实现-基于嵌入TransE统计推理

核心原理

定义： 将知识图谱中的实体和关系映射到连续的低维向量空间，通过简单的向量算术运算来进行逻辑推理。

核心假设： 若三元组(h,r,t)成立，则向量满足 $\mathbf{h} + \mathbf{r} \approx \mathbf{t}$ (h: 头实体, r: 关系, t: 尾实体)。

主要优势



推理效率高

向量运算速度快，适合实时查询与推理场景。



可扩展性强

模型简单，易于训练，能有效处理大规模数据。

经典代表模型

TransE - 翻译模型

最基础、最经典的模型。将关系视为从头实体到尾实体的“向量翻译”，结构简单且效果出色。

TransR - 空间映射模型

针对TransE改进，考虑实体与关系可能属于不同语义空间，引入投影矩阵实现空间映射，提升准确性。

DistMult - 矩阵乘法模型

将关系表示为矩阵，通过“头实体向量 $\times$ 关系矩阵 $\times$ 尾实体向量”的内积运算来建模，计算效率极高。

规则定义(RuleDefinition)

设定基础逻辑规则，作为推理的判断依据。例如：

“如果A是B的父亲，且B是C的父亲，那么A是C的祖父”



补全缺失关系

基于现有数据推导新的事实，自动补全图谱中缺失的连接，提升数据完整性与密度。

自动化推理过程(ReasoningProcess)

01

全网查找

检索满足“父亲”关系的所有三元组

02

路径匹配

寻找 $A \rightarrow B$ 且 $B \rightarrow C$ 的连续路径

03

逻辑推断

自动建立 $A \rightarrow C$ 的“祖父”关系



发现隐含知识

揭示实体间深层的间接关联，挖掘未被显性记录的业务逻辑与价值信息。

示例逻辑：父亲(A,B)+父亲(B,C)→祖父(A,C)

01

本体是基础

本体是知识图谱的“骨架”与“蓝图”，定义了知识的结构和规则，是构建高质量图谱的基石。

02

流程化构建

遵循“需求分析→概念定义→形式化表示→评估迭代”的科学流程，确保本体构建的系统性与完整性。

03

数据多样性

针对结构化、半结构化和非结构化数据，需采用差异化策略进行融合与处理，以最大化利用数据价值。

04

NLP是核心

NER、关系抽取、实体链接等自然语言处理技术，是从海量非结构化文本中提取关键信息的核心手段。

05

持续迭代

构建不是“一锤子买卖”，需持续进行数据更新、质量评估与模型优化，以适应业务变化。



未来展望

FUTURE OUTLOOK

随着大语言模型(LLM)的飞速发展，其强大的推理与生成能力正在重塑知识图谱构建范式。利用LLM辅助完成本体设计、自动化知识提取与补全，将大幅提升构建效率与智能化水平。未来，知识图谱提供的精确、结构化知识与大模型的泛化、生成能力相结合，将是实现可解释、可信的通用人工智能(AGI)的关键路径。

03 Palantir & Palantir Foundry

从语义到行动的完整闭环

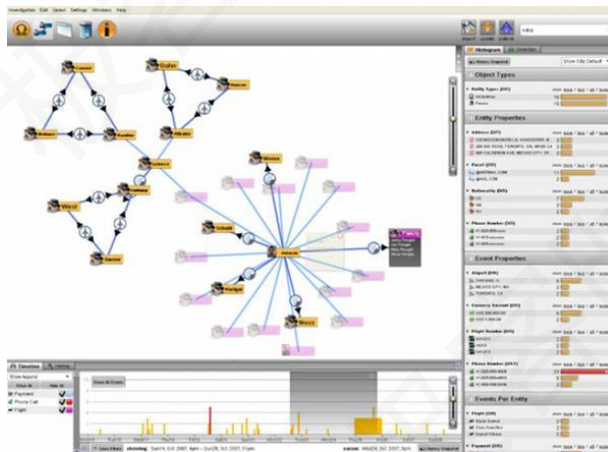
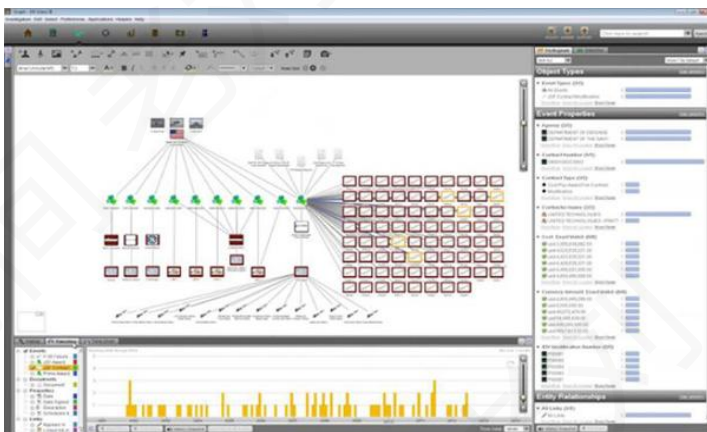
► Palantir认识

► 介绍:硅谷-软件公司, 应用大数据、AI等技术服务于客户的**海量数据处理、业务逻辑建模、执行操作流程**等业务需求。名称来源于《指环王》里的真知晶球 (**Palantir**), 是用于通讯和观察世界的魔法水晶球。成立于2003年, 起初致力于**反恐软件**的研发, 并于2005年成为美国中情局 (CIA) 的软件供应商。

► 定位:专注于为**处理数据**创造世界上最好的用户体验, 使人们能够提出和回答复杂的问题, 而不需要掌握查询语言、统计建模或命令行。为了实现这一目标, 构建了**用于集成、管理和保护数据**的平台, 并在其上构建了**用于完全交互式的人工驱动、机器辅助分析**的应用程序。

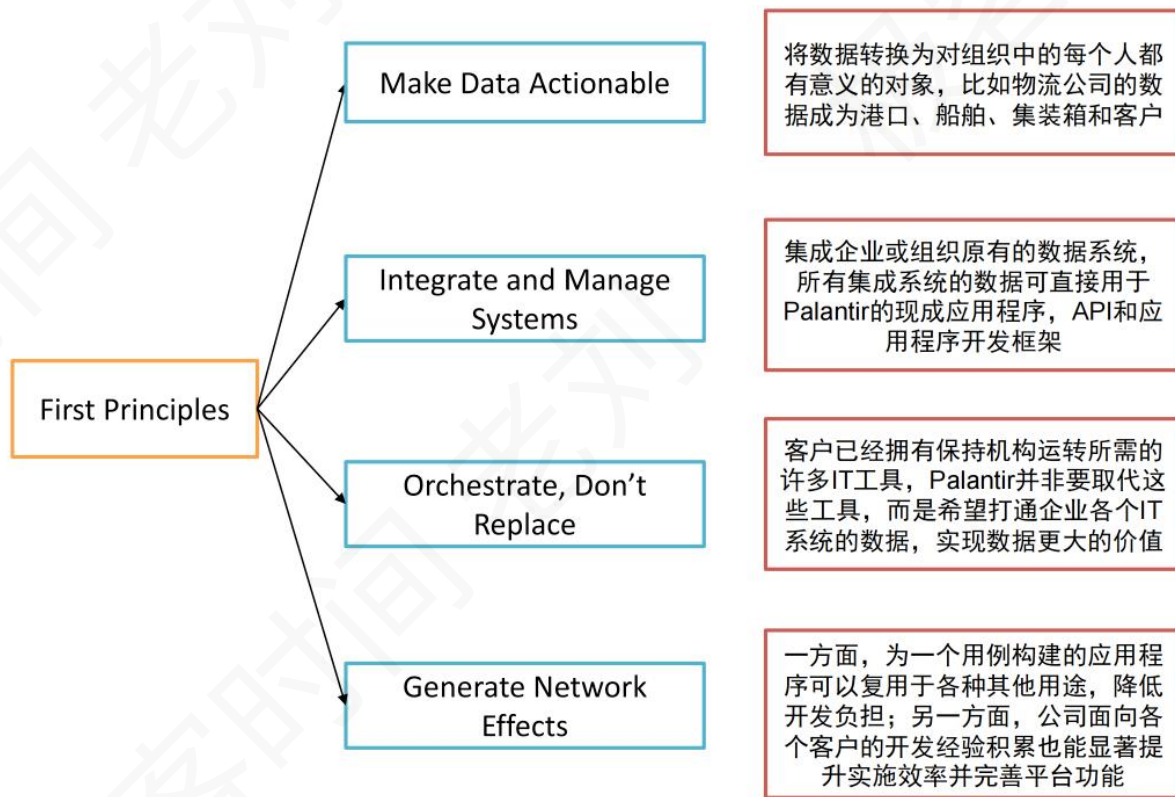
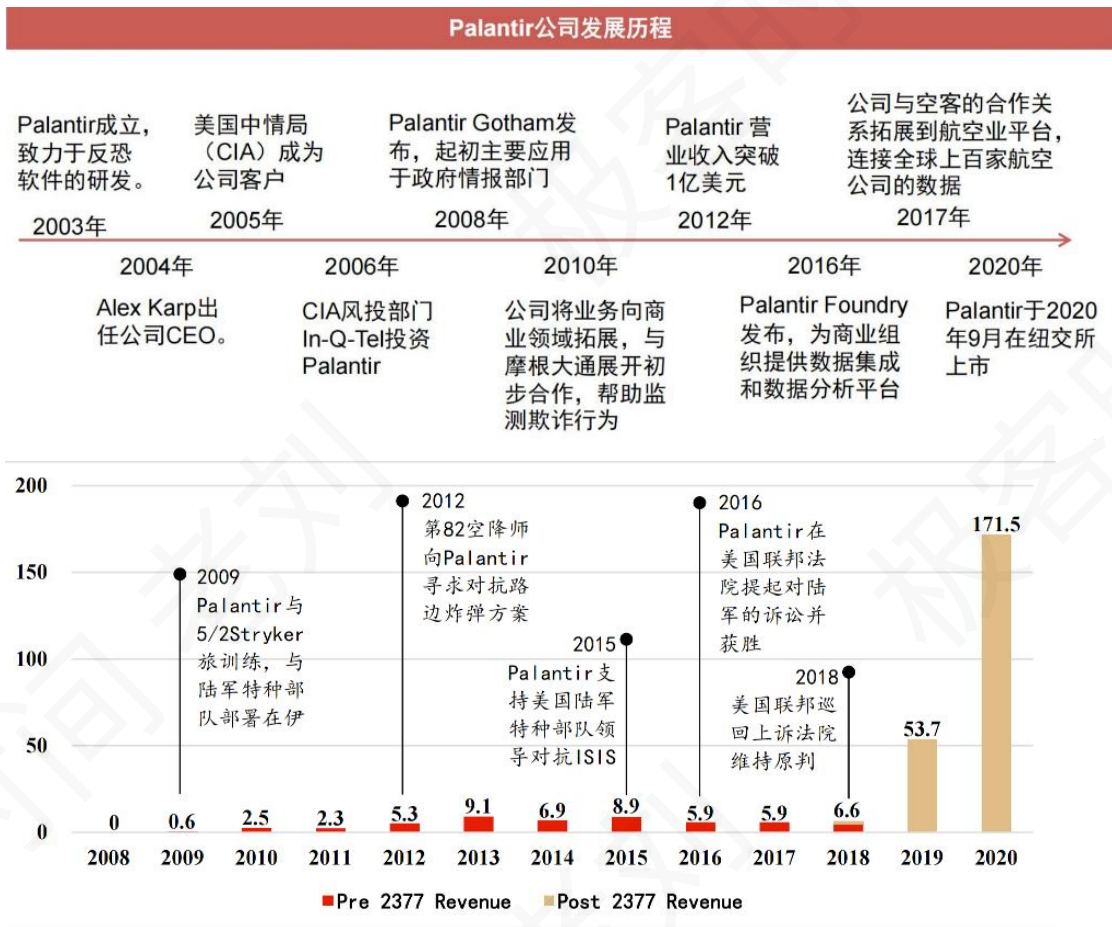
► 典型案例:

- 赶在阿富汗的简易爆炸装置引爆前成功预言了它们的位置;
- 帮助J.P.Morgan对付欺诈犯
- 协助捕获了奥萨马·本·拉登;
- 帮助多家银行追回了前纳斯达克主席Bernie Madoff所隐藏起来的数十亿美元巨款。



➤ Palantir发展历程

- 2008年，公司针对情报领域的客户发布了首个平台产品**Palantir Gotham**，帮助客户进行情报数据的集成、处理和分析，而后Gotham的客群逐步拓展至**国防机构**，乃至美国及其盟国的政府机构。
- 2016年，**Palantir Foundry**平台发布，与各个行业的头部公司合作，帮助各公司将自身数据转化为独特的竞争优势。



资料来源：Palantir招股书，中信证券研究部

- Palantir核心产品包括四大平台，应用于**政府、商业**的多个行业，开发了Apollo、Gotham、Foundry和AIP四大平台（Foundry/Gotham + Apollo + AIP）形成"数据→分析→决策→执行"的完整技术闭环；
- Gotham:用于生成全球决策的操作系统，服务于政府和国防；
 - Foundry:基于本体论的现代企业操作系统，在复杂环境中协调和自动化决策；
 - Apollo:底层技术平台，在**异构环境中部署、监控、修复和保护软件**，提供跨环境的持续交付；
 - AIP:在应用中使用AI实现**智能体**和自动化，如AI查看警报并自动提出解决方案、自定义AI工具，提供跨平台的AI能力

Palantir产品的竞争优势	
竞争优势	具体介绍
平台内置隐私控制	在平台中构建了用于数据保护和治理的关键功能
软件工程师始终在第一线 公司前线部署工程师已前往阿富汗的基地和中西部工业区的工厂进行产品部署	
政府级的安全性	公司的软件必须足够安全以处理国家机密，必须足够稳定以支持士兵的战时决定，并且必须足够透明以确保可以监视和跟踪系统中的所有数据和使用情况。公司为企业用户亦提供了政府质量的安全标准。
支持各种部署方式	公司产品支持公有云、私有云以及本地部署，并且支持多云部署
平台易拓展	公司产品支持跨部门扩展，帮助组织内部的各个决策者进行相互关联的决策
合作关系长久稳固	截至2019年，公司Top 20的客户平均已经与公司合作6.6年，且不断贡献更多价值

时间	应用领域	客户	金额	项目
2021年1月13日	国防	美国陆军	约2.5亿美元	交付陆军地面站现代化第一阶段的原型，支持陆军的战术情报目标接入节点（TITAN）计划
2021年1月19日	能源	太平洋天然气和电力公司（PG&E）	数百万美元	为期数年的合同，提供技术帮助简化数据管理，增强电力系统安全性和电网可靠性
2021年1月28日	采矿	力拓集团	/	多年企业合作协议，整合数据支持关键采矿作业决策
2021年2月23日	供应链	3M	数百万美元	扩大合作，支持 3M 数字化转型，建立动态供应链
2021年6月18日	航空	美国联邦航空管理局（FAA）	最高1840万美元	提供数据分析工具，支持 FAA 飞机认证和运营安全活动
2022年9月20日	造船	现代重工集团	4500万美元以上	扩大合作并扩展到造船领域
2022年9月26日	执法	国土安全调查局	5年9590万美元	续签案件管理软件合作伙伴关系，支持对人口贩运等犯罪调查
2022年10月25日	食品	美国食品药品监督管理局（FDA）	2200万美元	通过 21 FORWARD 计划帮助 FDA 实现食品供应链和弹性方法现代化
2022年12月7日	医疗	美国疾病控制与预防中心	5年4.43亿美元	合作提供“共同操作图”促进公共卫生准备
2023年5月23日	零售	C&A	/	开发人工智能模型优化产品采购和库存补货过程
2023年10月4日	金融	普华永道	/	合作加速运营转型
2024年6月20日	医疗	美国卫生高级研究计划局（ARPA - H）	2年1900 万	合作利用人工智能/人工智能和数据软件工具加速健康成果
2024年12月18日	国防	美国陆军	上限6.19亿美元	扩展与陆军 Vantage 的合作伙伴关系，支持“陆军数据平台”（ADP）

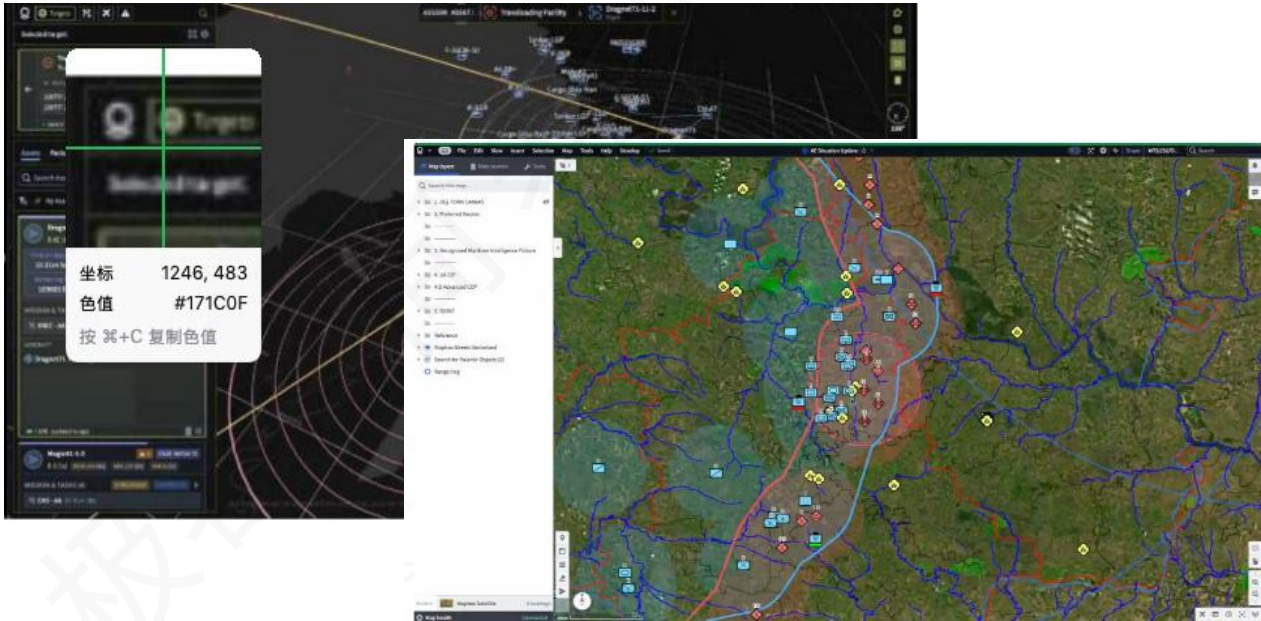
资料来源:Palantir官网，中邮证券研究所，不同应用领域的代表性订单

➤ Palantir明星产品1-Gotham政府机构数据平台

- 名字源于《蝙蝠侠》中的**Gotham市**，寓意在混乱数据中识别犯罪和威胁。将**不同规模、格式的数据集成至统一的数据平台**，并将其映射到已定义的不同数据对象（人、地点、事物和事件等）以及将他们连接起来的关系中。用于生成全球决策的操作系统，服务于政府和国防，识别出隐藏在从信号情报源到机密线人报告等数据集深处的关键信息。
- 核心逻辑是“找点”，解决的是**如何在海量非结构化数据（邮件、通话记录、卫星图像、线人报告）中，通过关联分析发现隐藏的关系网**，侧重于“寻找敌人/目标”（**线索追踪**）。
- 采用O-R-E 模型（Objects 对象, Raw data 原始数据, Events 事件），更像是一个高性能的图数据库（Graph Database）应用，强调对象之间的多维联系。
- 应用场景：**反恐侦察、战场态势感知、犯罪关联分析、人口贩卖追踪**。

Palantir Gotham平台主要应用	
应用程序	主要功能
Graph	通过彼此相连的网络节点，用户可以直观地浏览数据对象之间的关系。借助时间线、直方图等，用户可以可视化事件序列，过滤具有类似特征的对象，分析数据网络中的信息流
Map	Map可以实现地理空间分析功能。在地图中，用户可以导入并集成图像和图层，跟踪特定位置的对象和事件，并支持创建热力图，以按位置标识对象的密度。
Object Explorer	使用对象资源管理器，分析人员可以过滤数十亿个数据对象以得到所需要的子集，以便使用其他应用进行后续分析。Object Explorer由Horizon内存数据库提供支持。
Browser	帮助用户搜索对于他们最有价值的数据，并集成分析性功能
Palantir Gotham平台其他功能模块	
功能模块	主要功能
Ava	平台AI模块，可以自动执行机构已知的复杂且重复的工作流程，减少用户数据筛选等繁杂工作
Table	通过Table，任何用户都可以对Palantir Gotham生态系统中的大规模数据进行高性能搜索，通过复杂的方式过滤数据并得到有价值的信息
Dossier	Dossier使用户可以跨团队和组织进行实时协作，以在Palantir Gotham生态系统内创建实时和动态情报产品，并支持共享分析
Operations	主要用于现场操作过程中的协同工作，支持实时上传、实时跟踪、实时协调等功能
Nexus Peering	使组织能够在全局范围内同步数据，保证跨站点共享数据的一致性，完整性和安全性

资料来源：Palantir招股书，中信证券研究部

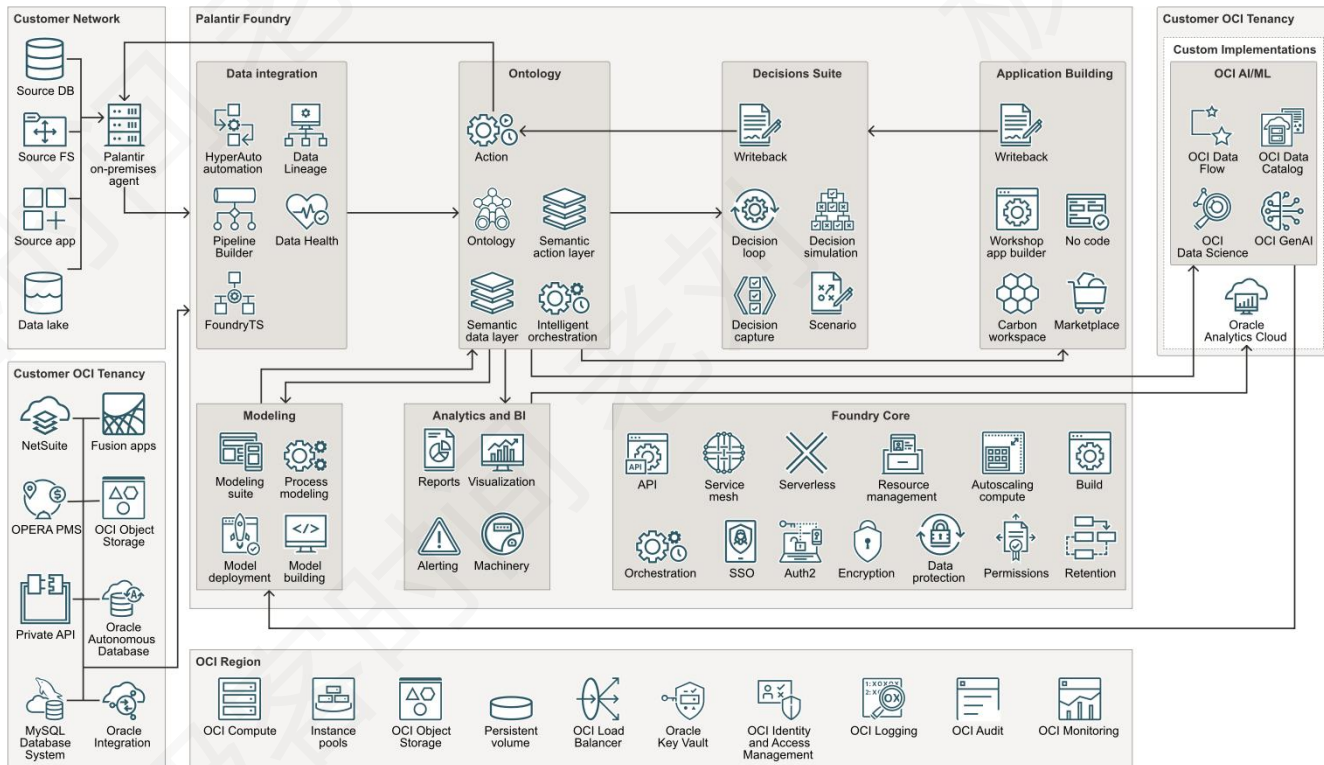


Palantir明星产品2-Foundry商业企业数据平台

- 名字寓意“铸造厂”，旨在将原始杂乱的数据打造成标准化的企业资产，用于数据整合与业务决策，基于本体论（Ontology）的数据操作系统。侧重于“优化业务/资源”（流程集成）。
- 核心逻辑是“建模”，解决的是企业组织内部“数据烟囱”的问题，将复杂的业务流程数字化，采用面向对象的集成模型（Object-oriented Integration），更接近于“数据中台”或 Data Lakehouse，强调数据的血缘（Lineage）和治理。
- 应用场景:供应链优化、飞机维护预测、药物研发、合规与反洗钱。

Palantir Foundry平台主要应用	
应用程序	主要功能
Monocle	Monocle使用户能够使用图形界面访问和管理数据链条，比如访问和管理产业链上下游的各种数据
Contour	Contour支持自上而下对大规模数据进行研究，用户可以在Foundry中过滤、添加以及可视化数据集
Object Explorer	允许用户与表示为对象（如客户、设备或工厂）的数据进行交互，支持搜索索引数据并遍历对象之间的连接
Fusion	Foundry的电子表格环境，用户可以创建单元格引用和函数，以创建新的数据集或报表
Workshop	帮助用户在低代码或无代码的环境中构建应用程序，可以动态地构建交互式工作流
Vertex	通过Vertex，用户可以模拟变化并执行假设性分析，帮助机构优化运营
Code Authoring	Code Authoring是为数据工程师设计的一套应用，便于数据工程师在平台上进行开发
Quiver	Quiver是一个多维图表应用程序，用于分析非常大的时间序列数据集，例如来自机器传感器的流式数据
AI/ML	Foundry将AI/ML和统计模型与Foundry平台的核心组件集成，用户可以进行调用以实现各种功能
Code Workbooks	每个计算步骤都可以保存为Foundry数据集，可供其他应用程序使用以及跨工作簿重用
Reports	Reports允许用户动态发布并实时更新工作及数据

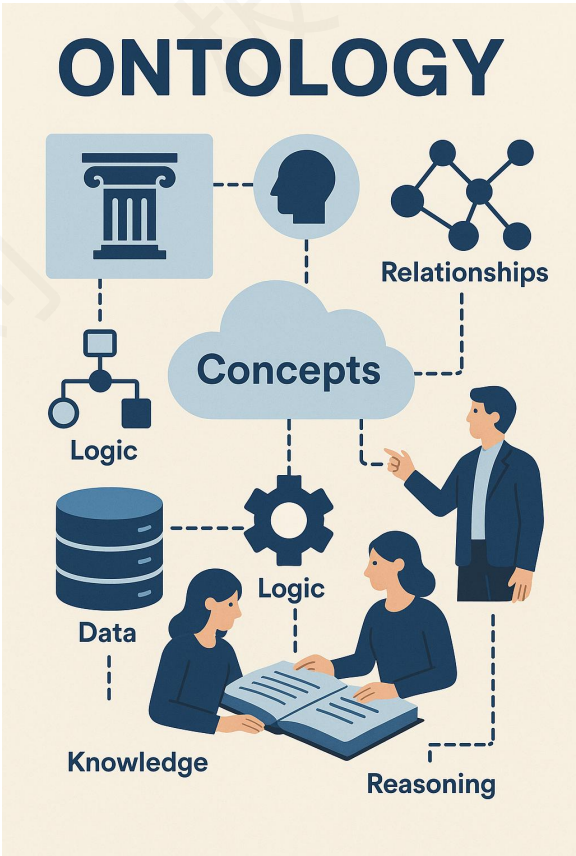
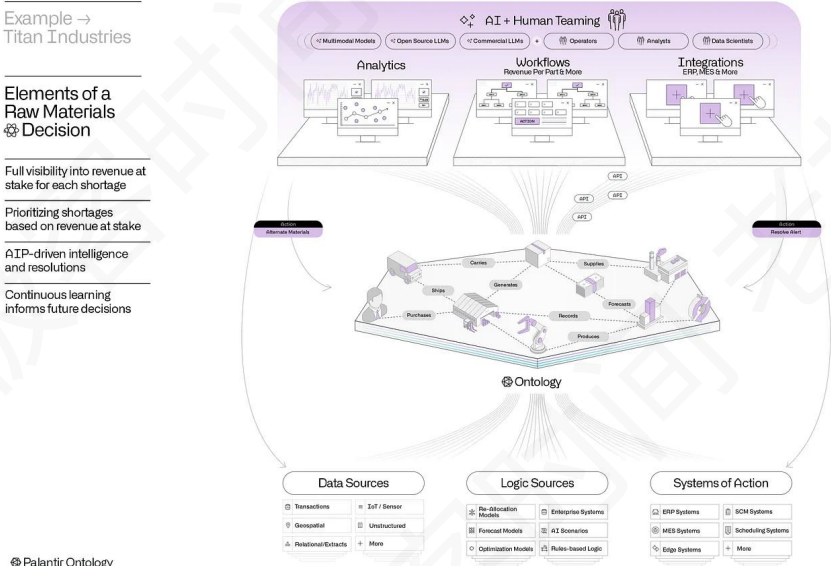
资料来源：Palantir招股书，中信证券研究部



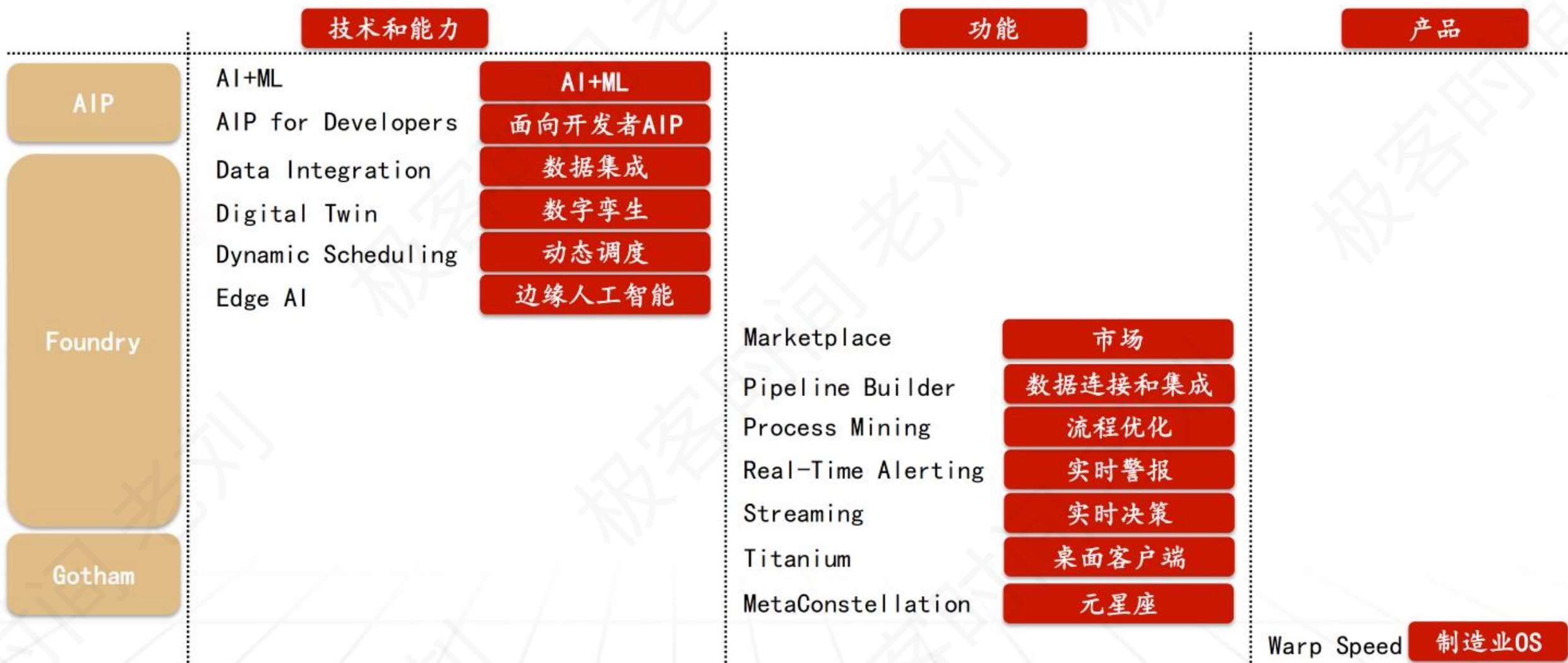
Palantir明星产品间的互补-Foundry vs Gotham

- Gotham 是 Palantir 2003 年成立后的首款产品，最初服务于 CIA、FBI 等情报机构，Foundry 是后来向商业领域扩展的产物，将 Gotham 的核心能力企业化、通用化。**Gotham实际上整合了Foundry的分析能力**，提供完整的分析和运营应用套件。
- 两者都通过 Apollo 平台进行持续交付和更新，两个平台都基于 Palantir 的核心技术栈构建：
 - 本体论（Ontology）驱动架构:将数据映射为"人、地点、事物、事件"及其关系
 - 数据融合能力:整合结构化与非结构化数据（日志、文档、图像、视频等）
 - 军事级安全体系:细粒度访问控制、数据血缘追踪、全面审计

维度	Foundry	Gotham
目标客户	商业企业（制造业、金融、医疗、零售等）	政府机构、国防、情报、执法
核心定位	"现代企业的操作系统"——数据整合与业务决策	"任务指挥操作系统"——情报分析与实时作战决策
典型场景	供应链管理、反洗钱、预测性制造、库存优化	反恐调查、战场态势感知、目标追踪、情报融合
部署环境	公有云、企业数据中心	机密网络、战术边缘（潜艇、飞机、无人机等）



➤ Palantir明星产品间的互补-技术、能力、功能、产品





Palantir Technologies

- **成立背景:** 2004年成立，总部位于美国科罗拉多州，专注于解决复杂数据问题。
- **愿景与定位:** 一家帮助政府和企业整合、分析并理解数据的企业级软件公司，致力于打破数据壁垒。
- **核心心智:** 以“解决复杂问题”和“极致数据安全”为基石，广泛服务于国防、金融、医疗、能源及制造等关键领域。



Foundry

智能整合与运营平台

帮助企业打通内部所有数据源，消除数据孤岛，构建统一、可信赖的数据底座，赋能业务运营的每一个环节，实现真正的数据驱动。



Gotham

国防与情报分析平台

专为国防、国家安全与执法机构设计，支持海量多源异构情报数据的快速关联与分析，在反欺诈、反恐、网络安全等高风险场景下提供关键决策支持。



AIP

人工智能平台

将前沿的大型语言模型(LLM)能力与 Foundry 的私有数据和业务逻辑深度融合，使 AI 能够理解复杂的企业业务规则，实现从“拥有数据”到“智能决策”的质的飞跃。



Palantir Foundry-现实背景-企业数据应用的核心挑战



数据孤岛 | Data Silos

- 数据分散在ERP、CRM、SCM等不同业务系统，格式不统一，整合难度大。
- IoT、日志等新数据源持续涌现，部门间壁垒导致数据共享与协作困难重重。



决策滞后 | Delayed Decisions

- 传统分析流程冗长，生成报告耗时久，导致业务决策总是慢于市场变化。
- 数据洞察多停留在静态报告层面，难以转化为实时、可落地的业务行动。



技术与业务脱节 | Misalignment

- IT团队专注底层技术实现，业务团队聚焦具体场景决策，双方存在巨大沟通与认知鸿沟。
- 分析严重依赖人工经验，难以规模化复用，阻碍企业向智能化分析转型。



缺乏自动化闭环 | No Automation Loop

- 从数据挖掘、洞察发现到最终的业务行动落地，链条断裂，无法形成完整价值闭环。
- 关键决策点仍需大量人工干预执行，不仅效率低下，还极易引入人为操作失误。



Foundry平台定位

定义(Definition)

Palantir Foundry是一个专为解决复杂业务挑战而设计的企业级数据操作系统，旨在打通海量、异构的企业数据，解决大规模数据集成、治理、分析与应用落地的全链路难题。

核心目标(Core Goal)

打破企业内部的“数据孤岛”，将分散在不同系统中的非结构化与结构化数据统一为可信的资产，赋能业务人员，最终实现组织层面的高效协同与数据驱动决策。

核心架构理念



以业务为中心(Business-Centric)

拒绝为技术而技术，平台功能与数据模型均紧密围绕具体业务场景与战略目标构建，让技术真正服务于业务价值创造。



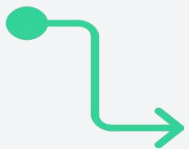
可执行性(Actionable)

不只是提供静态报表或洞察，更能将分析结果转化为可执行的指令与 workflows，直接驱动一线业务人员实时采取行动。



自适应进化(Adaptive Evolution)

系统能基于持续的业务反馈与新接入的数据，不断自我优化模型与逻辑，灵活适应快速变化的市场环境 with 业务需求。



CONNECT

连接

通过丰富的连接器，打破数据孤岛，整合所有数据源。



TRANSFORM

转化

提供强大的数据处理能力，将原始数据转化为高质量、可信赖的数据集。



MODEL

建模

通过本体论 (Ontology)，将技术数据转化为业务语言，构建统一的企业数据模型。



APPLY

应用

基于统一模型，快速构建面向业务场景的应用和决策支持系统。

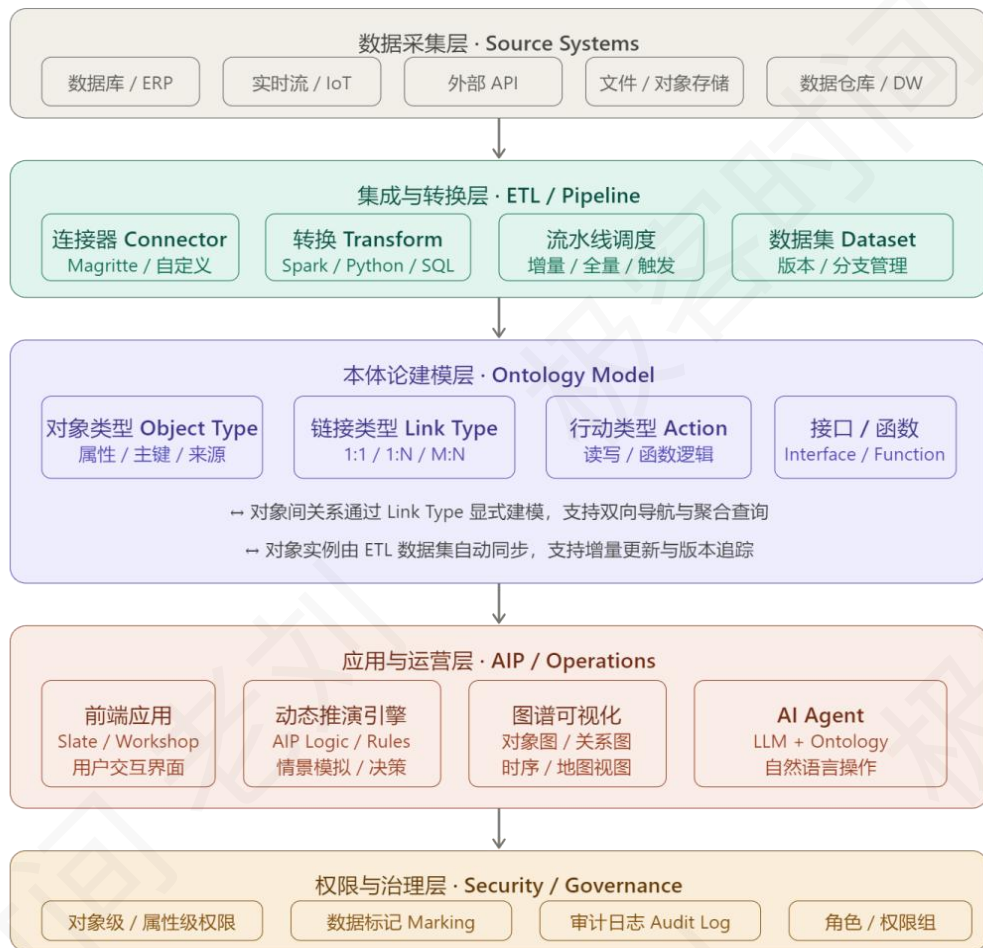


ACT

行动

通过AI Agent和自动化流程，实现从分析到行动的闭环。

Palantir Foundry-5层平台架构设计



01

数据采集

连接企业内外所有相关数据源，打破数据孤岛。



02

集成转换

将原始、异构的数据清洗、转换、整合为高质量、可信赖的数据集。



03

本体建模

构建统一的、面向业务的企业数据模型，打通业务与技术的语言障碍。



04

应用运营

基于统一数据模型，快速构建面向业务场景的应用和决策支持系统。



05

权限治理

确保整个数据生命周期的安全、合规和可审计，平衡安全与效率。

核心目标：连接企业内外所有相关数据源，打破数据孤岛

通过丰富的连接器，建立统一的数据接入与治理通道，消除碎片化存储带来的数据隔阂。



连接器生态

内置丰富的连接器生态系统，统一管理和调度各类数据源，支持多种主流数据库、数仓、SaaS应用及云存储，开箱即用。



多样化接入方式

灵活支持批量(Batch)与实时(Streaming)两种模式，可根据业务时效性要求自由组合，实现海量数据的高效、准实时同步。



零拷贝与虚拟化

支持数据虚拟化技术，实现不移动数据即可即时访问，大幅降低数据冗余和存储成本，打破物理壁垒，实现敏捷访问。

广泛兼容的
数据源生态

Oracle

Snowflake

Salesforce

SAP

Kafka

Files



核心目标：消除数据孤岛，构建统一可信的数据资产

将分散、异构的原始数据，通过标准化的清洗、转换和整合，构建成企业统一、高质量、可信赖的单一数据集，彻底打通企业内部的数据孤岛。

技术引擎 Technical Engine



数据管道 (Pipelines): Foundry 的 ETL/ELT 核心组件，用于构建可复用的数据处理与转换流程。



多语言支持: 全面支持 Spark (Scala/Java)、Python (Pandas/PySpark) 和 SQL 等主流数据处理语言。



自动化依赖管理: 自动追踪和维护数据集的版本、依赖关系与血缘图谱，降低维护成本。

质量保障 Quality Assurance



不可变版本 (Immutability): 每次 Pipeline 运行均生成全新的不可变版本，防止历史数据意外被覆盖或篡改。



全链路追溯 (Traceability): 记录完整的数据血缘关系，让数据来源清晰透明，便于问题排查和合规审计。



结果可重现 (Reproducibility): 基于代码化和版本化的转换逻辑，在任何时间点都能重现相同的数据处理结果。

连接器生态系统

Foundry 的连接器生态广泛而强大，打破数据孤岛，确保企业能够无缝连接到其运营所需的所有相关数据源，实现全域数据的统一汇聚。

关系型数据库

Oracle, PostgreSQL, SQL Server, MySQL

云数仓与湖仓

Snowflake, Redshift, BigQuery, Azure Synapse

SaaS应用

Salesforce, SAP, ServiceNow, Jira, Workday

消息队列

Apache Kafka

标准文件格式

CSV, JSON, Parquet, Avro, XML

转换语言与开发环境



多语言灵活支持

无需改变团队技术习惯，全面支持 Spark (Scala/Java), Python 和 SQL，灵活匹配不同技术背景的工程师技能，降低学习成本，最大化开发效能。



交互式集成开发环境 (IDE)

内置基于 Jupyter Notebook 的原生开发环境，支持交互式数据探索、分析与转换开发，实现“所见即所得”的快速验证与调试，显著提升数据处理的迭代效率。



Palantir Foundry-架构-集成转换层-三种集成模式



模式一：Palantir作为消费方

(推荐方案)

描述：不重复采集源系统，直接对接数据中台已加工好的数据层（比如 ODS 或 DWS 层），Palantir 只拿加工后的宽表，跳过原始数据采集。数据中台继续负责 ETL 和治理，Palantir 负责 Ontology 建模和 AIP 决策。

优势：口径高度一致，避免形成新的数据孤岛，技术与业务分工清晰明确。
Palantir 专注 Ontology + AIP 价值层

劣势：Foundry的分析能力受限于中台提供的接口类型和时效。

适用场景：企业已有成熟数据中台体系，希望快速利用Foundry解决特定复杂业务问题。



模式二：并行双轨模式

描述：BI 侧重报表、Palantir 侧重运营决策，两者并重。数据中台和 Palantir 各自从源系统独立采集，但通过统一数据资产目录（Data Catalog）保持元数据一致，避免口径打架。两套系统服务不同用途，不强行合并。

优势：各司其职，互不干扰，不强行合并技术栈，有效降低系统切换与替换风险。

劣势：存在明显的数据存储冗余，且长期存在“数出多门”的口径不一致风险。

适用场景：BI报表与敏捷运营并重的企业，需配合Data Catalog工具定期对齐数据口径。



模式三：承担中台职能

描述：直接用 Foundry 承担数据中台职能，原有 BI 工具（Tableau、Power BI）通过 Foundry 的 JDBC/ODBC 接口继续连接，不需要迁移。

优势：实现技术架构统一，消除数据在不同系统间的移动，充分发挥端到端闭环能力。

劣势：系统实施复杂度高，周期长，且对内部技术团队的技能要求大幅提升。

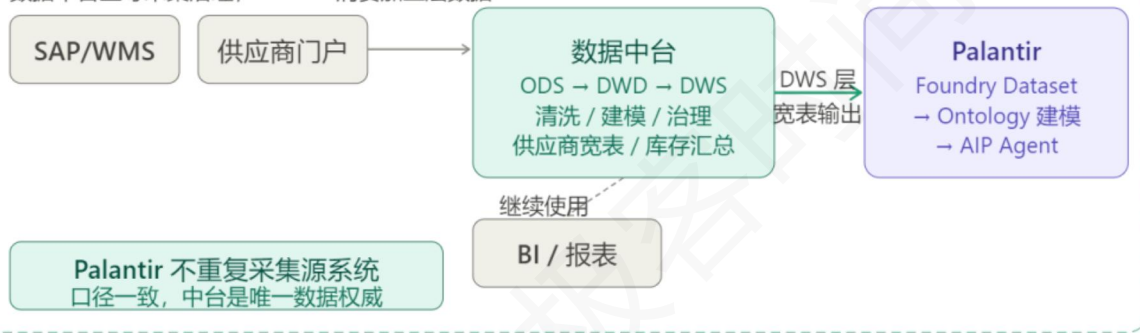
适用场景：原有数据中台建设不成熟，或无法支撑复杂业务场景需求，且有长期转型计划。



Palantir Foundry-架构-集成转换层-三种集成模式

模式一：Palantir 作为消费方（推荐）

数据中台主导采集治理，Palantir 消费加工层数据



模式二：并行双轨

BI 侧重历史报表，Palantir 侧重实时运营决策



模式三：Palantir 承担中台职能

数据中台尚不成熟，Foundry 直接替代，BI 工具通过接口继续连接



选型建议对照

数据中台成熟 → 模式一（推荐）：避免数据孤岛，Palantir 专注 Ontology + AIP 价值层
BI 报表 + 运营决策 并重 → 模式二：Data Catalog 对齐口径是关键

核心目标： 构建一个统一的、面向业务的企业数据模型，将技术化的“表与字段”转化为业务人员可直接理解和交互的“实体与概念”，打破技术与业务之间的语言壁垒。

核心概念Core Concepts



Object Type (实体) /Property(属性)

业务领域中的核心实体，如“客户”、“产品”或“设备”。



Link Type (关系)

定义实体间的关联，如“客户购买了产品”。



Action Type (动作)

可执行的业务操作，如“创建订单”或“触发通知”。



Function (函数)

封装的计算逻辑，如“计算LTV”或“生成推荐”。

核心价值Core Values

统一的“语义层” | 屏蔽底层存储与技术细节，让业务聚焦于业务逻辑而非SQL。

单一业务实体视图 | 自动打破数据孤岛，实时聚合分散数据，提供完整、一致的画像。

业务驱动的定义 | 由业务专家主导定义模型，实现真正的“业务定义业务”，提升响应速度。



❖ Object (对象类型)/Property(属性)

■ **定义：** 业务领域模型中的核心实体，是构成知识图谱节点的基础。例如：客户 (Customer)、产品 (Product)、订单 (Order) 等。

■ **作用：** 打破数据孤岛，将散落在CRM、交易系统、客服系统等不同数据源中的异构数据关联整合，构建统一的、全局唯一的业务实体视图。

■ **技术实现：** 通过逻辑映射整合一个或多个底层数据源，生成统一的业务语义实体，并分配全局唯一的 Identifier (ID) 以确保实体身份的一致性。

Link Type (链接类型)

■ **定义：** 定义不同 Object Type (业务实体) 之间的关联关系与连接语义。例如：订单与客户之间的“下单 (Placed By)”关系，产品与订单之间的“包含 (Contains)”关系。

■ **作用：** 将孤立的实体连接成一张结构化的业务知识图谱，让隐藏在复杂系统中的业务逻辑与关联关系显性化，支持复杂关系的直观查询与路径分析，避免编写复杂的 SQL 多表关联。

■ **技术实现：** 定义关系的标签与方向，并灵活支持一对多 (1:N)、多对一 (N:1) 以及多对多 (N:M) 的数据关联映射，确保业务逻辑的准确表达。



Action Type (动作类型)

定义: 定义可以在 Object Type 上执行的操作, 描述系统能够对外界或内部触发的响应行为。典型场景包括: 更新库存、创建订单、发送通知等。

作用: 实现业务决策闭环的关键技术组件。分析洞察的结果可直接触发 Action Type 执行, 无需人工干预。例如, 当智能 Agent 检测到库存低于安全阈值时, 可自动调用 `updateSafetyStock` 动作更新系统数据。

技术实现: 支持通过标准 API 调用或自定义代码 (Python/Java) 实现, 与企业内部系统无缝集成, 支持自动化的业务流程编排。



Function (函数)

定义: 封装可复用的业务计算逻辑, 用于处理 Object Type 的属性或关联对象之间的复杂关系, 作为数据层的逻辑扩展。

作用: 计算派生指标, 避免重复开发。例如: 计算客户生命周期价值(LTV)、订单金额的实时税费计算、基于多因子的信用评分等, 将复杂的业务规则统一管理和复用。

技术实现: 支持 Python/Java 代码编写并注册到本体论中, 作为对象的“虚拟字段”对外提供服务, 为上层应用提供统一计算能力。

核心目标： 基于统一的数据模型，快速构建面向业务场景的应用和决策支持系统，实现数据价值落地。



低代码界面

(Workshop & Slate)

- **Workshop (面向业务用户)：** 提供拖放式的界面设计工具，让业务用户也能快速构建交互式应用。
- **Slate (面向开发者)：** 基于React的开发框架，用于构建高度定制化、高性能的Web应用。



推演引擎

(AIP Logic Engine)

复杂决策分析中枢： 提供业务规则、机器学习模型和自然语言查询的融合能力，打破数据孤岛，为上层应用和AI Agent提供统一的决策支持与逻辑处理能力。



AI Agent

(自主执行的智能体)

具备自主执行能力的智能体，能够理解自然语言指令，自动进行任务分解，动态调用各类工具 (**Action**) 并完成自主执行，最终实现复杂业务流程的端到端自动化与智能化闭环。



AIP Logic 推演引擎

核心原理：决策图推理

结合业务规则(Rule-Based)、机器学习模型(ML-Based)与实时业务上下文(Ontology)，构建复杂决策图(Decision Graph)，通过正向或反向遍历推理，输出高置信度的业务结论。

灵活规则定义

支持可视化 Rule Builder 零代码构建，也支持通过 Python/Java 代码定义，轻松实现复杂的业务逻辑组合与迭代。

原生模型集成

无缝调用 Foundry Model Repository 中的训练模型，将预测性分析能力直接注入业务决策流，实现“预测即决策”。



AI Agent 智能体

自主闭环 workflow (Autonomous Loop)

1. 感知 (Perceive)

订阅对象变更或定时触发，持续监控并感知业务状态的实时变化。

2. 分析 (Analyze)

检测关键事件后，调用 AIP Logic 深度分析，或利用 LLM 进行意图理解与任务分解。

3. 决策 (Decide)

结合逻辑引擎与大模型，生成一个或多个可行的业务解决方案。

4. 行动 (Act)

自动选择最优方案，调用 Action Type 执行操作，完成端到端业务闭环。



LLM 赋能：无缝集成 GPT-4 等模型，赋予智能体强大的自然语言理解与生成能力。



Workshop

定位面向业务用户和分析师的低代码工具，无需编程知识，实现零门槛上手。

技术实现基于组件化拖拽，可视化生成JSON格式的应用描述，由Foundry前端框架解析并渲染。

核心能力内置丰富的可视化与数据操作组件，可直接绑定 Ontology 中的 Object、Link 和 Action，开箱即用。

适用场景快速构建临时分析看板、自助式数据探索界面及轻量级业务应用。



Slate

定位面向专业开发者的高度灵活 Web 应用开发框架，专注于极致的定制化能力。

技术实现提供 `@palantir/slate` NPM 包，包含 React 组件和 Hooks，可高效与 Foundry API 及 Ontology 交互。

核心能力支持服务端渲染(SSR)、代码拆分等现代技术，适用于构建高性能、高交互性的复杂应用。

适用场景企业核心业务系统、深度定制化的 UI/UX 应用及复杂业务流程驱动的系统。

统一身份认证与精细化权限



SSO / LDAP 身份集成

无缝对接企业现有的身份系统，实现一次登录即可访问所有授权资源，消除身份孤岛。



RBAC 基于角色的访问控制

建立精细化的角色访问模型，严格控制不同用户对数据、应用和功能模块的访问权限，强化数据安全。



属性级 (Attribute-level) 权限授权

支持对单个数据实体、甚至单个字段进行精细化权限控制，精准实现对高敏感数据的分级分类管理。

数据质量与合规体系



全链路数据质量监控

覆盖从原始数据接入、清洗处理到最终应用展示的全生命周期，自动监控并预警异常，确保数据准确可信。



敏感数据部分标记 (Masking)

对身份证号、手机号等敏感信息进行动态脱敏和部分化展示，在实现数据高效共享的同时，满足企业合规要求。



全量可追溯审计日志

完整记录平台内所有的数据查询、修改和删除操作，提供详细的操作审计报告，满足监管审查与安全审计需求。



版本控制

每次Pipeline运行后，都会生成一个新的、不可变的数据集版本。因此每次数据更新都是原子性的，每一次的修改都被记录和追踪，让数据回归与历史重现变得简单直观。



数据血缘可视化

提供直观的血缘图谱，清晰展示数据从源头到最终应用的完整链路，便于问题排查、影响分析和合规审计。



依赖管理

Foundry会自动发现和维护数据集之间的上下游依赖关系。当上游数据集变更时，能够智能识别并自动触发所有下游依赖的重新计算，确保数据的一致性和准确性。

业务场景

某制造企业希望利用 Foundry 实现对供应链的精细化管理，核心诉求包括：实时监控库存水位、评估供应商综合表现、追踪物流全流程状态，并建立敏捷的异常响应机制，从而有效降低供应链风险，显著提升整体运营效率。



01 数据采集

打通企业内部 ERP、WMS、MES 等核心系统，连接外部供应商门户与第三方物流平台，实现跨系统、跨组织的多源异构数据的统一接入与融合。



02 模型构建

将清洗整合后的原始数据，按照业务逻辑构建“供应商-零件-订单-库存”的关联模型，把碎片化的数据转化为高价值的业务实体对象，支撑复杂分析。



03 AI决策闭环

部署 AIP Agent 实时监控供应链关键指标。当出现库存预警或物流延误时，系统自动归因分析并生成解决方案，甚至支持自动触发补货指令或调整排产计划。



01 / 数据采集



ERP 系统

采购订单、实时库存水平、
供应商主数据



MES 系统

生产计划信息、产线物料消
耗与产出记录



物流平台

第三方物流(3PL)全程运输跟
踪数据



财务系统

应付账款、供应商历史定价
及发票数据



外部情报 准入与舆情API、国际政治风险事件等外部数据源

02 / 集成转换

Pipeline 自动化处理流程

数据清洗

处理供应商名称拼写、物
料编码不一致等问题，消
除“脏数据”

数据标准化

统一不同来源的日期格式、关联不同系统信息，如将
计量单位及地址信息，实
现格式对齐

数据整合

采购订单与入库记录打通，
构建全景视图



Foundry 数据资产化

将处理后的数据集注册为版本可控的 Foundry Dataset，消
除数据孤岛，直接赋能后续分析与业务应用。

定义核心业务对象

- **Object Type:**Supplier (供应商)
- **Object Type:**Part (零部件)
- **Object Type:**PurchaseOrder (采购订单)
- **Object Type:**Shipment (货运)

定义对象关系

- **Link Type: Supplies**
供应商向企业供应零部件的关联
- **Link Type: Contains**
一份采购订单包含多种零部件的关联
- **Link Type: Fulfills**
货运物流满足并完成采购订单的关联

定义函数与动作

FUNCTIONS / 计算函数

- calculateLeadTime(): 计算供应商平均交付周期
- calculateDefectRate(): 计算零部件批次缺陷率

ACTIONS / 动作类型

- createPurchaseOrder(): 触发创建新采购订单流程
- sendAlertToSupplier(): 自动向供应商发送预警通知



AIP Logic 业务规则引擎

核心逻辑规则示例:

“如果任何 Supplier 的交付评分 **Score > 70**, 且该供应商在关键物料的唯一来源, 则系统应触发动作

`createAlternativeSupplierRequest`, 为寻找替代供应商创建请求。”

Workshop 供应商风险管理仪表盘



全量分布可视化监控

实时展示各供应商的交付准时率、质量评分、价格波动等关键指标, 风险尽在掌握。



360° 供应商全景档案

点击任意供应商, 即可查看其完整的业务画像, 包括历史订单、交付记录、质量问题及财务状况。



一站式敏捷业务协作

在仪表盘内直接发起多方沟通、创建采购订单、实时更新供应商基础信息, 打破协作壁垒。



自动化执行



自动触发预警通知

当 AIP Agent 根据规则触发 **createAlertToSupplier** 动作时，Foundry 自动调用外部邮件或协作工具 API，向采购人员实时推送风险预警。



自动生成采购订单

当采购经理在 Workshop 确认并执行 **createPurchaseOrder** 动作时，系统调用 ERP API，自动完成新订单的创建与系统导入，减少人工操作。

核心业务价值



实时响应

打破数据到行动的壁垒，实现风险识别到应对的秒级响应，将潜在风险扼杀在摇篮中。



降本增效

降低大量事务性人工成本，减少与供应商的低效沟通，释放团队精力以聚焦战略采购。



智能预警

建立事前预警机制，帮助企业管理者提升面对突发状况的恢复力，增强供应链抗风险能力。



业务场景：精细化库存管理的挑战

某零售企业拥有海量SKU，传统的库存管理方式难以做到精细化，导致部分商品积压占用大量资金，而部分商品则因缺货造成销售损失。



库存积压风险高

无法实时识别滞销商品，导致库存成本持续高企。



人工决策低效

补货依赖人工经验，响应慢且准确率低。



缺乏数据驱动

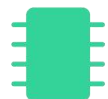
缺少有效机制，难以系统性优化库存策略。

业务目标：通过Foundry实现智能库存管理



实时监控与预警

建立SKU级库存周转率监控模型，对异常数据进行毫秒级告警。



自动化智能决策

系统自动识别滞销风险，并联动业务端自动触发促销或补货策略。

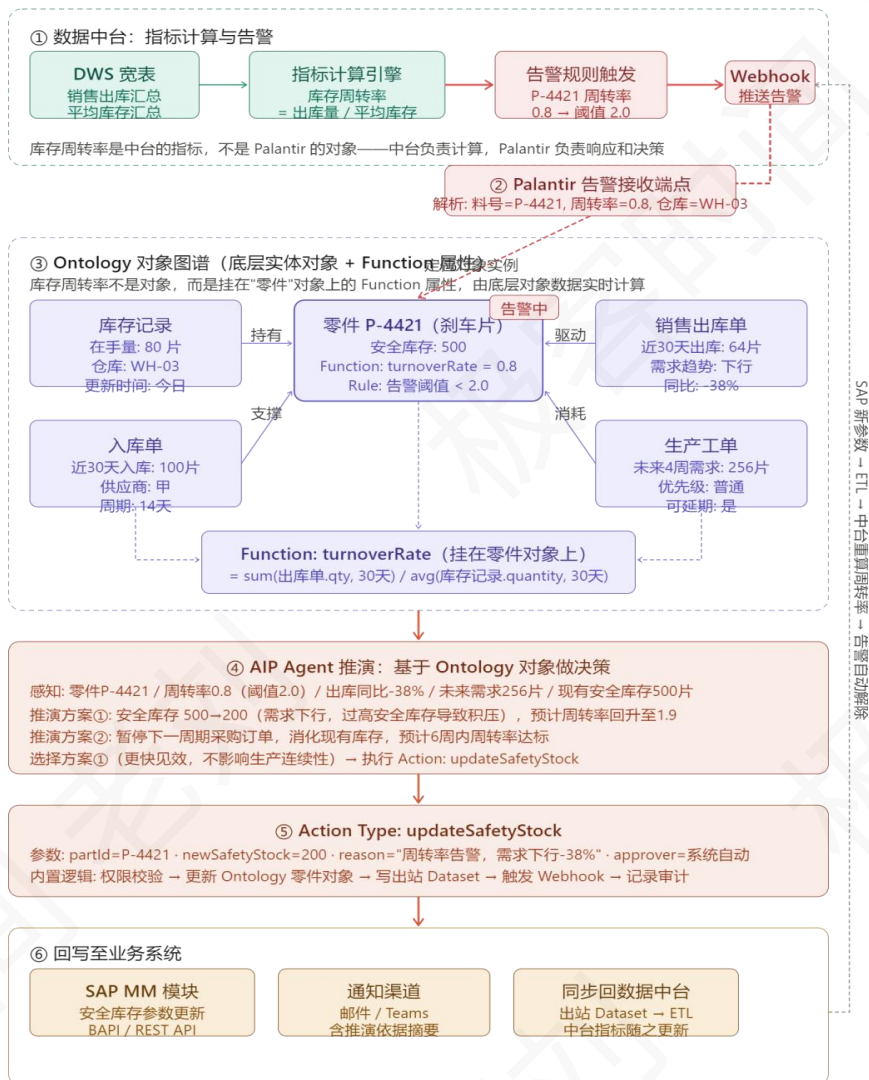


释放资金与提效

大幅降低呆滞库存，释放现金流，提升整体供应链周转效率。



Palantir Foundry-案例-库存周转率告警驱动决策

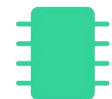


业务目标：通过Foundry实现智能库存管理



实时监控与预警

建立SKU级库存周转率监控模型，对异常数据进行毫秒级告警。



自动化智能决策

系统自动识别滞销风险，并联动业务端自动触发促销或补货策略。



释放资金与提效

大幅降低呆滞库存，释放现金流，提升整体供应链周转效率。



01 / 定义核心对象

Object Type Definition

核心对象定义:

在 Ontology 中, 将“产品”正式定义为核心对象

`palantir.ontology.test/Product`, 作为业务逻辑的基础载体。

数据来源映射:

打通异构数据源, 统一映射来自 ERP、WMS 等系统的产品主数据、实时库存数据和历史销售流水数据。



02 / 封装计算逻辑

Logic Encapsulation
(Function)

虚拟字段设计:

将复杂的库存周转率计算逻辑封装为一个可复用的 **Function**, 并挂载为 **Product** 对象的“虚拟属性”, 实现业务逻辑与数据的解耦。

```
@Function
calculateInventoryTurnover()
{
  return TotalSalesVolume /
    AverageInventoryLevel;
}
```



03 / 构建单一产品视图

Single Product View (SPV)

信息整合与统一:

通过 Ontology 的关联能力, 将分散在不同系统中的产品基本信息、多维库存信息和全渠道销售数据整合为一个统一的对象视图。

赋能决策分析:

为后续的智能分析与业务决策提供“单一可信源 (Single Source of Truth)”, 消除数据孤岛, 提升分析效率。



AIP Logic 规则定义

01. 促销触发规则 (Promotion Trigger)

```
Trigger: Product.inventoryTurnoverRate < 0.5 AND  
SalesTrend = 'Decreasing'  
Rule: Create a promotion event for this product.
```

02. 补货触发规则 (Replenishment Trigger)

```
Trigger: Product.currentInventory <  
Product.safetyStockLevel  
Rule: Create a replenishment order for this product.
```

AI Agent 执行逻辑



定时触发监控

按设定频率(如每日)自动查询,持续监控所有商品的业务状态变化。



数据驱动计算

调用
`calculateInventoryTurnover()` 函数, 实时计算并更新商品关键指标。



业务规则匹配

将计算结果与预设的AIP Logic规则库进行比对, 精准识别出需要干预的商品。



自动动作执行

对匹配成功的商品自动执行 Action, 例如直接生成促销方案或提交补货订单。



统一数据视图

通过本体论建模，打破数据孤岛，实现数据的统一与一致，消除部门间的沟通壁垒，构建企业级的“数据通用语言”。



智能决策支持

结合行业Know-how与AI大模型能力，打破信息茧房，打通从“海量数据洞察”到“科学智能决策”的关键链路。



自动化业务闭环

利用AI Agent和Action Type将业务流程代码化，真正实现从“洞察发现”到“业务执行”的自动化闭环，将决策转化为行动。



端到端数据治理

提供覆盖数据全生命周期的安全与合规能力，在保障数据流动效率的同时，确保数据的可信度、隐私安全和合规性。

核心愿景：将企业的数据资产转化为可落地、可操作的业务智能

04 Palantir Ontology的三层结构

Semantic语义层、Kinetic行为层、Dynamic动态层

核心定位COREPOSITIONING



定义 - Definition

语义层是Palantir三层架构的底层根基，它不等同于传统数仓，而是平台内置的一套完整、可扩展的业务本体体系(Ontology)。



作用 - Function

对业务进行统一抽象，涵盖业务概念、实体、属性、关联关系、业务规则与计算函数，是业务逻辑的标准载体。

核心价值COREVALUE



屏蔽底层差异，提供统一视图

封装异构数据源的复杂性，打破技术壁垒，为所有业务人员提供一致、直观的业务视角，降低数据使用门槛。



承上启下，支撑上层敏捷应用

作为稳固的基座，为行为层和动态层提供标准、可复用的语义模型，加速上层分析与应用的开发与迭代。



平台生态的“核心骨架”

贯穿Foundry平台，是数据建模、治理与跨业务联动的核心枢纽。



01/业务建模

- **业务领域概念标准化建模：**定义核心业务实体，统一业务视角。
- **实体/属性/关系统一定义：**规范业务模型，消除定义歧义。



02/数据映射

- **多源异构数据接入：**支持结构化与非结构化数据的统一接入。
- **灵活语义映射：**实现底层物理表与上层业务实体的动态关联。



03/逻辑与动作

- **复杂计算封装：**支持自定义函数与复杂业务规则的灵活配置。
- **动作语义绑定：**可配置动作挂载，打通“数据理解”到“业务行动”的闭环。



04/可视化构建

提供全流程可视化本体编辑器，降低技术门槛，让业务人员也能直接参与模型定义与迭代，打破IT与业务部门的沟通壁垒，加速语义模型的落地与优化。



■ 什么是“属性”？

属性是语义层中用来描述“对象类型 (Object Type)”具体特征的字段，它是构建业务实体的基础原子。所有属性都必须归属于特定的对象类型，不能独立存在。

■ 关键特性

- **强挂载性：** 属性不能孤立存在，必须隶属于某一具体的对象类型（如客户、设备）。
- **动态计算能力：** 对于复杂的派生属性，其值随依赖的基础属性实时更新，确保数据的一致性。



基础属性 (Base Properties)

直接从底层数据源（如数据库表、API）映射而来的原始数据字段，是语义层数据的基石。

示例： `Customer.name`, `Equipment.serialNumber`, `Transaction.amount`



派生属性 (Derived Properties)

基于一个或多个基础属性、或通过函数、业务逻辑计算得出的“虚拟”字段，用于封装复杂的业务规则。

示例： `Customer.fullName` (拼接名+姓);
`Equipment.riskScore` (算法综合评分)



Palantir Ontology-三层架构-语义、行为与动态



语义层(Semantic)

“世界是什么 (What is)”

- 业务名词 / 概念模型层
 - 定义“存在什么”
- 解决“语义统一”问题



行为层(Kinetic)

“世界如何变化 (What we do)”

- 业务动词 / 操作执行层
 - 定义“能做什么”
- 解决“数据可操作、可审计”问题



动态层(Dynamic)

“世界如何演化 (How it evolves)”

- 自适应演化层
 - 定义“系统如何持续优化”
- 解决“规模增长不重构”问题



01 / 定位

业务名词 / 概念模型层

语义层是整个架构的基石，它负责定义企业业务中的核心实体、它们的特征以及彼此之间的关系。

它旨在为企业创建一个统一的、无歧义的“业务字典”，确保业务术语在组织内的一致性和可理解性。

02 / 核心思想

解决“语义统一”问题

打破不同系统（如ERP、CRM、SCM）间的称呼壁垒，将异构数据统一映射到标准化的业务概念上。



沟通统一：让业务与技术人员基于同一套“业务语言”沟通，消除理解偏差。



消除孤岛：打通跨系统数据壁垒，实现全域数据的统一理解与融合分析。



Palantir Ontology-语义层-核心构成-Object Types

定义 Definition

对象类型是对业务领域中核心实体的抽象描述，是构建语义层的基本单元。它类似于面向对象编程中的“类”，但更侧重于业务含义，是连接技术数据表与业务语言的桥梁。

关键特性 Key Features

- **数据绑定:** 直接关联底层数据库表或实时数据流，是数据的“实时投影”。
- **唯一性:** 通过唯一标识符 (Primary Key) 区分对象实例，确保数据准确。
- **可扩展:** 可随业务发展灵活新增或修改，适应业务快速迭代需求。

行业示例 Industry Examples

制造业 Manufacturing

ProductionLine · Equipment · Product · Supplier

金融服务 Financial Services

Customer · Account · Transaction · Trade

医疗健康 Healthcare

Patient · Doctor · Medication · Diagnosis



01 / 定义 Definition

关系类型定义了不同对象类型之间的业务关联逻辑，是连接孤立数据实体的纽带。它是构建企业级知识图谱、打破数据孤岛、并最终实现高效关联分析的关键基石。

02 / 示例 Examples

- 一对多: `ProductionLine` **has many** `Equipment`
- 多对多: `Student` **attends many** `Course`, and `Course` **has many** `Student`
- 反向关系: `Equipment` **is installed on a** `ProductionLine`



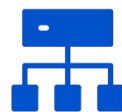
基数定义 (Cardinality)

支持精确配置关系的基数规则，如“一对一 (1:1)”、“一对多 (1:N)”或“多对多 (M:N)”，确保业务逻辑在数据模型中的精准映射。



双向可导航性 (Navigability)

用户可以自由地从任意一个对象节点出发，通过定义好的关系路径快速导航到其他关联对象，直观地探索数据间的连接网络。



原生图存储 (Graph Storage)

关系在底层以图数据库的“边 (Edge)”形式进行原生存储，而非传统的外键方式，这使得系统能支持毫秒级的复杂图遍历和多跳关联查询。



01 / 接口定义

接口是一种特殊的对象类型，它定义了一组可以被其他对象类型共享的属性和关系。它类似于面向对象编程中的“抽象类”或“接口”，为不同实体提供了标准化的交互规范。

02 / 常见接口示例

Inspectable 接口

包含属性:

- lastInspectedDate (上次检查日期)
- nextInspectionDue (下次检查到期日)

Measurable 接口

包含属性:

- currentReading (当前读数)
- unitOfMeasure (计量单位)

03 / 关键核心特性



共享结构 (Shared Structure)

任何对象类型都可以“实现”一个或多个接口，从而继承其所有属性和关系，实现逻辑复用。



代码复用 (Reusability)

避免在多个对象类型中重复定义相同的属性和关系，减少冗余，极大地提高了模型的可维护性。



多态性 (Polymorphism)

可针对接口编写通用的分析逻辑，该逻辑可无缝应用于所有实现了该接口的对象类型，提升灵活性。



什么是“活的图模型”？

语义层中的对象、属性和关系并非静态的 Schema 定义，而是直接绑定到底层实时数据流的“活”的投影。底层数据源的任何变化，无需繁琐的同步流程，都能即时反映在模型视图中。

核心价值



实时性：打破数据同步的时间差，确保业务分析与决策完全基于最新的实时数据。



敏捷性：无需漫长的开发周期修改底层Schema，业务逻辑的变化能快速映射到语义层。

与传统 Schema 的核心差异

维度	语义层（图模型）	传统数据库 Schema
数据同步	实时动态投影 底层变化即时反映	静态定义 依赖 ETL 批量同步
灵活性	易修改、易扩展 完全解耦底层存储	修改成本极高 可能导致服务中断
数据形态	统一的关联图模型 直观呈现业务关系	分散的表结构 关联关系隐式存在



Palantir Ontology-语义层-关键特点-统一异构系统语义



异构系统的语义壁垒

大型企业通常拥有众多异构系统，如 SAP、Salesforce、Oracle 等，它们对同一业务实体的定义和命名往往不一致，造成数据孤岛。

ERP 系统

客户主数据表

KNA1

CRM 系统

账户对象

Account

数据仓库

客户维度表

dim_customer



解决方案：语义层统一映射

- **统一对象**：将所有异构系统中的“客户”实体统一映射到语义层标准对象 **Customer**
- **属性整合**：将 KNA1.KUNNR / Account.Id 统一为 **Customer.id**



核心价值

单一事实来源 (SSOT)

为企业所有分析应用提供统一、一致的可信视图，打破壁垒。

消除歧义与沟通成本

统一业务术语标准，避免因定义不同导致的沟通与决策错误。



核心能力与查询示例

基于统一的语义模型，用户无需掌握专业查询语言，即可使用近乎日常交流的自然语言方式，直接对复杂数据进行灵活的查询与探索。

用户提问：“显示过去24小时内，状态为‘待维修’且位于‘上海工厂’的所有‘生产设备’的平均‘振动值’。”

系统自动解析：识别对象(生产设备) → 锁定筛选条件(状态/位置/时间) → 计算聚合值(振动值平均值) → 生成高效查询并自动执行。



降低使用门槛

打破技术壁垒，让不懂SQL的业务人员也能自主探索数据，真正实现“人人都是数据分析师”。



提高分析效率

将业务问题直接转化为查询逻辑，减少与技术团队的沟通成本，大幅缩短从提出问题到获取答案的时间周期。



Palantir Ontology-语义层-总结-企业的“动态业务字典”

语义层通过定义对象、属性、关系和接口，为企业构建了一个统一、实时、可理解的业务概念模型。它是连接底层复杂技术与上层业务逻辑的桥梁，消除了不同部门间的数据理解偏差。

它就像一本**动态更新的业务字典**，定义了企业运营的核心词汇表，是所有上层应用和分析的坚实基础。

核心价值：**解决“数据看不懂”的问题**



定位：业务动词 / 操作执行层

行为层定义了可以在语义层对象上执行的业务操作。它的核心作用是将原本静态的数据模型，从“只读”的分析工具，转变为可直接驱动业务运转的执行层，成为连接数据洞察与业务动作的桥梁。



核心：解决“数据可操作、可审计”

传统的数据模型通常是“只读”的，用户可以查询和分析，但无法直接在模型上执行业务操作。行为层的核心思想就是赋予数据“动作”能力，并确保所有操作都是安全、合规且可追溯的。

目标

将业务流程直接嵌入到数据模型中，实现数据驱动的自动化，减少人工干预成本。

价值

打通从数据洞察到业务行动的“最后一公里”，真正让数据成为业务的推动力。



📄 定义 Definition

动作类型是绑定在特定对象类型上的业务操作。它定义了用户或系统可以对一个对象实例执行的具体动作，是连接数据与业务流程的关键纽带。

⚙️ 关键特性 Key Characteristics

对象绑定 Object Binding

明确归属特定对象
多对象可复用同一动作

业务逻辑封装 Logic Encapsulation

包含完整业务步骤
屏蔽底层系统交互细节

多源触发 Trigger Sources

支持人工手动触发
或系统/流程自动触发

📋 应用示例 Examples

Customer (客户对象)

- InitiateCreditCheck (发起信用检查)
- SendMarketingEmail (发送营销邮件)

Equipment (设备对象)

- CreateMaintenanceTicket (创建维修工单)
- UpdateCalibration (更新校准信息)

Transaction (交易对象)

- ApproveTransaction (批准交易)
- FlagAsSuspicious (标记为可疑)



参数 (Parameters)

每个动作都可以定义输入参数，用于在执行时传递必要的上下文信息，以决定具体的执行逻辑和对象。

示例：执行动作`CreateMaintenanceTicket`时，必须传入`problemDescription`（问题描述）与`priority`（优先级）作为输入。



事务 (Transaction)

原子性 (Atomicity)

所有步骤要么全成功，要么全失败回滚，确保操作无中间状态。

一致性 (Consistency)

执行前后，业务规则与数据约束始终保持有效和一致。

隔离性 (Isolation)

多个并发执行的动作相互独立，互不干扰，保证并发安全。

持久性 (Durability)

一旦执行成功，结果将被永久保存，不会因故障丢失。



数据完整性保障

有效防止因系统故障或人为错误导致的业务数据损坏、逻辑冲突或状态不一致。



企业级高可靠性

通过严格的事务机制，为核心业务提供可信赖的执行保障，支撑系统在高并发下稳定运行。



Palantir Ontology-行为层-核心构成-数据映射与写入

双向映射：读与写的结合

语义层负责从外部系统读取数据构建对象，而行为层在此基础上，进一步定义了如何将操作结果回写到外部源系统，形成数据通路。



读取 (Read)

将 ERP、数据库、API 等异构数据源，统一映射为标准化的语义层业务对象。



写入 (Write)

定义业务动作执行时，如何将状态变化和结果自动同步并回写到源头业务系统。



场景示例：ApproveTransaction 动作

1. **执行逻辑**：触发动作后，立即更新内部系统交易状态为“已批准”。
2. **回写规则**：系统自动调用银行 API，将最终状态同步至核心银行系统。



关键价值点

业务闭环

打通从“数据洞察”到“业务操作”，再到“源系统落地”的完整链路。

低代码集成

无需编写复杂代码，通过配置化映射，快速实现跨系统业务流程自动化。



Palantir Ontology-行为层-核心构成-权限与血缘



权限模型 (Permissions)

每个动作都与精细的权限模型绑定，确保只有授权用户才能执行特定操作，防止越权访问。

示例：“ApproveTransaction”动作可能仅对“财务经理”角色可见且可执行。



审计日志 (Audit Logs)

所有操作均记录在不可篡改的日志中，包含：

- 执行者与时间：谁在何时执行了此操作
- 上下文：执行时的关键输入参数和运行环境
- 结果：动作执行成功或失败，及失败原因



数据血缘 (Data Lineage)

所有动作的执行都会自动生成详细的数据血缘信息，完整记录数据的来源、处理过程及产生的影响范围，确保数据流转透明化。



安全合规

满足企业对数据隐私与操作合规的严格要求，规避风险。



全链路可追溯

任何数据变化均可追踪至具体操作和人员，责任清晰。



Palantir Ontology-行为层-关键特点-让语义层“活起来”

从“只读模型”到“可执行模型”

行为层的引入，让语义层从一个静态的、仅用于分析的“只读模型”，彻底转变为一个能直接驱动业务流转的“可执行模型”，打通“分析”与“行动”之间的关键壁垒。

仅有语义层

用户分析出“哪些设备需要维修”后，需手动切换至工单系统，人工录入信息创建工单，流程割裂且耗时。

语义层 + 行为层

在分析结果页面直接点击“创建维修工单”，系统自动触发全流程，业务动作即时落地，无需人工干预。



业务自动化：洞察即行动

打破传统BI“看得清但动不了”的局限，将数据分析得出的洞察结果，直接转化为可执行的业务行动，大幅减少人工操作，提升业务响应速度。



流程优化：端到端闭环

将分散在CRM、ERP等不同业务系统中的独立步骤整合到统一的语义模型中，打破数据与流程孤岛，实现业务流程的无缝衔接与端到端自动化闭环。



Palantir Ontology-行为层-关键特点-业务人员可操作



低代码/无代码环境

动作的定义和配置主要在可视化的本体构建器中完成，无需复杂的编程知识。业务人员不再是旁观者，而是可以直接参与甚至主导业务流程的设计与迭代。



核心价值

赋能业务

打破技术壁垒，将数据处理与业务操作能力真正交还给最懂业务逻辑的一线人员。

快速响应

缩短需求响应链条，业务需求的变化能被快速转化为系统动作，敏捷应对市场变化。



所见即所得

在熟悉的业务对象（如客户、设备、工单）上直接看到可用的操作，逻辑直观清晰。



无需编程

摆脱技术门槛限制，执行复杂动作就像日常点击一个按钮一样简单，无需编写SQL或代码。



自主决策

基于系统实时反馈的数据洞察，业务人员可以自主、即时地做出业务决策并执行。



可追溯 (Traceable)

通过审计日志和数据血缘，任何一个动作的执行过程和结果都可以被完整追溯，清晰还原操作链路。



可回滚 (Reversible)

针对误操作或异常情况，系统支持关键业务动作的“撤销”与状态回滚，最大程度降低操作风险与损失。



可审计 (Auditable)

所有操作记录均存储于不可篡改的日志系统，充分满足企业内部管理审计与外部合规监管的双重要求。

核心价值 Core Values



风险控制 Risk Control

构建操作“防火墙”，通过标准化的管控流程，最大限度规避业务误操作与数据安全风险，确保系统稳定性。



信任建立 Trust Building

透明化、规范化的操作记录，让管理者、客户及监管机构对业务流程和数据资产建立充分的信心与信任。



Palantir Ontology-行为层-总结-企业的“标准化业务动词库”

行为层通过定义动作、事务、双向映射和权限审计，将静态的数据模型转变为动态的、可执行的业务流程引擎。

它就像一套**标准化的业务动词库**，定义了企业能够对数据执行的所有操作，并确保这些操作**安全、合规、可追溯**。

核心价值：解决“数据用不了、改不了”的问题



Palantir Ontology-行为层-Action 动作详解

01 / 核心定义 Definition

Action 是指可在业务对象 (Object) 上直接执行的自动化业务操作, 通过预设逻辑将系统指令转化为业务价值。

```
freezeAccount() • createWorkOrder() • sendAlert()
```

02 / 核心特点 Key Features



必须挂载对象
依赖Object Type存在
不可独立定义



可视化配置
直观定义触发条件
与执行逻辑



全链路可审计
完整记录执行日志
支持合规追溯

03 / 核心价值与作用 Value



连接数据与行动

打破数据与业务的断层, 将数据洞察转化为可落地的自动化业务动作, 让静态的数据资产真正“活”起来。



实现业务流程自动化

串联端到端的复杂业务流程, 大幅减少人工干预, 显著提升业务响应速度, 保障流程执行的标准化与高效性。



Palantir Ontology-行为层-Action 触发条件分类详解



属性变化触发(Property Change)

定义：当某个属性的值发生变化并满足预设条件时触发。

示例：当设备温度超过**80度**时，触发告警动作。



函数结果触发(Function Result)

定义：当某个函数的计算结果满足预设阈值时触发。

示例：当客户风险评分超过**90分**时，触发冻结账户动作。



时间触发(Scheduled Trigger)

定义：在特定时间点执行，或按照固定周期自动循环执行。

示例：每天**凌晨2点**自动生成并发送系统运行日报。



手动触发(Manual Trigger)

定义：由用户在系统交互界面上主动发起并执行的动作。

示例：客服人员在后台系统中手动点击“**创建工单**”按钮。



01 / 执行流程 Execution Process



触发

满足特定的业务触发条件，启动动作执行。



权限校验

严格检查执行用户是否具备执行该动作的操作权限。



参数验证

验证动作入参的完整性、格式正确性及业务有效性。



逻辑执行

执行动作定义的核心业务逻辑与计算逻辑。



数据回写

将动作产生的中间结果更新到相关数据存储中。



日志记录

完整记录动作的执行轨迹，便于后续审计与排查。

02 / 事务管理 ACID Compliance



原子性 Atomicity

动作的执行是原子的，要么所有操作全部成功，要么全部失败并回滚，无中间状态。



一致性
Consistency

确保数据在执行前后始终满足业务定义的约束条件和完整性规则。



隔离性 Isolation

并发执行的多个动作之间相互独立、互不干扰，防止数据读写冲突。



持久性 Durability

一旦动作执行成功，其结果将被永久化存储，任何系统故障都不会导致数据丢失。



权限模型 Permission Model



基于角色的访问控制 (RBAC)

用户被分配到不同的角色，角色拥有不同的权限，实现标准化、可扩展的角色权限管理体系。



基于属性的访问控制 (ABAC)

根据用户、资源对象及环境等多维度的属性动态计算并决定权限，适应复杂且灵活的业务场景。

安全机制 Security Mechanisms



细粒度权限管控

支持为系统中每一个具体的业务动作 (Action) 配置精细的权限规则，精准划定操作边界。



全链路操作审计

完整记录所有动作的执行日志，包含操作人身份、操作时间、执行内容等，满足合规与追溯需求。



端到端数据加密

对系统核心敏感数据实施存储加密与网络传输加密，从底层技术架构上保障企业数据资产安全。



数据回写Data Write-back

定义：动作执行后，将结果写回语义层或底层数据源，确保数据流转闭环。

方式：通过配置可视化的数据映射规则，实现数据的精准回传与更新。



外部系统联动External Integration

① 核心定义

动作执行时，系统自动调用外部业务系统的 API 接口，将分析结果直接转化为业务动作，无需人工二次操作。

② 实现方式

无需复杂代码开发，通过可视化界面配置 API 请求参数、鉴权信息与回调逻辑，即可快速完成系统对接。

③ 典型场景应用

- 业务闭环：基于销售分析数据，自动调用 ERP 接口创建订单。
- 即时触达：识别高价值潜在客户，自动调用短信/邮件网关发送营销通知。
- 流程协同：在审批流动作中，同步更新 CRM 系统的客户状态。



双向数据映射Two-way Sync

定义：打破单向壁垒，语义层与外部系统数据变化实时互相同步，保持一致性。

机制：基于实时数据流或自动化ETL任务，灵活配置同步策略与频率。



审计日志

■ **定义：**完整记录平台内所有动作（Action）的执行全量日志，不遗漏任何关键操作。

■ **核心记录内容：**涵盖动作的执行时间、操作的用户身份、执行动作的具体类型、输入的参数、最终的输出结果及动作的执行状态（成功/失败）。

■ **核心价值：**支持快速的线上故障与问题排查；满足企业合规性审计要求；提供关键操作的安全行为追溯能力。



数据血缘

■ **定义：**追踪数据从产生到最终消费的全过程流转路径，清晰展示数据的上下游关联关系。

■ **核心价值：**帮助业务与技术人员全面了解数据的“来龙去脉”，明确数据的质量源头，从而提升对数据资产的整体可信度。

■ **平台能力：**无需人工干预，平台自动记录并可视化展示数据间的血缘关系，降低维护成本，确保血缘的准确性与实时性。



核心定义

定义：由一系列特定动作和决策点有序串联而成的业务流转逻辑，是实现业务自动化与规范化的底层骨架。

典型场景：

企业级采购流程 · 跨部门财务审批流程 · IT系统故障自动处理流程 · 客户服务工单流转

三大设计原则



可视化

拖拽式配置，
逻辑直观清晰



模块化

功能解耦，灵
活组合



可复用

沉淀最佳实践，
快速复用

三大核心要素



节点 (Node)

流程中的具体
步骤与动作



连线 (Line)

定义节点间流
转的先后顺序



条件 (Cond)

决定流转路径
的逻辑判断



节点类型

动作节点

执行一个具体的动作，是流程中最基础的执行单元

条件节点

根据数据或系统状态判断，决定流程后续的流转方向

并行节点

同时触发并执行多个任务分支，互不干扰

子流程节点

将复杂逻辑封装，在当前流程中调用另一个独立的工作流，提升复用性

人工任务节点

触发待办通知，需要人工介入完成审批或填写信息后，流程继续流转

流转逻辑

顺序流转

按照节点在流程中的先后位置，逐个依次执行，是最基础的流转方式

条件流转

结合条件节点使用，根据不同的判断结果，流程进入不同的分支路径

并行流转

由并行网关触发，所有分支同时开始执行，最后汇聚到一起结束

循环流转

根据预设的次数或满足特定条件前，重复执行指定的节点或分支流程



01 / 核心定义与事件类型

定义：一种基于事件的“发布-订阅”设计模式，通过事件传递连接不同系统模块。

机制：事件发布者产生事件后，由事件总线负责通知所有已注册的订阅者，触发相应逻辑。



数据变化事件

对象属性更新
或关联关系变更时触
发



动作执行事件

特定业务动作
成功或失败后触发



时间触发事件

到达指定时间点
或周期性触发

02 / 核心应用价值



模块解耦 (Decoupling)

事件发布者不直接调用订阅者逻辑，系统模块间通过事件连接，互不依赖，降低维护难度。



系统灵活性 (Flexibility)

可灵活地增加或移除事件订阅者，实现业务逻辑的热插拔，快速适应业务规则变更。



可扩展性 (Scalability)

当系统规模扩大时，新增功能只需订阅相关事件，无需修改现有代码，符合开闭原则。



01 / 场景设定 & 实现步骤



核心场景目标

当工业设备的实时监测温度数值超过安全阈值时，系统自动触发流程并向维修系统推送创建工单的指令。

Step 1: 语义层建模

定义实体对象 `Equipment`，并配置属性 `temperature (数值型)` 用于承载实时数据。

Step 2: 行为层配置逻辑

- 动作定义: 创建自定义动作 `createWorkOrder()`。
- 触发规则: 配置条件 `Equipment.temperature > 80°C`。
- 执行逻辑: 调用工单系统OpenAPI，自动填充异常信息完成工单创建。

02 / 关键实现价值



流程自动化 · 降本增效

彻底替代人工巡检与手动提报工单的传统模式，大幅降低运维部门的重复性劳动成本。



毫秒级响应 · 规避风险

基于实时数据流的规则引擎即时触发，确保设备异常被第一时间处理，避免故障扩大化。



操作全记录 · 合规可溯

自动化流程的每一步（触发源、API调用记录、工单ID）均被完整记录，满足企业内控要求。



01 / 定位

自适应演化层

动态层负责管理本体模型（语义层和行为层）的持续迭代和优化，确保其能够适应不断变化的业务需求和数据环境。它是连接底层技术与上层应用的“润滑剂”，让整个系统架构保持灵活性。

02 / 核心思想

解决“规模增长不重构、平滑迭代”问题

传统企业系统往往在规模扩大时面临“推倒重来”的困境。动态层旨在构建“自生长”系统，让模型随业务同步演化，避免成为发展瓶颈。



目标

实现模型持续交付演进，
支撑企业长期发展



价值

降低维护成本，显著提升
企业敏捷性与适应性



什么是本体迭代？

本体迭代是指根据数据质量、用户反馈和业务变更，对语义层进行自动或半自动的更新，确保数据模型始终与业务发展保持一致。

触发因素 · Triggers

- **数据驱动**: 发现新数据源或模式，自动建议新增对象/属性
- **使用反馈**: 响应用户高频搜索行为，将“长尾”字段转化为标准属性
- **业务变更**: 响应新业务需求，由建模者进行定向的手动更新

迭代方式 · Methods

自动建议: 基于AI分析生成变更草案，减少人工干预 | **版本控制**: 全流程变更留痕，支持追溯与一键回滚 | **影响分析**: 自动评估变更对下游应用与查询的潜在风险

核心价值 · Core Values



持续优化

确保模型始终精准反映最新的业务现实，避免“模型滞后”



降低建模成本

通过自动化建议大幅减少手动建模工作量，提升响应速度



定义 Definition

动作优化是指根据动作的执行情况和用户反馈，对行为层的逻辑进行持续优化，确保系统行为始终高效、贴合实际业务需求。



核心价值 Core Value



提升效率

持续优化业务流程，减少等待，提升员工整体协作效率。



改善体验

让系统行为更贴合用户的实际工作习惯，降低操作门槛。



优化维度 Optimization Dimensions

性能优化

分析耗时与频率，优化底层查询逻辑，提升响应速度。

流程简化

基于用户反馈，剔除冗余步骤，让操作路径更短。

新增动作

快速响应业务需求变化，动态新增或修改关键动作。



应用示例 Scenario Examples

智能流程建议

若90%的审批流程由同一角色通过，系统可自动建议增加“自动批准”功能。

失败归因优化

针对高频失败动作，系统自动分析根因并生成优化方案，降低失败率。



01 / 定义

随着数据的不断积累和业务的发展，系统具备了自我进化的能力，可以动态地新增或调整轻量级的业务规则与推理逻辑，实现从被动响应到主动适应的转变。

02 / 核心价值



智能增强

系统决策能力随数据积累呈指数级增强



高适应性

快速适应市场变化与环境波动

03 / 三大演化方式



新增规则 · 发现新模式

基于历史数据挖掘潜在的业务规律。如：识别出某种特定的设备运行参数组合常导致故障，自动生成一条新的故障预警规则。



调整阈值 · 动态反馈

基于实时数据的反馈机制，动态优化现有规则参数。如：根据季节气温变化，自动上调或下调机房的温度报警阈值。



传递规则扩展 · 泛化能力

将特定场景下的规则逻辑进行层级扩展。如：将适用“居住在A市”的用户优惠规则，扩展到适用“整个A省”的范围。



Palantir Ontology-动态层-核心构成-兼容性保障

兼容性定义

兼容性保障是动态层维持生命力的基石。其核心职责是确保模型的任何变更都严格遵守向后兼容原则，防止变更对现有运行的应用程序和历史数据查询产生破坏性影响。

核心价值

平滑升级

支持不停机的系统升级与功能迭代，减少维护成本与时间。

系统稳定

从底层保障业务连续性，规避因模型变更引发的业务中断风险。

关键实现机制



版本化管理 (Versioning)

对本体模型实施严格版本控制，支持新旧版本共存，保证调用方的平滑过渡。



自动化迁移 (Auto-Migration)

系统自动适配并迁移存量数据与应用逻辑，大幅降低人工迁移成本与出错率。



弃用警告 (Deprecation Warning)

针对将移除的功能提前发出警告，为用户预留充足的时间进行调整与改造。



从“静态模型”到“持续进化的数字孪生”

动态层的存在，使得整个本体系统从一个需要人工维护的静态模型，转变为一个能够自我进化、自我优化的“活”的数字孪生，始终与业务发展同频共振。

■ 进化闭环：感知 → 分析 → 决策 → 执行



1. 感知

通过监控数据和用户行为，敏锐捕捉业务与环境变化



2. 分析

深度挖掘数据，识别变化规律，发现潜在优化机会



3. 决策

基于分析结果，自动生成并推荐模型变更与优化方案



4. 执行

在保障系统兼容性的前提下，安全、自动执行系统变更

■ 核心价值：打造“活”的数字资产



生命力 · 自适应环境变化

赋予系统“自我造血”的能力，无需人工干预即可适应业务场景与外部环境的持续变化，保持系统生命力。



长期价值 · 避免资产折旧

改变传统软件系统随时间推移而价值递减的规律，通过持续的自我进化与优化，实现系统价值随使用时长呈指数级增长。



传统架构的性能天花板

当业务快速发展，数据量从**GB 级跨越至 PB 级**，并发用户数从**10 人激增到 10 万人**时，传统单体架构极易受限于底层设计，导致查询响应延迟、系统稳定性下降。最终往往只能被迫“推倒重来”，付出高昂的重构成本与业务停摆风险。



解决方案：动态层的弹性架构

- **分布式架构**：底层存储与计算资源支持水平线性扩展，从容应对海量并发。
- **自适应优化**：基于实时负载动态调整资源分配与查询路径，保持系统高效运行。
- **模型解耦**：语义与业务逻辑层独立于底层实现，支持灵活迭代而不影响基础架构。



核心商业价值

确保系统随业务“平滑生长”，最大化**可扩展性**；同时有效规避“重构陷阱”，充分保护企业在系统建设上的长期**技术投资**。



01 / 定义 Definition

数据驱动建模意味着模型的构建和演进不再仅仅依赖于业务专家的先验知识，而是更多地基于对实际数据的分析和学习。

通过引入数据驱动的机制，系统具备了自我进化的能力，能自动适应业务环境的复杂变化，并持续产出更贴合真实场景的价值洞察。



实现方式 Implementation

- **模式发现：** 从海量数据中自动发现实体、关系和属性模式，减少人工干预。
- **反馈循环：** 将用户的查询和操作行为作为反馈信号，持续迭代优化模型。
- **假设验证：** 利用数据快速验证业务假设，并将验证结果快速融入模型。



核心价值 Core Value

客观性： 模型构建基于真实数据分布，避免了纯人工构建中常见的主观偏差和理想化设想。

发现新知： 能够挖掘出隐藏在复杂数据关联中、人工难以察觉的业务价值和潜在机会。



Palantir Ontology-动态层-总结-系统的“免疫系统 + 进化引擎”

动态层通过本体迭代、动作优化、规则演化和兼容性保障，赋予了整个系统持续适应和进化的能力，成为支撑系统长期稳定运行的关键支柱。



“免疫系统”

主动识别并抵御内外部环境变化带来的风险，保障系统在复杂多变的业务场景下保持健壮性与稳定性。



“进化引擎”

持续驱动系统本体迭代与规则动态演化，确保系统架构随业务规模扩大而不断优化，避免性能衰减。

核心价值：彻底解决企业级系统普遍面临的“系统僵化、规模大了就崩”难题



核心定义 / Definition

什么是动态层?

它是系统的自适应演化层，不局限于静态功能实现，而是专注于系统能力的持续优化、结构的动态调整和功能的快速迭代，是架构的“新陈代谢”中枢。

核心价值与作用

统一管理语义层（数据与概念定义）和行为层（业务逻辑与交互）的版本演化。通过标准化的版本控制与依赖管理，消除演进冲突，确保系统整体架构能够平滑、无断裂地迭代升级。

核心定位 / Positioning



系统进化引擎

敏锐感知市场趋势与业务痛点，驱动系统架构、功能模型与数据标准不断自我更新，保持技术架构对业务需求的高度适配性。



版本管理中心

作为系统的“大后方”，集中管控全链路模型、流程与交互规则的版本基线，支持灵活的版本回溯、分支管理与特性开关，降低维护复杂度。



平滑迭代保障

通过兼容性设计与自动化的验证机制，确保在功能高频迭代过程中，新旧版本平滑过渡，业务连续性不受影响，实现“无感升级”。



Dynamic Layer

打破传统架构限制
赋能业务持续进化



系统僵化 | 业务适应性差

痛点：传统系统架构固化，难以快速适应业务变化，代码级修改成本高昂。

解决方案：动态层支持模型与业务流程的配置化动态调整，无需重启。



版本混乱 | 维护成本高企

痛点：多套业务逻辑与模型版本并存，缺乏统一标准，管理困难且易出错。

解决方案：提供统一的版本管理机制，支持版本追踪、回滚与并行发布。



迭代风险 | 稳定性挑战大

痛点：系统频繁的功能迭代，容易导致服务中断或旧逻辑失效，影响稳定性。

解决方案：内置平滑迭代能力，严格保障新旧逻辑的向后兼容性与无损切换。



本体迭代

版本管理

管理语义层模型的版本，确保演进过程中的可追溯性与一致性。

向后兼容

确保模型的新版本兼容旧版本的数据结构与接口，降低系统维护成本。



动作优化

流程优化

持续优化行为层的业务逻辑与执行 workflow，提升操作流转效率。

性能监控

全方位监控核心动作的执行性能，快速识别并解决潜在瓶颈。



规则与推理演化

规则更新

灵活响应业务变化，快速更新推理逻辑与业务规则，适配新场景。

模型优化

基于实时运行数据的反馈，持续迭代优化推理模型的准确性。



版本管理

定义

为每个语义层模型和行为层流程分配唯一的版本号，建立清晰的标识体系，确保系统中的每个实体都可识别。

核心作用

构建完整的历史档案，便于在系统升级或出现异常时，能够快速定位到特定版本，并支持安全、精准地追溯与回滚操作，降低维护风险。



迭代策略：全生命周期管理

新增：扩展本体能力，灵活添加新对象、属性、关系或业务动作。

修改 / 删除：适配业务变化，对现有模型流程进行优化调整或清理不再使用的冗余资产。



版本控制：保障协作与稳定

分支管理：支持并行开发，互不干扰。 **合并与回滚：**合并开发成果上线；遇异常时一键回滚至安全版本。



01 向后兼容 (Backward Compatibility)

核心定义：新版本的模型和流程能够完全兼容旧版本的数据存储结构和外部调用方式，保障历史业务逻辑和数据资产的连续性与可用性。

数据迁移

内置自动化脚本，将旧版本存量数据无损转换并迁移至新版本，大幅降低人工维护成本。

接口兼容

严格遵循“增加而非替换”原则，保持对外API接口参数与通信协议的向下兼容，确保第三方集成应用无缝对接。



02 平滑扩容 (Seamless Scaling)

核心定义：系统支持在完全不中断线上业务服务、不影响用户体验的前提下，灵活完成底层架构升级、功能扩展及性能扩容。

无停机部署

采用容器化与滚动更新技术，实现新版本的无感热部署，确保服务在升级过程中始终保持高可用性。

灰度发布

支持按用户比例或特定维度分批推送更新，在验证稳定性与业务逻辑无误后，再逐步扩大覆盖范围。



Palantir Ontology-动态层-自适应进化与数据驱动优化



自适应进化 Adaptive Evolution

定义 Definition

系统能够根据实时反馈数据和历史经验，自动动态地调整模型参数与业务执行流程，无需人工干预即可应对环境变化。



模型推荐

分析用户画像与历史行为模式，自动匹配并推荐当前场景下效果最优的AI模型。



流程优化

基于流程执行的时间、成本和质量数据，自动优化节点连接方式，消除冗余步骤。



数据驱动优化 Data-Driven Optimization

定义 Definition

以客观量化的数据分析结果为决策基石，通过系统性的诊断与迭代，持续提升系统的整体运行效率与产出价值。



性能监控

全链路监控关键动作与流程的响应时间与资源消耗，确保系统稳定运行。



瓶颈分析

基于数据分析定位性能短板与流程阻塞点，提供针对性优化方案。



持续学习

利用机器学习算法不断挖掘数据价值，实现系统模型能力与决策精度的自我迭代。



01 / 管理语义层

职责：负责语义层模型的版本迭代和演化，确保模型逻辑的一致性与可维护性。

协同：建立平滑过渡机制，确保语义层的结构变化不会对上层应用造成破坏性影响。



02 / 优化行为层

职责：持续优化行为层的业务逻辑流程，并建立完善的性能监控体系。

协同：通过技术手段保障执行效率，并确保系统在高并发场景下的稳定性与可靠性。

03 / 闭环协同机制



语义层变化 → 行为层调整

当模型定义发生变更时，动态层即时触发行为层的流程重绘与适配，减少人工干预成本。



行为层反馈 → 驱动语义优化

基于行为层的运行数据与用户操作反馈，动态层分析并驱动语义层模型的持续迭代与重构。



动态层：全生命周期协调中枢

统筹管理从模型定义到行为落地的全链路迭代过程，打破孤岛，形成自洽的正向闭环。



场景设定：构建反馈驱动闭环

目标：利用用户的实时交互反馈，自动迭代优化推荐模型，解决静态模型“过时”问题。

关键实现步骤



Step 1: 行为层触发 · 原始信号

用户对推荐内容进行点赞、点踩或停留时长反馈，生成数据输入源。



反馈收集

清洗、聚合海量用户行为数据，形成训练样本。



模型训练

调用算法，基于新样本增量更新模型参数。



生产部署

将新模型无缝上线，实时服务线上流量。

实现价值与效果



持续自我优化

突破静态模型的局限性，让推荐系统具备“成长性”，随用户兴趣迁移而进化，始终保持推荐准确性。



极致个性化体验

千人千面的效果不断增强，能更敏锐地捕捉并满足用户的长尾需求与实时偏好。



全流程自动化

无需人工介入即可完成模型的迭代和上线，大幅降低算法运营成本，提升迭代效率。



核心价值

01. 自适应进化系统能够持续适应业务变化，根据市场趋势和用户行为自动调整策略，保持敏捷性。

02. 平滑迭代确保系统在版本更新与功能迭代过程中不中断业务服务，保障高可用性与稳定性。

03. 数据驱动基于全链路数据反馈持续优化系统逻辑，实现决策智能化，避免主观臆断的偏差。



面临挑战

01. 技术复杂性实现自适应进化与数据驱动的底层架构难度较高，对技术团队的架构能力有极高要求。

02. 数据质量依赖数据驱动的优化效果高度依赖高质量、高时效的反馈数据，数据噪音会直接导致决策偏差。

03. 治理成本攀升随着系统动态性增强，需要建立更完善的版本管理与治理机制，相应的运营与维护成本会增加。



01 / 语义 Semantic

1. 定义 (Definition)

定义 `Equipment` 对象，将物理实体数字化，并明确包含 `temperature`（温度）、`status`（状态）等关键属性。

2. 分析 (Analysis)

通过预设的业务逻辑 `Function` 实时计算，判断出设备的 `riskScore`（风险分）是否超过设定阈值。



02 / 行为层 Kinetic

3. 触发 (Trigger)

一旦监测到风险评分超过阈值，立即自动触发 `CreateMaintenanceTicket`（创建维修工单）动作。

4. 执行 (Execution)

动作自动执行，将工单信息实时回写到企业外部系统（如ERP或工单管理系统），实现业务流转。



03 / 动态层 Dynamic

5. 反馈 (Feedback)

系统持续监控并分析所有动作。当发现某类设备的“创建维修工单”动作高频发生时，触发根因分析。

6. 优化 (Optimization)

若发现是特定型号的设计缺陷，则更新语义层增加 `modelVulnerability` 属性，并在行为层增加 `ProactiveReplacement`（主动更换）策略，实现闭环优化。

这一闭环完整展示了从“数据定义与分析”到“业务自动执行”，再到“持续优化”的端到端价值实现过程，确保业务逻辑随真实世界动态演进。

05 Palantir Ontology的五大原子要素

object、property、link、action、function



Object(对象)

业务中的核心实体，如客户、设备，是构建业务模型的基础骨架。



Property(属性)

描述对象特征的具体字段，如客户的姓名、联系方式或设备的实时温度。



Link(关联)

定义不同对象之间的业务逻辑关系，例如“客户发起交易”或“设备属于站点”。



Function(函数)

封装的计算逻辑，基于对象的属性和关联关系，进行复杂指标的计算和分析。



Action(动作)

可在对象上执行的自动化业务操作，例如“设备重启”或“发送营销邮件”。



Palantir Ontology-基础模型-Object, Link, Property



Object Type | 业务世界的“名词”

定义:代表业务领域中的核心实体，是模型的基石。例如：Customer (客户), Product (产品), Order (订单)。

作用:将分散在不同系统中的数据（如基本信息、订单记录）聚合到一个统一的业务实体上。



Property | 对象的“特征”

定义:描述 Object Type 的具体特征或数据字段。例如：`name` (姓名), `price` (价格), `stockQuantity` (库存)。

作用:属性是对象承载的具体数据，是进行后续业务分析、报表计算和数据洞察的基础。



Link Type | 对象间的“关系”

定义:定义不同 Object Type 之间的关联。例如：`Order` 由 `Customer` “Placed By”，`Order` 包含 “Contains” `Product`。

作用:将孤立的对象连接成有意义的业务网络(知识图谱)，直观表达复杂的业务关系。

Customer (对象)

属性: name, email, phone



Placed By (链接)

Order (对象)

属性: orderId, date, amount



Contains (链接)

Product

price, stock



Palantir Ontology-动态能力-Function & Action Type



Function (函数): 对象的“计算能力”

定义: 封装可复用的业务计算逻辑, 通常作用于一个或多个属性, 返回计算结果, 是对象的“虚拟属性”。

作用: 计算派生指标与复杂逻辑。例如, 基于客户所有订单记录, 动态计算其生命周期总价值 (LTV)。



Action Type (动作类型): 对象的“行为能力”

定义: 定义可在一个或多个对象类型上执行的业务操作, 可理解为对象的“方法”或“行为”。

作用: 实现“分析”到“行动”的闭环。例如, 当检测到风险订单时, 自动调用cancelOrder()动作执行操作。



数据 → 分析 (Analysis)

Customer 对象
→ calculateLifetimeValue()
→ 生成动态计算结果



洞察 → 行动 (Action)

Order 对象 → cancelOrder()
→ 执行操作并同步外部系统

场景：创建一笔新订单 (New Order Creation)



定义基础模型

Object · Link · Property

- **Object Type:** `Customer`, `Product`, `Order`
- **Property:** `Customer.name`, `Product.price`, `Order.status`
- **Link Type:** `Order` Placed By `Customer`; `Order` Contains `Product`



执行计算逻辑

Function:

`calculateTotalAmount()`

当一个订单包含多个商品时，系统自动触发计算函数：

1. 遍历该 `Order` 通过 `Contains` 链接指向的所有 `Product` 对象。
2. 聚合读取所有 `Product` 的 `price` 属性值。
3. 执行求和逻辑，自动计算并填充订单总金额。



触发业务操作

Action Type:

`submitOrder()`

当用户确认提交订单时，触发动作链：

- ✓ 状态变更：更新 `Order.status` 为“已提交”。
- ✓ 系统集成：调用外部 API 将数据同步至 ERP 系统。
- ✓ 库存扣减：触发 `deductInventory()`，更新对应 `Product.stockQuantity`。

01/核心定义

定义：业务中的核心实体，是所有其他概念的载体。
Object代表了业务中不可再分的最小单元，它是数据模型的基石，所有的关系、行为和分析都围绕它展开。

典型业务对象示例：

`Customer` · `Product` · `Equipment` · `Transaction`

02/核心特点



逻辑视角：实体表抽象

对用户而言，一个ObjectType可直观理解为一张包含特定业务属性的逻辑实体表。



物理视角：分布式多形态存储

底层并非简单映射为单一数据库表，可根据业务需求灵活组合图节点、列式存储等多种形态。



集成能力：多源数据融合

支持将来自不同业务系统、不同格式的多源异构数据，统一映射并融合为标准的对象模型。

Object 的两层核心概念

Object Type (对象类型 / 模板)

抽象的业务分类标准。例如：「员工、生产设备、销售订单、医院患者」，是定义数据“长什么样”的蓝图。

Object Instance (对象实例)

具体存在的、唯一的业务实体。例如：员工“张三”、设备“A001”、订单“SO-2026-001”，是数据“具体是什么”的体现。

业务数据映射实操流程

1. 源数据接入：将企业内部 ERP 表、IoT 库、Excel、API、日志等异构数据源接入 Palantir 平台。

2. 定义对象蓝图：进入 Ontology Builder，新建 Object Type，为其命名并定义唯一的“业务主键”（如员工 ID、设备编号）。

关键映射规则：将外部源表的业务主键映射为 Object 的全局唯一 ID；将源表中的实体类型，对齐为平台本体中的一个 Object Type。

→ **最终封装：**平台自动将结构化、非结构化、时序数据统一封装为标准的 Object 语义实体。

3. Object 底层到底怎么存储？是不是一张表？——不是单张 MySQL 业务表！

元数据表层 (Metadata)

独立存储 ObjectType 的核心定义：ID、业务名称、所属业务域、业务描述以及数据源的映射规则。

图节点存储层 (Graph Node)

每一个 Object 实例 = 知识图谱中的一个 Node 节点。节点自带**全局唯一 UID**，是所有关联、属性、动作的唯一数据锚点。

实体分片存储层 (Sharded)

- 基础固定字段 → 列式宽表
- 动态扩展字段 → 键值稀疏存储
- 实体全量快照 → 时序分区表

逻辑视角：你可以将其视为一张「业务实体表」，屏蔽底层复杂性；

物理视角：元数据 + 图节点 + 列式宽表 + 稀疏 KV + 时序表的联合存储，彻底打破传统固定表结构限制。



举个例子：ObjectType 定义为「生产设备」，映射来源为 IoT 设备台账表，最终将设备编号作为全局唯一 UID 锚点进行统一存储。



01/命名规范



使用业务术语

如Customer、Order，直观反映业务含义，避免使用Table1、Table2等无意义名称。



使用单数形式

类名代表一个实体概念，应使用单数形式，如Customer而非Customers，以符合直觉。



使用驼峰命名法

采用大驼峰（PascalCase）命名复合对象，如CustomerOrder，保持代码风格的一致性。

02/设计原则



高内聚，低耦合

一个对象类型应专注于一个核心业务概念，减少与其他对象的依赖，提升代码的独立性和复用性。



单一职责原则

一个对象类型在业务逻辑中只负责一个明确的功能，避免职责过重，降低维护和理解成本。



面向未来的可扩展性

在设计初期就应充分考虑未来的业务发展需求，预留合理的扩展空间，避免频繁重构底层逻辑。



01/数据映射

定义：将数据源中的数据映射到对象实例，建立底层数据与上层业务对象的直接关联。

方式：通过配置灵活的映射规则，将数据源的原始字段精准映射到业务对象的属性中。



02/实时同步

定义：当底层数据源发生增删改操作时，平台自动触发同步，保证业务对象数据实时更新。

机制：支持通过流式计算或定时ETL任务实现，满足不同场景的时效性要求。



03/多源融合

打破数据孤岛，支持一个业务对象类型从多个异构数据源映射数据，构建统一的实体视图。

应用示例：客户360°画像

一个“客户”业务对象，可以同时关联：

- CRM系统：获取基础身份与销售跟进记录
- 订单系统：获取消费频次、客单价等交易数据
- 客服系统：获取服务工单与投诉记录

从而在单一对象上汇聚所有信息，实现全景洞察。

核心定义Definition

描述对象特征的字段。

示例：

`Customer.name|Product.price|Equipment.temperature`

类型分类Categories

基础属性

直接从数据源映射而来的原始数据

派生属性

通过函数计算或关联对象属性得到

核心特点KeyCharacteristics



必须挂载到对象(ObjectType)

属性不能独立存在，它是描述对象的维度，必须依附于具体的对象类型而存在。



支持丰富多样的数据类型

内置支持文本、数值、日期、布尔、枚举等多种基础类型，满足业务场景需求。



可定义严格的数据约束

支持配置非空、唯一性、数值范围、正则表达式等约束，保障数据的准确性。



01 / 核心规则

Property 属性本身不能独立存在，它必须作为依附体，**100% 挂载归属到某一个 ObjectType 之下。**

就像“发动机转速”必须归属于“汽车”这个物体，不能脱离主体单独存在。



02 / 本体构建器实操步骤

STEP 1 · 基础准备

先创建好对应的 ObjectType (如“生产设备”)，在该对象详情页内点击「新建 Property」按钮。

STEP 2 · 定义核心配置

设置名称、选择数据类型 (文本/数值/布尔/枚举等)、并设定读写权限与空值策略。

STEP 3 · 数据源映射

将外部 IoT 数据表中的物理字段 (如“温度”) 直接映射到当前 Property 上，实现数据关联。

STEP 4 · 完成挂载

保存配置后，该属性将自动归属到当前 ObjectType 下，成为其结构化数据的一部分。

提示：若需支持实时监控，可开启“时序采集”功能。



03. 底层存储逻辑设计



元数据集中存储

维护单独的元数据表，统一记录属性归属的ObjectType、数据类型、业务约束及与物理表的映射规则，保障一致性。



实例值：双模式混合存储

- 常用固定属性：存入关联对象的列式宽表，读写高效
- 动态/稀疏属性：以 [ObjectUID + Key + Value] 键值对形式存储，扩展性强



时序属性：独立分区存储

实时温度、负荷、流量等高频变化属性，单独存入时序数据库分区，保留全量历史快照，支持快速时序检索。



04. 场景举例：生产设备 Object

核心对象：生产设备 (Production Equipment)

设备型号

TYPE: 文本 (Text)
固定属性 / 基础信息

实时温度

TYPE: 数值 · 时序
高频变化 / 历史快照

维保到期日(Date)

用于生命周期管理

运行状态(Enum)

在线 / 离线 / 故障

基础定义

基础属性是直接从原始数据源映射而来的基础数据单元，是数据资产中不可再分的原子信息，反映了实体最基本的客观特征。

核心配置项

- **数据源字段**: 明确关联底层数据库表中的物理字段，确保数据映射的准确性。
- **类型匹配**: 设定与源数据兼容的逻辑类型，以支持后续计算与分析。
- **数据约束**: 设置非空、唯一、长度限制等规则，保障数据质量。

支持的数据类型体系

文本(Text)

用于描述性信息，如：产品名称、用户昵称、商品描述。

数值(Number)

用于定量指标，如：商品价格、用户年龄、库存数量。

日期/时间(Date/Time)

记录时间维度，如：用户生日、订单创建时间、发布日期。

布尔(Boolean)

用于二元状态标识，如：账号是否活跃、订单是否支付。

枚举(Enum)

有限离散值，如：订单状态(待支付/已发货)。

地理位置(Geo)

用于定位，如：经纬度坐标、IP归属地。



什么是派生属性？

并非直接存储于数据库，而是通过对基础数据进行函数计算或关联其他对象的属性间接得到的动态属性。它是数据资产化的关键一环。

三大核心价值



实现复杂计算

支持多维度公式运算，轻松实现折扣价、复合风险评分等复杂逻辑。



简化业务逻辑

将复杂逻辑封装为属性，降低系统耦合度，提升代码复用性和维护效率。



提高查询效率

通过预计算和缓存机制，避免高频场景下的实时计算，显著提升查询性能。

关键业务场景落地



金融风控领域

整合客户征信、消费记录及行为特征，实时计算客户综合风险评分，辅助信贷审批决策与贷后风险预警。



电商推荐领域

基于用户画像与商品标签，动态计算个性化推荐指数，优化“千人千面”的商品排序逻辑，提升营销转化率。



智能制造领域

实时聚合分析设备传感器数据，动态计算设备健康度指标，助力工厂实施预测性维护，降低停机风险。

八大约束类型



空值约束

定义字段值是否允许为空(NULL)



唯一性约束

保证属性值在表中必须唯一



取值范围约束

限定数值或日期的合法区间



格式约束

验证文本格式, 如邮箱/URL/手机号



枚举约束

值必须是预设枚举集合之一



长度约束

限制文本字段的最大字符数



只读约束

一旦创建, 属性值不可修改



依赖约束

属性值依赖于其他属性或表

核心价值与作用



保证数据质量(DataQuality)

通过定义严格的属性规则, 从源头拦截和过滤无效、错误、不一致的数据, 确保进入系统的数据准确、完整且符合规范。



确保业务逻辑(BusinessLogic)

将复杂的业务规则转化为技术约束, 自动化强制执行业务规则, 防止因人为操作失误导致的逻辑冲突, 保障业务流程稳健运行。



Palantir Ontology-Property-依赖约束与联动约束



依赖约束(Dependency)

定义: 属性A的有效性（如必填性、可选性）直接依赖于属性B的值。这是一种逻辑上的前置条件关系。

示例: 若“支付方式”选择“信用卡”，则“信用卡号”字段变为必填项。



联动约束(Cascading)

定义: 属性A的值发生变化时，系统自动触发属性B的值进行更新或重算，以保持数据逻辑一致性。

示例: 当用户修改购物车中的“商品数量”时，系统自动重新计算并更新“商品总价”。



规则引擎(RuleEngine)

将业务约束转化为可配置的规则，统一管理并高效执行复杂的依赖与联动逻辑，降低硬编码维护成本。



事件驱动(Event-Driven)

以“属性值变更”为核心事件，触发订阅者进行实时的依赖检查或联动计算，确保数据状态的即时同步。

核心定义

定义：用于定义业务场景中，不同对象（Node）之间的关联与互动逻辑。

典型示例：

- 客户(Customer) **下单(places)** 订单(Order)
- 员工(Employee) **任职于(works in)** 部门(Department)

核心特点

- **底层存储**：作为图数据库的“边(Edge)”，实现毫秒级的图遍历与关联查询。
- **类型灵活**：天然支持一对一、一对多、多对多等复杂关系形态。
- **携带信息**：关系本身可附加属性，如记录“下单时间”或“关系创建人”。

核心价值与作用

- **构建关联网络**：将离散的业务实体连接，还原真实的业务全景。
- **赋能复杂分析**：支持深度图遍历查询，例如：“查找某客户的所有订单及相关商品”，轻松挖掘数据价值。



01 / 基础概念

Relation (关系类型)

定义实体之间的关联模板。例如：「属于」、「部署于」、「负责」、「归属」等通用关系类别。

Link (关联实例)

基于 Relation 模板创建的、连接具体两个 Object 的实际数据记录。例如：「张三」→「属于」→「研发部」这条具体的连线。



02 / 本体构建器配置流程

1. 准备工作：在项目中已创建好待关联的两个 ObjectType，如「员工」与「部门」。
2. 新建关系：在构建器中创建 Relation，自定义名称（如“隶属于”）。
3. 设定方向：指定起始对象（如“员工”）与目标对象（如“部门”）。

★ 核心配置：基数 (Cardinality) ★

- **多对一**：多个员工隶属于同一个部门 | **一对多**：一个部门包含多个员工
- **多对多**：如“客户”与“银行账户”之间的关联

进阶配置：可开启双向关联查询、设置关系有效期及添加附加属性。



Palantir Ontology-LinkType -关系与存储



三种关系具象解释

■ 多对一 (Many-to-One)

多名员工 → 同一个部门 (如所有研发人员归属研发部)

■ 一对多 (One-to-Many)

一个厂区 → 多台生产设备 (如A工厂包含10条产线)

■ 多对多 (Many-to-Many)

供应商 <-> 采购项目 (双向关联, 灵活扩展)



Link 底层怎么存储

1. 关系元数据 (Schema)

定义 Relation 类型、两端 Object 类型、基数约束及连接方向。

2. 关联实例 (Instance)

以图数据库 Edge (边) 存储。

结构: 源UID → 关系类型 → 目标UID + 属性(时间/状态)。

不是传统数据库“外键”

是独立的边集合, 支持动态新增、临时关联及高性能的全链路穿透查询。



逻辑模型举例

Object 1: 员工 张三

Link: 【隶属于】

Object 2: 研发部



Palantir Ontology-LinkType-一对一关系建模



核心定义

概念: 在实体关系模型中，一个对象的实例只能与另一个对象的唯一实例建立关联，反之亦然。

示例: 在系统中，一个“自然人(Person)”只能拥有一本有效的“护照(Passport)”。



典型应用场景

- 身份验证: 公民个人与唯一的身份证号码一一对应
- 商业合规: 企业法人实体与营业执照注册号一一对应



配置关键要素



关系类型选择

在数据建模工具中，明确将关系类型设置为“一对一(One-to-One)”。



对象端点定义

准确指定关系的“父级”与“子级”对象，明确数据关联的起始和目标实体。



基数约束设置

确保关系的两端基数均为“1”，严格限制每个实例只能关联一个对应实例。



01/核心定义

指在实体关系建模中，一个对象的单个实例可以与另一个对象的**多个实例**建立关联，是数据模型中最基础、应用最广泛的关联类型。

经典示例：Customer(1)→Order(N)

配置三要素：

- **类型**：关系属性中明确选择“一对多”类型。
- **对象**：指定“起始对象”(一/父级)与“目标对象”(多/子级)。
- **基数**：起始端基数为`1`，目标端基数设为`N` (多)。

02/典型应用场景



客户与订单管理

在电商或ERP系统中，单个客户账户下可创建并关联多个订单记录，实现按客户维度进行订单的聚合与管理。



企业组织架构管理

在人力资源系统中，一个部门作为组织单元可容纳多名员工，且每位员工在同一时间仅归属于一个具体部门。



01 核心定义&应用场景

什么是“多对多”关系？

在数据模型中，当多个对象实例可以与另外多个对象实例相互关联时，即为多对多关系。它是连接两个独立实体集的桥梁，消除了数据冗余并保证了关联的灵活性。

典型应用场景

学生&课程

一个学生可选择多门课，一门课也可被多名学生选择。

演员&电影

一名演员可出演多部电影，一部电影也有多名演员参与。

02 关键配置步骤



选择正确的关系类型

在关系属性中明确将类型设置为“多对多(Many-to-Many)”，系统将自动生成关联中间表。



指定关联的两端对象

确认关系连接的两个实体对象，例如“学生”与“课程”，并定义关系的业务名称。



确认双向基数为“多”

确保关系的两个端点基数都被设置为“多(*)”，以保证数据关联的灵活性和正确性。



Palantir Ontology-LinkType-双向关联关系



核心定义Definition

关系不再是单向的，支持从任意一端进行双向遍历。例如：在订单系统中，既可以从**Customer(客户)**快速定位到**Order(订单)**，也能反向从订单直接追溯到所属客户。



应用价值Value Proposition

查询极度灵活

打破数据孤岛，无需预先设计关联方向，支持从任意节点开始探索。

简化业务逻辑

无需手动维护反向索引，大幅降低开发成本，减少冗余代码。



底层原理Underlying Principles



反向关系自动生成

当创建任意方向的关联时，系统会在内部自动构建并维护一条逻辑上的反向关系索引，保证数据的一致性。



图数据库原生支持双向遍历

基于图结构存储，利用邻接表优势，双向遍历性能与正向查询几乎一致，无性能损耗。



智能查询路径优化

内置查询优化器会根据数据分布情况，自动选择从正向还是反向开始遍历，确保最优的执行效率。



01.关系与对象(Relationship&Object)

- **连接桥梁**: 关系本质上是连接不同对象实体的纽带, 确立了实体间的逻辑关联, 让孤立的对象形成网络。
- **强类型约束**: 每一种关系都定义了严格的“定义域”(起始对象类型)与“值域”(目标对象类型), 确保关联的准确性与数据的一致性。

02.关系与属性(Relationship&Property)

- **关系的自有属性**: 关系不只是一个连接符, 本身也可作为一个实体承载属性, 例如关联的“创建时间”、“关联权重”等元数据。
- **属性依赖与推导**: 一个对象的属性值可以依赖其关联对象的属性进行动态计算, 打破了数据孤岛限制。

03.核心联动应用场景



跨实体精准查询(Query)

利用关系作为检索路径, 在不同对象集合间进行关联查询。
示例: “查找所有由‘张三’创建的订单”, 通过“用户-创建-订单”的关系路径, 快速定位符合条件的目标数据。



基于关系的动态计算(Calculation)

系统自动遍历对象间的关联关系, 对目标属性进行聚合计算。
示例: “计算客户A的年度订单总金额”, 通过遍历“客户-拥有-订单”的所有关系, 动态累加所有关联订单的金额属性值。



定义与示例

Function函数及Interface是语义层中基于属性和关联关系，进行逻辑计算与分析的核心单元，它充当了连接原始数据与业务决策之间的“逻辑桥梁”。

应用示例：基于设备访问频次、敏感操作记录等属性计算风险评分；整合多点定位数据与时间戳，分析人员或物品的活动轨迹。



挂载与依赖逻辑

函数必须“挂载”在特定的**ObjectType(对象类型)**上。计算逻辑可以灵活依赖：对象自身的基础属性、关联对象的属性，甚至是其他已定义的函数。



底层技术实现

在建模阶段，函数逻辑会被编译为**AST(抽象语法树)**以确保逻辑的正确性；在运行阶段，由内置的高性能规则计算引擎统一调度和执行计算。



Palantir Ontology-Function-定义与能力



Function 编写在哪里？

写在 Ontology Builder 本体构建器内部，无需外部 IDE 或后端部署。直接在平台内通过可视化规则或表达式编辑框进行编写。



Function 挂载在哪儿？

需绑定在特定的 **ObjectType** 上。例如：将其挂载在「生产设备」Object 下，使其成为该对象的衍生计算指标。



Function 依赖什么？

支持依赖：自身 Object 的 Property 属性、通过 Link 关联的其他 Object 属性、以及平台内已定义的其他 Function 计算结果。

支持的核心运算能力

公式计算

条件判断

加权评分

聚合统计

时序均值/极值

关联穿透取值



03. 底层背后怎么实现

建模时

平台自动将用户输入的表达式，编译为AST（抽象语法树）结构进行存储与解析。

运行时

由Palantir内置的高性能规则计算引擎，根据配置策略进行实时或定时的任务调度与执行。

计算模式支持

- 实时触发计算 (Event-driven)
- 定时批量计算 (Scheduled)
- 按需查询计算 (On-demand)



04. Function 怎么存储

元数据存储

核心信息：函数ID、归属ObjectType、表达式逻辑文本、关联依赖 (Property/Link)、返回值类型定义等。

计算逻辑存储

以“配置化规则+AST抽象语法树”形式保存在规则引擎中，不编译为系统硬代码，支持灵活修改。

计算结果存储

结果可被缓存为“虚拟Property”，像原生属性一样支持查询、筛选、聚合，也可直接被 Action 逻辑调用。



05. 具象：设备风险分

挂载在 **【生产设备】** Object 上的函数
故障风险分 = (实时温度 * 0.6) + (维保逾期扣分 * 0.4)

核心价值

整合设备自身实时监测数据与维保流程数据，通过逻辑运算动态生成“风险指标”，作为后续触发预防性维护Action的关键判断依据。



编写位置

- **本体编辑器**: 在平台提供的本体建模工具中, 直接为特定的对象类型 (如“商品”、“订单”) 编写自定义函数。
- **强挂载关联**: 函数无法独立存在, 必须显式关联并挂载到具体的对象类型上, 作为该对象的计算属性或行为方法。



语法规范

- **标准化表达式**: 使用平台内置的表达式语言, 支持类JavaScript的简洁语法, 降低上手成本。
- **变量与关联引用**: 支持直接引用当前对象的基础属性, 也支持跨层级引用关联对象的属性。
- **内置函数库支持**: 内置数学运算、字符串处理、日期转换等高频通用函数, 无需重复造轮子。



代码示例

计算商品最终折扣价

```
//逻辑: 商品原价×(1-折扣率)  
price*(1-discountRate)
```



Palantir Ontology-Function-挂载对象与触发时机



挂载对象(MountingObject)

- 定义:** 函数逻辑必须显式挂载到特定的对象类型上, 不能独立存在。
- 作用:** 使函数能够直接访问并关联该对象类型的所有属性、关联关系和方法。



性能优化: 缓存机制

- 结果缓存:** 计算结果自动存储, 大幅提升高频查询场景下的响应速度。
- 智能失效:** 当依赖的源数据发生任何变化时, 系统会自动将旧缓存标记为失效。



核心机制: 三大触发时机

1. 依赖属性变化时(On-Change)

当函数逻辑中引用的任何属性或关联关系被更新时, 函数自动触发并进行重新计算, 确保数据实时同步。

2. 主动查询时(On-Query)

当用户或系统在界面、报表或API中主动查询该派生属性时, 若缓存未命中, 函数立即进行实时计算并返回最新值。

3. 定时任务触发(Scheduled)

对于对实时性要求不高但计算复杂的场景, 支持配置固定的时间间隔 (如每日/每周), 批量执行计算并更新缓存。



01/依赖项Dependencies



自身属性(OwnAttributes)

函数可以直接引用当前挂载对象的基础属性，作为计算的直接输入源。



关联对象属性(RelatedAttributes)

支持跨对象引用，可获取并使用通过关联关系连接的其他对象的属性值。



其他函数(OtherFunctions)

支持嵌套调用逻辑，直接引用系统内其他函数的最终计算结果。

02/运算逻辑LogicOperations



基本运算

支持加、减、乘、除、取模等基础算术运算，满足日常数值处理需求。



条件判断

内置IF-ELSE分支判断逻辑，灵活处理不同条件下的差异化计算规则。



循环操作

支持对关联对象集合进行循环遍历，逐一对集合内的元素进行迭代计算。



聚合函数

提供SUM(求和)、AVG(平均值)、COUNT(计数)等统计函数，实现快速数据聚合。



Palantir Ontology-Function与派生属性联动实现

核心联动逻辑



派生属性：容器

作为数据的载体，其最终显示的数值完全由关联的函数计算得出。



函数：计算引擎

定义具体的业务计算逻辑与规则，是驱动派生属性产出数据的核心动力。

标准实现流程

01.编写函数在本体编辑器中定义计算逻辑。

02.创建属性在数据模型中新建派生属性字段。

03.关联映射将新属性与已编写的函数进行绑定。

04.配置缓存设置缓存策略以平衡性能与实时性。



代码示例：计算商品折后价

定义一个用于计算折扣的函数`calculateDiscountPrice()`，并将数据模型中的派生属性`discountPrice`关联到该函数，系统即可自动计算并展示最终的折扣价格。

核心定义

Action代表在对象（Object）上可执行的**自动化业务操作**，是语义层将数据能力转化为业务能力的关键载体。

典型方法示例：

```
freezeAccount()//冻结账户  
createWorkOrder()//创建工单  
sendAlert()//发送预警通知
```

核心特点

- 对象绑定**：Action不能独立存在，必须挂载（Mount）到具体的ObjectType上，具有明确的归属关系。
- 可配置与可审计**：支持可视化配置触发条件与执行逻辑；所有调用均生成全链路审计日志，满足合规要求。

核心价值

打通“数据洞察”到“业务行动”的最后一步，消除数据孤岛，实现业务流程的**端到端自动化闭环**。

01/ActionType(动作类型)

定义：绑定在特定对象类型上的业务操作，定义了用户可对该对象执行的动作范围。

示例：**Equipment**对象的**CreateMaintenanceTicket**

02/Triggers(触发条件)

定义：动作的执行需满足的前置条件，用于确保业务逻辑的合规性和数据的完整性。

示例：当设备状态异常时，即**Equipment.riskScore>80**时自动触发维修工单创建。

03/ExecutionLogic(执行逻辑)

定义：动作内部封装的具体业务步骤。通常包含对对象本身的状态更新，以及对上下游外部系统的调用或通知，以完成完整的业务闭环。



核心：绑定在哪里？

Action 必须挂载到
ObjectType 上

动作不是孤立存在的，必须归属于某个具体的业务对象。

例：

- 绑定在「生产设备」Object
- 绑定在「客户账户」Object

逻辑：谁的对象，谁就承载对应的业务动作。



触发条件来源（关键）

触发规则的数据源头：

1. 自身 Object 的 Property 属性值
2. 挂载的 Function 函数计算结果（最常用）
3. 关联对象(Link)的属性/函数结果

典型链路：

原始数据(Property) → 计算风险等级
(Function) → 达到阈值 → 触发 Action



Action 有哪些类型

状态变更

告警推送

工单下发

API 接口调用

流程审批

数据冻结/通知

04 触发流程

- 设备实时温度 Property 更新入库
- 触发 Object 绑定 Function, 自动重算风险评分
- 规则引擎判断: 计算结果 > 预设阈值 (如 80 分)
- 自动触发同 Object 挂载的 Action 动作
- 执行动作: 生成工单 / 推送告警 / 变更设备状态

05 底层存储

■ 动作元数据 (Metadata)

存储归属 ObjectType、触发条件表达式、执行逻辑、权限配置与执行模板。

■ 流程规则存储

动作流转逻辑、前置触发条件与后置处理逻辑, 托管于 workflow 引擎与规则引擎中。

■ 执行流水存储

每次触发的时间、原因、操作人、结果与日志存入“动作执行时序流水表”, 全链路可审计。

06 具象场景

绑定 Object: 工厂核心生产设备

触发条件: Function 计算出的“设备故障风险分” ≥ 80 分

执行 Action: 1. 自动在ERP系统创建维保工单

2. 向运维群推送高风险告警

3. 设备状态变更为「高风险」

06 Palantir Ontology构建流程及应用案例



01

定义 Object (对象)

从数据源映射出核心业务实体，确立知识图谱的基石。

输入:数据源 (表/CSV等)

输出:Object Type
(如: 图书、作者)

02

添加 Property (属性)

为对象赋予描述性特征，丰富实体的属性维度。

输入:源字段 / 手动定义

输出:对象的属性列表
(如: 书名、出版时间)

03

建立 Link (关联)

定义实体间的业务逻辑联系，编织网状的知识结构。

输入:对象间关联字段

输出:知识图谱网络
(如: 作者 "写了" 图书)

04

编写 Function (函数)

基于已有数据进行计算与推理，生成新的衍生价值。

输入:属性、关联、其他函数

输出:计算逻辑与
业务衍生指标

05

创建 Action (动作)

定义触发条件与响应逻辑，驱动知识模型落地应用。

输入:属性变化、函数结果

输出:自动化的
业务执行流程



Palantir Ontology-构建编辑器-所有模型都在哪配置？

Object / Property / Link / Function / Action 100% 在Palantir 本体构建器 (Ontology Builder)中可视化拖拽 + 表单配置完成



无需编写代码

- 无需编写底层数据库建表语句
- 无需手动定义图数据库 Schema
- 无需进行任何代码硬编码工作



业务人员视角

- 在可视化界面中直接“画”实体、“加”属性
- 连接数据关系，定义函数规则
- 配置业务流程触发动作



平台底层视角

- 自动生成底层元数据模型
- 自动创建图节点与边结构
- 自动分配存储资源、调度计算与触发流程

所有建模动作，统一入口：Ontology Builder

操作流程

01. 选择数据源

支持 DB 表、CSV、API 响应等多种格式

示例: 导入 `books.csv` (含字段: book_id, title, publication_date, price...)

02. 映射为 Object Type

将文件映射为实体“图书 (Book)”，系统自动识别字段并推荐 book_id 作为唯一标识符 (PK)。

03. 重复操作

处理 `authors.csv`，创建“作者 (Author)”对象，并设 author_id 为唯一 ID。

可视化结果示意



图书 (Book) | 核心业务实体

- 唯一ID: **book_id** • 属性: title, publication_date, price



作者 (Author) | 关联实体

- 唯一ID: **author_id** • 属性: name, birth_year



01. 选择对象

在本体构建器的项目资源树中，选中已创建的`图书`对象，进入其属性编辑工作区。



02. 添加属性 (两种方式)

- 直接映射：从`books.csv`源数据拖拽字段（如title）自动生成属性。
- 手动定义：补充数据源无的信息，如手动添加`库存状态`作为自定义属性。



03. 配置属性

为每个属性指定数据类型（文本、日期、枚举等），并设置是否为必填、是否可编辑等业务规则。

图书 (Book) 属性配置清单

书名 (Title)

类型：文本 (String)
来源：源数据`title`字段映射

出版日期 (Date)

类型：日期 (Date)
来源：源数据`publication\_date`

价格 (Price)

类型：数字 (Number)
来源：源数据`price`字段映射

库存状态 (Status)

类型：枚举 (Enum)
自定义：["在库", "已借出", "已售出"]



☰ 操作流程指引

01. 选择起始对象

在本体构建器中，选中需要作为关联起点的核心对象，例如`图书`对象。

02. 创建关联并指定依据

点击“添加关联”，选择目标对象`作者`。系统会提示选择关联的依据，通常是对象间的公共字段，如：图书表中的`author\_id`与作者表中的`author\_id`进行关联匹配。

03. 命名关系

将关联命名为具有语义的标识，例如`written\_by`（由...所写），使图谱关系清晰可读。

📌 定义基数 (Cardinality)

基数定义了两个实体间的数量关系，是构建准确知识图谱的核心逻辑：

- 一对多 (1:N)：一个作者可写多本书（作者 → 图书）
- 多对一 (N:1)：一本书（简化模型中）仅属于一个作者（图书 → 作者）

`图书` (N:1) ———— **written_by** ————> `作者` (1:N)

图形化界面中，对象间将生成此语义化连线



操作流程

- 01. 选择对象** | 在本体构建器中，选定数据模型的核心对象：`图书`
- 02. 创建函数** | 点击“添加函数”，命名为`Calculate\_Discounted\_Price`，进入可视化编辑器。
- 03. 编写逻辑** | 基于属性计算 (如: 价格 * 0.8)，或结合条件判断实现动态定价策略。
- 04. 配置输出** | 将计算结果定义为对象的虚拟属性“折扣价”

函数逻辑可视化



INPUT 输入

数据源: 图书对象的原生属性 → `图书.价格`



PROCESS 计算逻辑

表达式: 简单计算 → `价格 \* 0.8` (打八折)



OUTPUT 输出

生成: 动态计算属性 → `图书.折扣价` (虚拟属性)

■ 操作流程 Workflow



1. 选择对象 (Select Object)

在本体构建器中，选中核心业务对象，例如：`图书 (Book)`。



2. 创建动作 (Create Action)

点击“添加动作”按钮，定义动作名称，如：`Mark\_As\_Sold`，描述其业务含义。



3. 定义触发 & 4. 配置执行

- **触发：** 设置条件，如“图书.库存状态”变更为“已售出”。
- **执行：** 定义联动操作，如更新数据库记录、发送通知邮件或调用外部库存系统API。

■ 动作配置编辑器 (Editor)

ACTION: Mark_As_Sold

归属对象：图书 (Book)

触发条件 (WHEN):

`图书.库存状态` 被更新为 "已售出" (Sold)

执行操作 (THEN):

1. 更新数据库：将 `图书.状态` 字段锁定为 "已售出"。
2. 消息通知：发送邮件至admin@example.com提醒。
3. 系统集成：调用 `/api/inventory/reduce` 减少库存数量。



Palantir Ontology-应用案例-金融服务风险与合规管理



数据孤岛效应

交易、客户与市场数据分散在不同业务系统，难以统一查询与关联。



欺诈识别滞后

缺乏实时监控能力，难以从复杂的交易网络中快速识别潜在的欺诈行为。



合规成本高昂

跨部门数据整合困难，导致合规报告生成周期长，且容易出现人工错误。

Ontology 解决方案： 构建统一模型，打破数据壁垒，实现风险的实时洞察与自动化合规管理。



核心对象 (Object)

- Customer (客户)
- Transaction (交易)
- Account (账户)
- Watchlist (监控名单)

关键属性 (Property)

- risk_score (风险分)
- amount (交易金额)
- is_suspicious (是否可疑)
- status (状态)

关联逻辑 (Link)

- 客户 "发起" 交易
- 客户 "属于" 监控名单
- 交易 "涉及" 账户
- 账户 "归属" 机构



01 / Action (动作模块)

- **Investigate_Suspicious_Transaction (调查可疑交易)**

当交易被标记为可疑时，自动创建调查任务，并将客户、交易详情及历史记录打包推送给合规分析师。

- **Generate_Compliance_Report (生成合规报告)**

支持一键生成符合监管要求的标准化报告，自动汇总特定周期内所有高风险交易与异常客户活动。



实时风险监控

打破传统事后审查模式，实现对交易流的毫秒级监测与主动预警，防患于未然。



提升合规效率

自动化流程替代繁琐的人工操作，减少人为疏漏与错误，让合规团队聚焦于高价值的分析与决策。



降低欺诈损失

快速识别并拦截偏离用户正常行为模式的可疑交易，最大程度减少资金损失并维护品牌声誉。

场景：系统监测到一笔大额跨境交易，触发自动风控流程，进行全链路风险评估与拦截。

01. Object (对象)

- 客户: John Doe (ID: C1001)
- 交易: 跨境转账 (TXN-12345)
- 账户: ACC-67890 (Source)
- 名单: **Global Sanctions (命中)**

02. Property (属性)

- 金额: **\$1M (大额预警)**
- 流向: **High-Risk Country X**
- 画像: 评分 85/100 (高风险)

03. Link (关联)

John Doe 拥有 ACC-67890
John Doe 发起 TXN-12345
John Doe 匹配 制裁 #S-789

04. Function - Calculate_Risk()

因子: 金额(30%)+国家(30%)+画像(20%)+名单(20%)
结果: 风险评分 = **92 (极高风险)**

05. Action - Block_Transaction()

条件: 风险分>90 且 命中一级制裁。
动作: **1. 实时拦截;** **2. 警报至合规部。**



Palantir Ontology-应用案例-金融Ontology构建流程



STEP 01 定义对象 (Object)

目标: 识别业务中的核心实体。

输入: 业务需求文档、客户/交易数据库。

操作: 创建 Customer, Transaction 等对象。

输出: 定义了业务主角的空模型。



STEP 02 添加属性 (Property)

目标: 为每个对象添加描述性特征。

输入: 源数据字段 (如客户姓名、交易金额)。

操作: 为对象添加 risk_score, amount 等属性。

输出: 带有具体特征描述的对象模型。



STEP 03 建立关联 (Link)

目标: 定义对象之间的业务关系。

输入: 业务逻辑 (如“客户发起交易”)。

操作: 建立 Customer 与 Transaction 的连接。

输出: 形成相互连接的知识图谱。



STEP 04 编写函数 (Function) — 实现复杂计算

输入与操作: 基于“计算风险分”的分析需求, 编写

`Calculate\_Risk\_Score()` 函数, 综合客户历史属性和关联交易记录进行计算。

输出: 一个能够处理复杂逻辑、支持深度业务分析的模型。



STEP 05 创建动作 (Action) — 驱动业务流程

输入与操作: 依据“风险分过高则阻止交易”的规则, 创建

`Block\_Transaction()` 动作, 并配置自动触发条件。

输出: 一个能够驱动业务自动化、具备动态决策能力的智能模型。



供应链层级复杂

上下游供应商众多，零部件来源分散，难以实现全链路精准追溯。



数据孤岛严重

生产执行数据与质量检测数据分散在不同系统，缺乏统一的数据视图。



根因定位困难

质量问题发生时，需人工排查海量记录，定位根本原因耗时费力。

Ontology 解决方案： 构建从“供应商-生产-成品”的全链路知识图谱模型，打通业务数据壁垒，实现质量问题的秒级端到端追溯与根因分析。

核心数据模型构建 (Core Data Model)

- 1. 关键对象 (Objects):** 定义供应商 (Supplier)、零部件 (Part)、生产工单 (WorkOrder)、缺陷记录 (Defect) 四大核心实体。
- 2. 关键属性 (Properties):** 关联批次号、工单状态、缺陷严重等级等关键特征。
- 3. 业务关系 (Relations):** 建立“供应商供应零件 → 零件用于工单 → 工单产生缺陷”的逻辑链路。





Action (动作)

- **Initiate_Root_Cause_Analysis:**当发现缺陷时，自动追溯所有使用了相同批次零部件的工单，并通知相关团队。
- **Update_Supplier_Rating:**根据零部件的缺陷率，自动调整供应商的绩效评级。

Function (函数)

- **Trace_Part_To_Finished_Goods():**输入一个零部件批次号，快速找到所有使用了该批次零件的最终产品。
- **Calculate_Defect_Rate_By_Supplier():**计算每个供应商提供的零部件所导致的缺陷率。



快速召回

质量问题发生时，能迅速定位受影响的产品范围，减少召回成本和声誉损失。



优化供应链

精准识别高风险供应商，基于数据优化采购策略，推动供应链整体降本增效。



提升质量

通过全链路数据分析，精准定位工艺瓶颈，实现生产流程与产品质量的持续改进。



场景：质检部门在例行抽检中发现一批“Laptop Model XYZ”存在严重的电池掉电(Battery Drain)问题，需在2小时内定位根本原因，防止问题产品流入市场。



01. Object (对象定义)

- 产品: Laptop Model XYZ (SN-98765)
- 缺陷: Battery Drain (性能衰减)
- 部件: Battery Batch #B123
- 供应商: PowerCell Inc.



02. Property (关键属性)

- 产品属性: SN = "SN-98765"
- 缺陷属性: Type = "Battery Drain"
- 部件属性: Batch = "B123"



03. Link (关联关系)

- 产品“包含”电池零部件
- 零部件“由...供应”PowerCell Inc.
- 产品“报告了”电池掉电缺陷



04. Function: 缺陷追溯算法

逻辑: 缺陷ID → 关联产品SN → 解析BOM表 → 获取部件批次
输出: 锁定根源为 PowerCell Inc. 的 #B123 批次电池。



05. Action: 自动触发响应机制

触发: 系统自动判定该供应商为问题源头。
执行: 冻结B123批次库存，向供应链与品控团队发送预警。

结论：基于 Ontology 的知识图谱技术实现了从“成品缺陷”到“上游供应商”的毫秒级精准追溯，将排查周期缩短至分钟级。



01

定义对象 (Object)

目标:识别业务中的核心实体。

输入:产品设计文档、生产系统数据。

操作:创建 `Product`, `Part`, `Supplier` 等对象。

输出:定义了业务主角的空模型。

02

添加属性 (Property)

目标:为每个对象添加描述性特征。

输入:源数据的字段 (如序列号、批次号)。

操作:为对象添加 `serial\_number` 等属性。

输出:带有具体特征描述的对象模型。

03

建立关联 (Link)

目标:定义对象之间的业务关系。

输入:业务逻辑 (如“产品包含零部件”)。

操作:创建 `Product` "包含" `Part` 等关联。

输出:形成相互连接的知识图谱。

04

编写函数 (Function)

目标:实现复杂的分析和计算。

输入:分析需求 (如“追溯缺陷到供应商”)。

操作:编写 `Trace\_Defect\_to\_Supplier()` 函数。

输出:能够进行深度分析的模型。

05

创建动作 (Action)

目标:定义基于模型的自动化操作。

输入:业务流程规则 (如“发现缺陷则启动分析”)。

操作:创建动作并设置触发条件。

输出:驱动业务流程的动态智能模型。



行业挑战

- 患者、临床试验、药物数据分散，缺乏统一视图，难以综合分析
- 临床试验中不良事件难以实时监测，存在安全隐患
- 患者招募与筛选流程繁琐，依赖人工比对，效率低下



Ontology 解决方案

构建统一的医疗知识图谱，打破数据孤岛，实现多源异构数据的整合与关联，建立患者与临床试验的全景视图，从而加速临床研究并全方位保障患者安全。

Object (对象)

Patient (患者)
ClinicalTrial (试验)
AdverseEvent (不良事件)
Drug (药物)

Property (属性)

Patient.age (年龄)
Trial.status (状态)
AE.severity (严重程度)
Drug.dosage (剂量)

Relation (关联)

Patient → 参与 → Trial
Trial → 使用 → Drug
Patient → 报告 → AE
Drug → 引发 → AE



Action (动作)

- **Flag_Serious_Adverse_Event (标记严重不良事件):**当检测到严重不良事件时，立即通知试验负责人和伦理委员会，启动应急预案。
- **Identify_Eligible_Patients (识别符合条件的患者):**基于结构化的入组标准，自动从患者数据库中快速筛选出潜在的合格候选人。

Function (函数)

- **Calculate_Trial_Efficacy():**多维度分析试验组与对照组疗效差异，量化评估药物临床价值。
- **Detect_Drug_Drug_Interaction():**实时检查合并用药情况，识别并预警潜在的药物相互作用风险。



加速临床试验进程

通过自动化筛选与流程优化，显著缩短患者招募周期，提高试验启动与执行效率。



全方位保障患者安全

建立全流程监测预警体系，实时识别用药风险与不良事件，最大化保障受试者权益与安全。



提升临床研究质量

整合多源异构医疗数据，通过深度关联与分析，消除数据孤岛，支持更精准的科研决策。



Palantir Ontology-应用案例-医疗患者入组



01. Object (对象定义)

- 患者: Alice Smith (55岁, 女性)
- 临床试验: Trial-CANCER-001 (招募中)
- 疾病: 肺癌 III期 (Lung Cancer Stage III)
- 药物: OncoCure-X (实验性新药)



02. Property (关键属性)

患者属性: 年龄55岁 / 性别女
临床试验状态: 正在招募 (Recruiting)
核心入组标准: 年龄50-65岁 / III期肺癌



03. Link (语义关联)

- 患者“**诊断为**”特定疾病
- 临床试验“**针对**”该类型疾病
- 患者与试验建立“**潜在匹配**”关系



04. Function: 智能筛选 (Screen_Eligible_Patients)

逻辑: 系统自动遍历患者库, 基于“疾病类型”和“年龄范围”属性进行匹配计算。

结果: 确认 Alice Smith 完全符合 Trial-CANCER-001 的严格入组标准。



05. Action: 自动触达 (Send_Invitation)

触发: 一旦筛选结果为“匹配”, 系统立即触发后续动作。

执行: 生成个性化入组信发送给主治医生, 并将患者自动加入候选池, 全程无人工干预。



价值结论: 基于Ontology的自动化筛选, 消除了人工匹配的延迟与误差, 大幅提升临床试验招募效率, 加速救命新药上市进程。



01

定义对象 (Object)

目标: 识别业务中的核心实体。

输入: 医院信息系统(HIS)数据、临床试验方案。

操作: 在Ontology中创建 Patient, ClinicalTrial 等对象。

输出: 定义了业务主角的空模型。

02

添加属性 (Property)

目标: 为每个对象添加描述性特征。

输入: 源数据的字段 (如患者年龄、入组标准)。

操作: 为 Patient 添加 age 属性, ClinicalTrial 添加 criteria。

输出: 带有具体特征描述的对象模型。

03

建立关联 (Link)

目标: 定义对象之间的业务关系。

输入: 业务逻辑 (如“患者被诊断为某种疾病”)。

操作: 创建 Patient "诊断为" Disease 等关联。

输出: 形成相互连接的知识图谱。

04

编写函数 (Function)

目标: 实现复杂的分析和计算。

输入: 分析需求 (如“筛选符合入组条件的患者”)。

操作: 编写 Screen_Eligible_Patients() 函数。

输出: 能够进行深度分析的模型。

05

创建动作 (Action)

目标: 定义基于模型的自动化操作。

输入: 业务流程规则 (如“符合条件则发送邀请”)。

操作: 创建 Send_Invitation() 动作并设条件。

输出: 驱动业务流程的动态智能模型。



✕ 行业挑战

- 电网设备数量庞大且分布广泛，实现全面状态监测难度极高。
- 故障发生后，传统方式定位慢、修复耗时长，严重影响供电稳定性。
- 缺乏有效的预测性维护手段，难以在故障发生前进行干预。

⚡ Ontology 解决方案

整合分散的设备档案、实时传感器数据、历史工单与地理信息，构建**智能电网数字孪生模型**，打通数据孤岛。实现设备状态的可视化与故障的精准预测，将被动抢修转变为主动预测性维护，大幅缩短响应时间。

■ 核心模型构建 (Core Model)

- 1. 对象 (Object):** 定义关键实体: PowerPlant(发电厂), Transformer(变压器), Sensor(传感器), MaintenanceTicket(维护工单)。
- 2. 属性 (Property):** 关联关键状态数据: 温度、实时读数、工单优先级等。
- 3. 关联 (Relation):** 建立逻辑映射: 发电厂包含变压器 → 变压器安装传感器 → 变压器关联维护工单。





核心模型构建 (续)

Action (动作)

- Create_Maintenance_Ticket**: 传感器数据预警时, 自动生成高优先级工单并调度工程师。
- Isolate_Faulty_Equipment**: 检测严重故障时, 自动向控制系统发送指令, 快速隔离故障区域。

Function (函数)

- Predict_Equipment_Failure()**: 基于历史数据与运行模型, 预测设备未来故障概率。
- Optimize_Maintenance_Schedule()**: 结合健康状况优化维护计划, 实现预测性维护。

应用价值



减少停机时间

从“被动响应维修”转变为“主动预测与维护”, 消除意外停机风险。



提高电网可靠性

毫秒级故障响应与自动隔离机制, 最大程度防止故障范围扩大化。



降低运营成本

优化人力与物料维护资源配置, 同时延长核心设备使用寿命。



场景：智能电网系统实时监测到一台位于“Substation Alpha”的变压器传感器温度读数异常且持续上升，系统迅速启动预测性维护流程以防止故障发生。

01 OBJECT / 对象

- 变压器 (ID: TX-5678) | 传感器 (ID: S-1001)
- 待派工单 (ID: WO-123)
- 工程师 Bob (状态: 待命)

02 PROPERTY / 属性

- 温度读数: 120°C (持续上升中)
- 设备位置: Substation Alpha
- 工程师位置: 变电站附近 (Available)

03 LINK / 关联

- 变压器“安装有”传感器
- 工单“关联到”故障变压器
- 工程师“负责”该区域变电站



04 FUNCTION / 预测分析

基于传感器历史数据和设备模型，调用 `Predict_Failure_Probability()` 算法。
计算结果：未来24小时故障概率 **75%** (高风险等级)



05 ACTION / 自动响应

触发条件满足 (概率 > 70%)，系统自动执行 `Create_Maintenance_Ticket()`。
执行结果：高优工单已自动派发至工程师 Bob 的移动端。

核心价值：Ontology 将实时监测、预测性分析与自动化工单无缝集成，实现从“被动响应”到“主动预防”的根本性转变。



Palantir Ontology-应用案例-能源领域Ontology构建流程

Step 1:定义对象 (Object)

目标:识别业务中的核心实体。

输入:SCADA数据、设备台账、工单系统。

操作:创建Transformer, Sensor, WorkOrder等对象。

输出:定义了业务主角的空模型。

Step 2:添加属性 (Property)

目标:为每个对象添加描述性特征。

输入:源数据的字段（如传感器读数、状态）。

操作:为对象添加reading, status等具体属性。

输出:带有具体特征描述的对象模型。

Step 3:建立关联 (Link)

目标:定义对象之间的业务逻辑关系。

输入:业务逻辑规则（如“变压器安装有传感器”）。

操作:创建“安装有”、“属于”等对象间关联。

输出:形成相互连接的完整知识图谱。

Step 4:编写函数 (Function) —— 深度分析

目标:实现复杂分析和计算能力。| **输入:**故障预测、能耗分析等具体业务需求。

操作:编写Predict_Failure_Probability()等函数，基于历史数据训练预测模型。

输出:具备风险预测、趋势分析能力的“大脑”模型。

Step 5:创建动作 (Action) —— 闭环执行

目标:将分析转化为自动化业务操作。| **输入:**“风险高则自动建单”等流程规则。

操作:配置Create_Maintenance_Ticket()动作及触发条件。

输出:驱动业务流程的动态智能模型，实现“预测即行动”。



Palantir Ontology-应用案例-政府机构应急响应与情报分析



行业挑战

- 应急事件涉及多部门协同，跨部门信息壁垒导致沟通不畅，响应滞后。
- 情报来源分散(社交媒体/传感器/报告)，数据异构严重，难以整合与关联分析。
- 突发状况下，需快速评估影响范围并高效调度人、车、物等关键资源。



Ontology 解决方案

构建跨部门、跨数据源的**应急事件知识图谱模型**，打破数据孤岛。通过本体技术对碎片化情报进行语义化整合，实现对突发事件的**实时态势感知**与**智能决策支持**，辅助快速资源调度与处置。



Object (对象)

- **Incident**(事件)
- **Resource**(资源)
(人员、车辆、物资)
- **Location**(地点)
- **Agency**(响应机构)



Property (属性)

- **type**: 事件类型
(火警/地震/事故)
- **status**: 资源状态
(待命/已派遣/在用)
- **population_density**:
地点人口密度
- **level**: 响应等级



Relation (关联)

- 事件发生在地点
- 机构响应/处置事件
- 机构调度/调配资源
- 资源到达/部署于地点



Action (动作)

- **Dispatch_Resource (调度资源):**根据事件位置和类型, 自动推荐并调度最合适的应急资源 (如消防车、救护车)。
- **Generate_Situation_Report (态势报告):**一键生成含事件进展、资源部署及潜在风险的综合报告。

Function (函数)

- **Assess_Impact_Area():**结合人口、基建等多维数据, 智能评估事件潜在影响范围与严重程度。
- **Analyze_Social_Media_Activity():**实时分析社媒数据, 快速捕捉热点与公众情绪。



提升响应速度

快速整合碎片化信息, 消除数据孤岛, 显著缩短决策时间, 实现“秒级响应”。



优化资源调配

基于算法精准匹配需求, 确保人力与物力资源优先投入高风险、高需求区域。



增强态势感知

构建“一张图”指挥体系, 提供全维度、动态化的实时事件视图, 辅助科学研判。



场景设定：城市中心突发 5.8 级地震，应急指挥中心需要整合多源信息，从感知、评估到调度，快速做出科学的自动化决策。



1. Object (对象定义)

- 事件: Earthquake-M5.8 | 地点: 市中心中央公园
- 资源: Ambulance-01 / FireTruck-05
- 机构: 消防部门 (Fire Dept) / 医疗急救 (EMS)



2. Property (关键属性)

- 震级: 5.8 (破坏性) | 人口密度: High (高危)
- 资源状态: Available (待命) | 机构类型: 医疗/消防
- 数据来源: 实时监测系统与GIS图层



3. Link (逻辑关联)

- 事件发生在特定地点 (空间关联)
- 机构响应于突发事件 (职能关联)
- 机构调度对应资源 (动作关联)



4. Function: Assess_Impact_Area (智能评估)

基于地震震级、震中位置、人口密度及建筑抗震数据，系统自动计算出核心救援区域: Downtown Central Park周边1公里范围。



5. Action: Dispatch_Resources (自动调度)

触发条件: 影响区域锁定。系统自动向距离最近的“待命”救护车和消防车发出指令，并同步推送实时态势报告给指挥中心。

核心价值：Ontology 打破数据孤岛，将孤立信息转化为自动化决策链条，实现从“被动响应”到“主动处置”的质变。



01

定义对象 (Object)

目标: 识别业务中的核心实体。

输入: 应急指挥、GIS及部门数据。

操作: 创建事件、资源、地点等对象。

输出: 定义了业务主角的空模型。

02

添加属性 (Property)

目标: 为对象添加描述性特征。

输入: 震级、人口密度等源数据字段。

操作: 为对象挂载具体数值属性。

输出: 带具体特征描述的对象模型。

03

建立关联 (Link)

目标: 定义对象间的业务关系。

输入: 如“事件发生在地点”等逻辑。

操作: 构建对象间的关联关系网络。

输出: 相互连接的业务知识图谱。

04

编写函数 (Function)

目标: 实现复杂分析和计算逻辑。

输入: “评估事件影响范围”等需求。

操作: 编写算法函数综合多源数据。

输出: 具备深度分析能力的模型。

05

创建动作 (Action)

目标: 定义模型驱动的自动化操作。

输入: “评估后自动调度资源”等规则。

操作: 创建动作并配置触发条件。

输出: 驱动业务的动态智能模型。

07 Ontology驱动数据治理及决策



典型故事:四层技术架构协同工作实现从原始数据到智能决策的端到端流程



极客时间



应用与分析层

Application & Analytics Layer

◆ 核心功能

将本体能力以标准接口和可视化形式暴露给用户、业务应用及分析平台，赋能业务价值实现。

◆ 关键技术组件

RESTful API, GraphQL, 交互式图谱可视化工具, 机器学习平台集成 SDK



本体建模与推理层

Ontology Modeling & Reasoning Layer

◆ 核心功能

定义统一的实体模型、关系类型及业务规则；利用推理引擎挖掘隐藏关联与潜在洞察，是架构的“大脑”。

◆ 关键技术组件

可视化本体建模工具, 语义规则推理引擎, 图算法库 (社区发现/路径分析/影响力计算)



实体解析与链接层

Entity Resolution & Linking Layer

◆ 核心功能

消除数据孤岛，跨异构数据源识别并合并指向现实世界同一实体的记录，确保数据的一致性与完整性。

◆ 关键技术组件

确定性匹配引擎, 概率匹配算法 (ML-based), 数据标准化规则库, 幸存者偏差规避策略



数据连接与集成层

Data Connection & Integration Layer

◆ 核心功能

作为架构的基石，建立与企业内外部各类结构化、半结构化及非结构化数据源的标准化连接。

◆ 关键技术组件

预构建连接器 (JDBC/ODBC/Cloud APIs), 数据虚拟化引擎 (查询联邦), 高性能 ETL/ELT 流水线



数据连接层:通过预构建连接器与数据虚拟化，实现异构数据源的广泛连接



极客时间

CORE FUNCTION / 核心功能:连接一切，打破孤岛

作为企业数据本体论的基石，该层的核心使命是无缝连接企业内部业务系统与外部第三方平台，消除物理和逻辑上的数据孤岛，为上层应用提供统一的数据访问接口。

01. 预构建连接器 (Connectors) | 降低接入门槛

内置数百种连接器，覆盖传统数据库 (PostgreSQL/MongoDB)、云存储 (S3/ADLS)、消息队列 (Kafka)、主流ERP/CRM (SAP/Salesforce) 及文档系统 (SharePoint)，大幅降低接入成本。

02. 数据虚拟化 (Data Virtualization) | 实时访问，拒绝冗余

采用查询联邦 (Query Federation) 技术，避免大规模物理复制(ETL)。按需实时访问源数据，确保数据绝对新鲜，规避数据不一致风险，并显著降低存储与治理成本。

03. ETL / ELT 流水线 | 非结构化数据结构化

内置处理引擎，针对文档、邮件、图像等非结构化数据进行提取、清洗与结构化转换。



关系型/NoSQL
PostgreSQL, MongoDB



云存储/对象存储
AWS S3, ADLS Gen2



业务系统
(SaaS/On-Prem)
SAP, Salesforce, ERP



非结构化/流数据
Kafka, SharePoint, IM

数据连接与集成层 (Data Integration Layer)

统一接入标准 · 屏蔽底层复杂性 · 全局数据目录

数据虚拟化引擎

查询联邦 / 实时计算

数据处理流水线

ETL / ELT / 数据清洗



实体解析层:通过确定性与概率性匹配，实现跨数据源的实体统一



多源异构数据输入

整合来自 CRM、HR、ERP 等系统的碎片化记录 (如: "John Doe" vs "J. Doe")

01 确定性匹配

基于身份证号、员工工号等唯一标识符进行精准匹配
准确率 100%，处理高效
覆盖范围有限

02 概率性匹配

分析姓名、地址、电话等属性的相似度
应用TF-IDF、余弦相似度、随机森林等算法
计算同属一实体的置信度得分



03 幸存者偏差规则 (Survivorship Rules)

定义冲突仲裁逻辑，解决同一实体的信息不一致问题 (如以最新更新时间、核心业务系统数据为准)，确定“黄金记录”。

OUTPUT: 企业级统一实体视图 (Golden Record)

核心价值:构建“单一事实视图”

消除跨系统的数据孤岛与冗余，将描述同一真实世界实体 (客户、员工、供应商) 的多条记录清洗、合并，形成统一、权威的实体画像，支撑精准营销与风险管控。

确定性匹配 (Deterministic Matching)

利用不可变的唯一标识符进行硬性匹配。优点是无误差且速度极快；缺点是覆盖范围受限于唯一ID的普及率，难以处理没有统一ID的“长尾”数据。

概率性匹配 (Probabilistic Matching)

核心解决无唯一ID的匹配难题。算法综合分析姓名、电话、地址等多维度属性的相似度，计算记录指向同一实体的概率，平衡匹配的召回率与准确率。

幸存者偏差规则 (Survivorship Rules)

为解决同一实体信息冲突而设计的业务规则库。例如:“以最近更新的系统数据为准”或“以核心交易系统的客户信息为基准”，确保最终视图的准确性。



本体建模层:以可视化与推理引擎，将数据转化为结构化业务知识

核心功能 · CORE FUNCTION

定义实体、关系与属性，并在此基础上执行逻辑推理与知识发现，是构建业务智能与自动化决策能力的核心技术基座。

01. 可视化建模工具 (Visual Modeling)

描述:提供拖拽式图形界面，让业务用户定义实体(如Person)、关系(如works_for)及属性(如Person.name)。

价值:打破技术壁垒，实现业务逻辑到数据模型的直接映射，大幅降低本体构建门槛。

02. 语义推理引擎 (Semantic Reasoning)

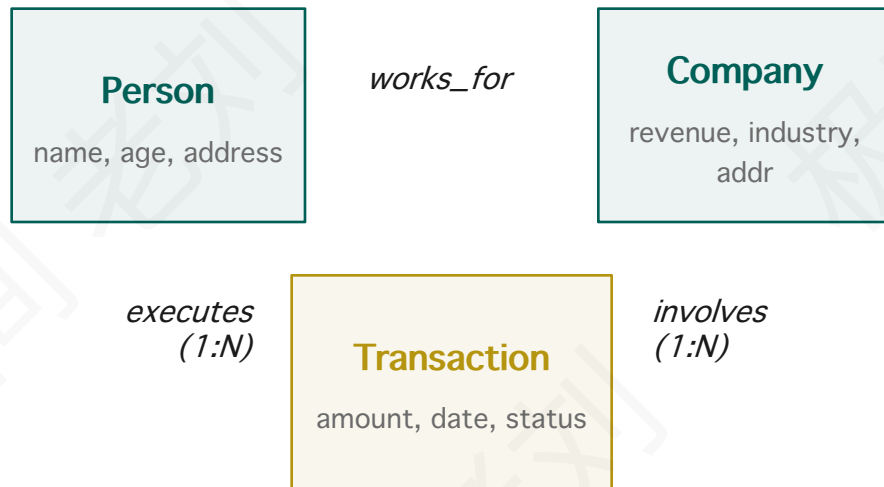
描述:基于预定义的本体规则进行自动逻辑推演，挖掘隐式知识。

示例:系统自动从“A is_parent_of B”与“B is_parent_of C”推导出“A is_grandparent_of C”。

03. 知识发现 (Knowledge Discovery)

利用图算法（社区发现、路径分析、中心性分析等），从海量互联数据中识别隐藏模式、关键节点及紧密关联实体群。

本体结构示意 | Ontology Structure



逻辑推理示例 (Reasoning Logic)

已知: A is_parent_of B + B is_parent_of C

规则: $(X \rightarrow Y) + (Y \rightarrow Z) \Rightarrow X \text{ is_grandparent_of } Z$

推导: A is_grandparent_of C (新增知识)



应用层:通过API与可视化工具，将本体能力转化为业务价值



自定义应用

开发者集成 API
构建专属业务系统



可视化分析

业务分析师的
探索与洞察工具



机器学习

数据科学家的
特征工程基石

动态本体论核心能力层

Dynamic Ontology Core Engine

价值出口:连接技术与业务的桥梁

将底层复杂的本体逻辑与计算能力，转化为灵活、易用的接口与工具，直接赋能开发者、分析师与科学家，实现商业变现。

01. API 接口标准化服务

实现逻辑:提供 RESTful 与 GraphQL 双接口标准，支持高并发、低延迟的数据查询与图谱遍历，适配各类企业级应用架构。

核心价值:实现本体能力的“平台化”，打破技术壁垒，支持跨系统、跨部门的能力复用，降低二次开发成本。

02. 零门槛可视化探索工具

实现逻辑:内置节点图谱、动态时间线、热力分布图等交互式组件，支持拖拽式关联分析与自动路径挖掘。

核心价值:将复杂的“图数据”转化为直观的“图信息”，让非技术背景的业务分析师也能快速发现关键实体间的隐藏关联。

03. AI 模型特征工程深度集成

实现逻辑:无缝对接主流机器学习平台，自动提取本体中丰富的结构化关系特征（如“客户-社交-流失”链路）。

核心价值:解决传统AI“黑盒”问题，提升模型解释性；利用关系特征提升预测准确度（如客户流失预测提升20%+）。



核心技术:数据编织 - 通过逻辑层实现异构数据的“虚拟整合”



01 语义标注

Semantic Annotation

对来自不同数据源的原始数据字段进行业务语义标注。例如，将CRM系统中的`cust\_id`字段和ERP系统中的`client\_number`字段都标注为本体中的`Customer.id`。这一步是建立数据与业务概念之间映射的基础。



02 映射规则

Mapping Rules

定义从源数据到本体实体的详细映射规则。这些规则可以是简单的字段映射，也可以是复杂的转换逻辑（例如，将多个字段合并为一个，或通过正则表达式提取信息）。这些规则存储在逻辑层，而非物理数据中。



03 实时解析与构建

Real-time Resolution & Construction

当用户或应用发起查询时，系统根据映射规则和实体解析结果，实时地从各个原始数据源提取数据，并在内存中动态构建出符合本体结构的、统一的实体关系视图。这个过程对用户是透明的，仿佛数据已经被物理整合。

核心优势:数据编织避免了昂贵且耗时的物理数据迁移（ETL），保证了数据的实时性和一致性，同时提供了极高的灵活性，因为修改逻辑层的映射规则远比重新进行物理数据整合要容易得多。



行业应用:金融风控 - 基于本体对欺诈行为的实时识别与响应

01

定义实体与关系

首先, 在本体中定义核心业务实体, 如Transaction(交易), Account(账户), Customer(客户), Merchant(商户), Device(设备), IP_Address(IP地址)。并定义它们之间的关系, 如has_account, originates_from, uses_device。

02

连接多源数据

通过数据连接层, 实时接入交易系统、客户信息系统、设备日志、网络日志等多个数据源, 打破信息孤岛, 为分析提供完整的数据基础。

03

实时构建交易子图

当一笔新交易发生时, 动态本体论立即根据实体关系, 实时构建包含该交易、关联账户、客户、设备和IP的“子图”, 完整描绘本次交易的上下文, 捕捉复杂关联。

04

图算法分析与风险评分

利用图算法识别异常:

- 环路检测: 识别资金快速流转
- 异常连接: 发现高风险关联
- 地理异常: 定位与设备不符

实时计算综合风险评分。

05

动态响应与本体演化

- 自动响应: 触发预警或拦截
- 自我进化: 分析师更新规则, 系统可快速识别新欺诈模式, 持续提升防御能力。

➤ 行业应用:金融风控 - 实体关系网络完整描绘交易的上下文，揭示隐藏风险 极客时间





行业应用:金融风控 - 多维度图分析的风险评分模型, 实现欺诈识别

资金环路风险

85 / 100

■ 风险说明

检测到资金在过去24小时内, 通过**Account A-789**在**Account X, Y, Z**之间形成了闭合资金环路。

■ 风险解析

快速且无商业目的的资金流转模式, 高度符合洗钱特征。环路涉及的节点数量、流转金额及频次均为评分关键因子。

异常关联风险

92 / 100

■ 风险说明

高价值客户**Customer C-123**首次与过去30天内已关联3起欺诈案件的**Merchant M-456**发生交易。

■ 风险解析

优质客户行为模式突变, 且与高风险实体建立连接, 是典型的“账户接管”或“羊毛党”预警信号, 系统给予高权重评分。

行为异常风险

78 / 100

■ 风险说明

常用设备**Device D-001**的交易IP-**192.168.1.1**与过去一年的常用地理位置相距超5000公里。

■ 风险解析

结合设备指纹与地理位置, 识别出“不可能的旅行”。跨地域时间间隔越短、物理距离越远, 风险评分越高。

综合风险评分: **85 分 (High Risk)**

触发高级别预警 · 交易已自动冻结 · 等待人工审核



行业应用:情报分析 - 整合多源信息加速威胁识别

01

定义实体与关系:定义情报分析所需的核心实体（如关注人物、地点、事件、组织、通信记录），并建立关键关系模型（如 located_at、attended_event、member_of、communicated_with），为情报整合建立标准化数据底座。

02

连接多源情报:打破数据孤岛，连接并整合来自异构渠道的情报，包括高分辨率卫星图像分析、通信网络截获记录、人力情报 (HUMINT) 报告以及各类开源情报 (OSINT) 数据，实现情报全域互联。

03

关联分析与查询:支持分析员提出复杂的、动态的查询请求（如“所有与目标X在过去72小时内通信且出现在地点Y附近的关联人员”），系统实时聚合并关联多源数据，自动生成直观的情报关联图谱。

04

假设检验与证据链构建

针对分析员提出的假设（如“组织A正策划针对地点Z的行动”），系统自动溯源，寻找并串联支持或反驳该假设的完整证据链（如物资采购记录、异常通信模式、人员轨迹），大幅降低人工核查成本。

05

知识发现与网络分析

利用图算法与社区发现技术，深度挖掘海量情报中的潜在规律，自动识别隐藏的组织网络层级、关键核心人物以及潜在的协同行动关系，为战略决策提供超越人工分析能力的前瞻性洞察。



行业应用:情报分析 – 揭示人物、组织与事件间的隐藏联系





行业应用:医疗健康 - 构建患者本体，实现个性化治疗与精准医疗

01

定义医疗实体

定义核心医疗实体:Patient (患者), Disease (疾病), Medication (药物), Lab_Result (化验结果), Imaging_Report (影像报告), Gene (基因) 等, 构建领域知识基础。

02

整合多源异构医疗数据

打通并标准化分散在不同系统中的数据孤岛:电子病历系统(EHR)、基因测序原始数据、影像存档与通信系统(PACS)、医院运营系统及公开的临床试验数据库。

03

构建患者“数字孪生”本体

基于整合后的数据, 为每一位患者建立高保真度的数字映射, 聚合其全生命周期的临床病史、基因组特征、治疗方案历史与疗效反应记录。

04

■ 个性化治疗方案推荐

基于相似基因型与病史患者的疗效数据, 精准匹配药物反应模型, 规避禁忌, 推荐最优用药方案。

■ 精准疾病风险预测

结合基因标记、生活方式及家族病史, 构建预测模型, 识别高风险人群, 实现慢性病与遗传性疾病的早期干预。

■ 加速临床试验招募

快速检索患者本体库, 精准筛选满足临床试验严格入组与排除标准的受试者, 缩短新药研发周期。



行业应用:智能制造 - 构建工厂数字孪生，实现预测性维护与智能调度



STEP 01 · 定义制造实体

精准定义**Machine** (设备)、**Sensor** (传感器)、**Production_Order** (生产订单)、**Part** (零件)、**Worker** (工人)、**Supplier** (供应商)等核心业务实体与关系，建立统一的数据语义底座。



STEP 02 · 连接工厂数据流

打通异构系统壁垒，实时接入设备传感器IoT数据、生产计划系统(MES)、企业资源计划(ERP)及供应链管理系统(SCM)的全域数据。



STEP 03 · 构建数字孪生

基于统一本体构建与物理工厂毫秒级实时同步的数字孪生体，完整映射所有设备健康状态、实时生产进度、物料流转及资源配置信息。



预测性维护

融合传感器实时数据与历史故障知识库，建立AI预测模型，精准预判设备潜在故障点与时间，变“被动抢修”为“主动维护”，大幅降低非计划停机。



动态智能调度

基于孪生体实时感知的物料库存、产线负载及订单优先级，通过算法动态推演并调整排产方案，最大化平衡交付周期与资源利用率。



全链路质量追溯

一旦发现产品缺陷，依托数字孪生体关联的“人机料法环”全维度数据，秒级定位异常根源（批次原料/工序参数/设备状态），实现质量闭环管理。



01 定义交通实体 (Entity Definition)

定义核心交通要素:**Vehicle**(车辆),**Road**(道路),**Traffic_Signal**(交通信号灯),**Pedestrian**(行人),**Lane**(车道) 以及**Event**(事故、施工等突发事件)。建立统一、标准化的交通要素分类与属性体系, 为后续融合与建模奠定基础。

02 融合多源感知数据 (Data Fusion)

集成异构数据源:融合来自**车载传感器** (摄像头、毫米波雷达、激光雷达点云) 的实时感知数据、**高精地图**的先验静态信息、**V2X 车联网通信**数据以及云端下发的宏观交通流信息, 解决单一视角的感知盲区与不确定性。

03 构建动态交通本体 (Dynamic Ontology)

构建实时数字孪生底座:将静态的路网拓扑结构与动态的交通参与者状态 (如速度、位置、意图) 进行有机关联, 形成一个高保真、可演进的动态交通知识图谱。该本体能够实时反映环境变化, 是系统进行推理和决策的核心逻辑基础。

04 赋能核心自动驾驶决策场景

- 路径规划**:基于实时交通流与静态路网, 动态生成最高效、安全的行驶路径。
- 环境感知与决策**:预判行人与车辆意图, 动态决策避让、变道或停车。
- 异常事件响应**:毫秒级识别事故与施工, 并自动触发重新规划与安全停车逻辑。



01

定义金融实体

定义核心业务实体:Company (公司), Stock (股票), Industry (行业), Economic_Indicator (宏观经济指标), Investor (投资者), News (新闻) 等，为数据整合提供标准化框架。

02

整合市场多维数据

无缝连接异构数据源:高频市场交易记录、上市公司定期财报、全量新闻资讯、宏观经济数据库及社交媒体情绪数据，打破数据孤岛。

03

构建金融知识图谱

基于动态本体构建关系网络，清晰刻画公司竞争、行业上下游、宏观经济传导路径等复杂逻辑，还原金融市场的“全景关系视图”。

04

赋能核心场景

投资研究

快速分析单一事件（如原材料涨价）对整条产业链上下游上市公司的传导与估值影响。

财富管理

融合客户画像与资产特征，精准匹配风险偏好与投资组合，提供千人千面的资产配置建议。

合规监管

穿透识别复杂交易结构与关联关系，实时监控异常资金流向，防范市场操纵与内幕交易。



行业应用:生命科学-构建药物研发本体，加速新药发现

01

定义生物实体

定义标准化核心实体:**Molecule (分子)**、**Target (靶点)**、**Disease (疾病)**、**Clinical_Trial (临床试验)**、**Publication (学术论文)**。建立行业通用的术语体系与关系定义，消除语义歧义。

02

整合科研数据

整合异构多源数据:分子结构数据库、基因测序数据库、全球临床试验数据、海量科研文献与专利数据。打破各机构、各系统间的数据孤岛，构建统一的数据底座。

03

构建知识图谱

基于标准化本体，构建连接“分子-靶点-通路-疾病-临床结果”的全局知识图谱，挖掘海量数据背后隐藏的因果关联与潜在规律，实现数据的知识化。

药物重定位 (Drug Repurposing)

挖掘现有药物的新适应症，如发现治疗A病的药物对B病有效，大幅缩短研发周期。

新靶点发现 (Target Identification)

基于疾病的基因特征和分子网络，快速筛选与验证潜在的高价值药物作用靶点。

临床试验优化 (Clinical Trial)

精准匹配受试者群体，优化试验设计方案，提高成功率并降低研发成本。

08 Ontology驱动大模型Agent执行



Agent落地大背景:企业AI落地“最后一公里”难题



语义鸿沟 (Semantic Gap)

企业内部存在大量异构系统 (ERP, MES, CRM等), 同一概念在不同系统中可能有不同的表述 (如“客户”、“用户”、“买家”), 导致AI无法形成统一的理解, 跨系统协同困难重重。



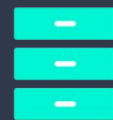
模型幻觉与不可控 (Hallucination & Uncontrollability)

通用大模型作为概率模型, 其生成的内容可能与事实不符 (幻觉), 且其推理路径“黑盒化”, 难以预测和控制。在需要高确定性和合规性的企业场景中, 这是不可接受的风险。



执行断层 (Execution Gap)

AI可以分析数据、提出建议, 但往往无法直接驱动业务系统执行操作 (如创建工单、修改计划), 导致“决策”与“行动”之间存在断层, 价值无法完全释放。



知识沉淀难 (Knowledge Silos)

企业的核心知识大量存在于员工的大脑中, 以隐性经验的形式存在, 难以被系统捕获和复用。每个AI应用都像一个“信息孤岛”, 无法形成持续进化的智能体系。



痛点直击:系统“语言不通”

企业核心数据分散在 ERP、MES、CRM、WMS 等多个异构系统中，形成严重的数据孤岛。同一业务指标在不同系统中定义不同、计算口径不一，对象间的关联关系模糊。

不同业务系统对同一核心概念（如“客户”、“产品”、“订单”）的表述和定义不一，导致数据在流转中无法被有效关联和理解，形成了严重的数据孤岛，阻碍了跨系统的数据融合与分析。



方案:统一语义“翻译层”

构建企业级统一的生产语义模型（本体），对订单、物料、设备、工艺等核心业务对象进行标准化定义，清晰界定实体、属性及关联关系，消除歧义。

通过本体技术对核心业务概念进行标准化定义，无论源系统的字段名和结构有何差异，都能映射到统一的本体实体上，实现企业内部所有数据和系统的跨域语义贯通。



Agent落地破局:本体驱动智能体 (Ontology-driven Agent, ODA)

本体驱动智能体 (Ontology-driven Agent, ODA)



在企业AI语境下，它是一种将企业核心业务知，包括其**对象(Entities)**、**关系(Relations)**、**事件(Events)**、**行为(Actions)** 和 **逻辑(Logic)**进行结构化、语义化建模，并以此为基础驱动智能体推理、决策和执行的技术体系。将本体作为智能体的“业务大脑”和“操作系统”，让智能体在一个由业务规则明确定义的框架内进行感知、推理和行动。



核心思想：业务规则约束

强制智能体的推理逻辑、路径选择与最终决策，必须**严格遵循由本体定义的业务规则、逻辑关系与语义约束**，拒绝不可控的自由发挥。



数字世界的“语义骨架”

为企业的海量业务数据赋予结构化的语义内涵，构建清晰、严谨的知识脉络，确立企业数字资产的基础框架。



动态可更新的“业务地图”

描绘了业务流转的逻辑路径与价值流向，随业务变化实时迭代，指引智能体在复杂的企业环境中精准导航。



它不是一张静态的元数据表，而是一个活的、可执行的“知识图谱 + 业务逻辑引擎”，智能体不再是一个不可解释的“黑盒”，而是一个在清晰定义的“业务地图”上行动的“导航员”。它清楚地知道行动的边界在哪里、规则是什么，以及每一步决策背后的逻辑与依据。



问题:不可预测与黑盒推理

模型的生成式输出具有随机性与不可控性，且推理过程完全“黑盒”化，难以追溯逻辑链条，导致无法满足企业级业务场景对输出准确性、流程合规性的严苛要求。

在需要高确定性的企业级决策场景（如财务审批、生产调度、合规检查）中，这种“自信地”生成违反业务规则或事实错误指令的行为，会带来严重的合规风险和经济损失，是阻碍落地的核心痛点。



方案:本体驱动-业务硬约束

将企业核心业务规则（如“采购金额超10万元需经VP审批”、“敏感数据不得外发”）显式化、结构化地“嵌入”本体知识库中。智能体的任何决策与操作均需通过认知引擎的实时规则校验，一旦触发违规逻辑将被**直接阻断**，确保业务流程严格合规。

场景示例:生产工单自动调整

当智能体意图执行“将A产品工单产量从500件调整为800件”时，认知引擎会实时查询本体规则库，触发“单次产量变更幅度超过20%必须经产线主管审批”的校验逻辑，因缺少前置审批，该指令将被立即阻断并触发人工介入，从而规避风险。



问题:不可预测与黑盒推理

模型的生成式输出具有随机性与不可控性，且推理过程完全“黑盒”化，难以追溯逻辑链条，导致无法满足企业级业务场景对输出准确性、流程合规性的严苛要求。

在需要高确定性的企业级决策场景（如财务审批、生产调度、合规检查）中，这种“自信地”生成违反业务规则或事实错误指令的行为，会带来严重的合规风险和经济损失，是阻碍落地的核心痛点。



方案:本体驱动-全链路可解释性

打破AI黑盒：系统每一次的审批通过、拦截拒绝或决策建议，均能清晰回溯到本体知识图谱中的具体节点和逻辑推理路径。无论是内部审计还是外部监管，系统都能提供可量化、可追溯的决策依据，回答“为什么通过”或“为什么拒绝”，消除合规风险。

场景示例:生产工单自动调整

当智能体意图执行“将A产品工单产量从500件调整为800件”时，认知引擎会实时查询本体规则库，触发“单次产量变更幅度超过20%必须经产线主管审批”的校验逻辑，因缺少前置审批，该指令将被立即阻断并触发人工介入，从而规避风险。流程可解释。



执行断层 (Execution Gap)

AI 分析往往止步于“发现问题”，输出的洞察无法自动转化为业务系统的具体操作，仍需人工手动介入执行，导致响应滞后，且难以实现大规模、高频次的业务优化。

当系统发现异常问题后，往往需要人工介入，手动切换到工单系统、预警系统等不同平台进行后续操作。这种“执行断层”导致AI价值无法完全释放，最终陷入“看得破，管不了”的尴尬局面。



本体驱动:Action元素构建自动化桥梁

通过在本体中定义标准化的**Action**（行为）元素，将业务系统（MES/ERP等）的操作能力封装为智能体可直接调用的接口。当智能体完成分析后，自动触发对应**Action**，驱动系统执行任务，真正实现“发现-决策-执行”的端到端闭环。

例如:智能体诊断出设备异常时，可直接调用“生成工单”接口，自动填写故障详情并派发任务，真正实现从“发现问题”到“任务解决”的全自动业务闭环。



痛点：隐性经验的“孤岛效应”

企业内部，专家的经验往往存在于员工个人的脑海或零散文档中，属于“隐性知识”，难以被系统有效捕获和转化。

更严重的是，各业务线、各部门的AI应用之间知识无法互通，形成一个个信息孤岛，导致重复造轮子，知识价值无法最大化。



01 / 知识显性化：数字资产沉淀

将专家的业务经验、行业Know-how与企业内部规则，统一转化为**结构化的本体模型**，打破“隐性”壁垒，让经验成为企业统一、可复用、可共享的数字资产。



02 / 持续进化：智能闭环系统

利用智能体执行任务中产生的新数据与用户反馈，持续识别知识缺口、动态优化本体规则，构建“使用-反馈-优化”的自我进化闭环。

本体驱动Agent:核心理念-从“数据驱动”到“知识驱动”

传统的AI应用主要是“数据驱动”，模型从海量数据中学习模式。而本体驱动的核心理念是“知识驱动”，它将企业隐性的、分散的业务知识和规则显性化、结构化，形成一个统一的认知模型，为智能体提供坚实的“世界观”。



01 / 统一语言

将企业内部不同系统、不同部门对同一概念的不同称呼（如“原料批次号”、“入库编号”）统一映射到标准的本体概念上，消除语义歧义，实现跨部门、跨系统的无障碍沟通。



02 / 结构化关系

将业务逻辑中隐含的复杂关系（如“原料 - 工单 - 成品”的投料关系、“客户 - 合同 - 订单”的关联）显性表达为可计算、可推理的图谱关系，让机器理解业务实体间的内在联系。



03 / 可执行规则

将专家经验和业务规则（如“库存低于安全阈值需自动触发采购申请”、“异常质检结果需立即锁定批次”）固化为本体中的逻辑（Logic）和行为（Action），让智能体具备自主决策能力。

统一的语义骨架

本体通过对概念的标准化定义，消除了不同部门、不同系统之间的“巴别塔”，为所有AI应用提供了统一的业务语言，打破信息孤岛与沟通壁垒。

动态的数字孪生

本体实时映射企业的业务状态、流程和规则，构建了一个与物理世界同步演进的数字孪生体。AI智能体可在此环境中进行推演和预演，从而做出更可靠、更符合实际的决策。

本体驱动Agent:核心理念-超越知识图谱与RAG，赋予业务逻辑与行动能力

极客时间



传统知识图谱

被动查询工具 | 静态关联

核心目标:数据关联与查询检索。侧重于描述实体及其之间的关系，回答“是什么”。

知识形态:静态（实体、关系）

推理方式:符号推理（图遍历）

可解释性:高（路径清晰可查）

执行能力:无（仅回答问题）



检索增强生成 (RAG)

文本生成器 | 事实增强

核心目标:增强LLM的事实准确性

知识形态:动态（实时检索的文本）

推理方式:概率推理（LLM生成）

可解释性:低（生成过程黑箱）

执行能力:无（仅生成文本）



本体驱动智能体

业务参与者 | 自主执行

核心目标:业务理解与自主执行任务

知识形态:动态（实体+关系+事件+逻辑+行为）

推理方式:符号推理 + 概率推理 协同增强

可解释性:高（可追溯至本体定义规则）

执行能力:有（**Action**驱动业务系统）

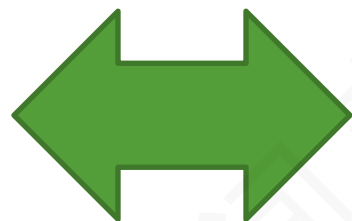


区别:本体是知识图谱的“超集”与“灵魂”，为其赋予了业务逻辑与行动能力。

从单纯的“记录者”升级为业务的“指挥者”，让数据真正驱动业务流程。

🔗 本体的角色 (确定性与约束)

- 提供结构化的知识体系与确定性的业务规则
- 限定智能体的行为边界，确保操作合规可控
- 为复杂逻辑推理提供可解释、可追溯的依据



🧠 大模型的角色 (灵活性与生成)

- 精准处理自然语言理解，实现流畅的人机交互
- 在本体框架内，执行灵活的任务规划与推理
- 高效应对本体未覆盖的长尾场景与不确定性需求

两者结合，既保证了AI在关键业务上的可信、可控，又保留了其在创新场景中的灵活、智能。

代表案例:Knora本体驱动Agent-三层协作体系

通过本体 (Ontology) 将企业内的多源异构数据 (如设备、物料、订单等) 与具体的业务场景 (如质量追溯、设备运维) 深度连接, 构建一个统一、可理解的语义网络, 打通数据孤岛。



底层 · 数据与集成层

作为坚实的地基, 负责整合来自 ERP、MES 等各类异构系统海量业务数据, 实现数据的统一接入与标准化处理。



中层 · 语义与智能层

系统的“大脑”, 集成本体 (Ontology)、认知引擎 (Cognitive Engine) 与大模型 (LLM), 将原始数据转化为结构化知识, 驱动智能决策生成。

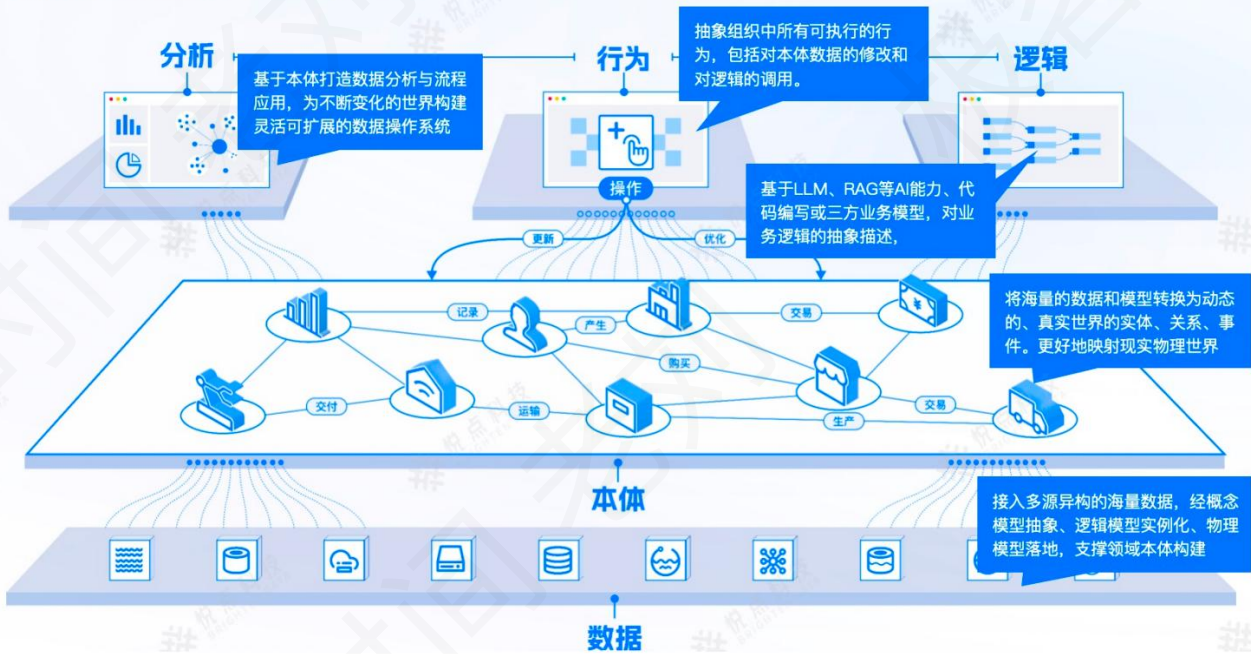


上层 · 应用与交互层

直接面向用户的终端, 通过智能体 (Agent) 与多样化应用 (Application) 提供自然、高效的交互界面, 将智能能力转化为实际业务价值。

悦点的解法和主张: Ontology+LLM

悦点科技
BRIGHTEN DATA





代表案例:Knora本体驱动Agent-本体要素定义



实体 (Entity)

业务对象，是本体的基本单元。
例如：工单、设备、人员、物料。



关系 (Relation)

实体之间的语义连接。
例如：工单由人员负责，设备属于产线。



事件 (Event)

业务中有意义的状态变化。
例如：设备故障、订单下达。



行为/动作 (Action)

系统可以执行的操作，如“创建工单”、“派发任务”。
这是将本体从静态知识变为可执行规范的关键。



逻辑 (Logic)

可执行的业务逻辑规则。
例如：“当库存低于阈值时，自动触发采购申请”。



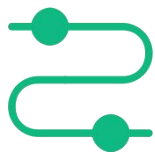
代表案例:Knora本体驱动Agent-三类本体



陈述性本体 (Knowledge)

核心内容:将企业的静态知识、数据结构化表达。例如产品分类体系、组织架构、物料主数据等。

关键作用:回答“是什么”的问题,构成企业业务知识体系的基石。



程序性本体 (Knowhow)

核心内容:描述企业的业务逻辑与操作流程规范。例如质量检验的标准步骤、设备维修的操作规程等。

关键作用:回答“怎么做”的问题,为一线业务执行提供标准化的行动指南。



动态本体 (Dynamic Context)

核心内容:反映业务实时环境、系统状态与上下文变化。例如当前设备的运行参数、实时的库存水位等。

关键作用:赋予本体感知和适应能力,确保决策能基于最新的业务情境进行。

代表案例:Knora本体驱动Agent-一个例子

以“工单产量变更须经审批”为例，完整的执行链路如下：

01 用户意图

用户在系统发起指令：
“将工单WO-2026-0312的计划产量，
从初始设定的500调整为800”。

02 引擎查询本体

- 定位:遍历图找到节点 WO-2026-0312 (类型: WorkOrder)
- 计算:变更幅度 60%，超过 20% 阈值
- 规则:命中 requiresApproval = true
- 关联:需 BOM工程师 & 生产经理审批

03 约束校验

系统进行合规性判定：
当前操作记录中，缺少关键的
“approvedBy (审批人)”关系数据。

→ 校验不通过，变更被拦截

04 系统响应

- 阻断写入:拦截直接对数据库的写操作，防止违规数据落地。
- 生成报告:输出结构化错误报告，精确定位并指出违反的规则节点。
- 任务路由:自动触发并创建审批任务，根据规则自动流转给对应负责人。
- 审计留痕:将原始操作意图完整记录至审计日志，状态标记为“待审批”。

05 审批通过后

当审批人确认并通过后，系统自动在图模型中补充 **approvedBy** 关系，以关联审批人与该业务操作。随后，流程重新触发规则校验，确认所有合规性要求均已满足后，最终执行数据写入操作，完成闭环。

最后的总结

- 当下国内没有统一的本体论理论体系，也没有统一落地标准，各摸一角
- 盲人摸象，不是好事儿，跳出来看，会更清醒
- 现在国内说“本体论落地”的，本质就不是学术流派，就是有套路的落地范式。
 - 用本体做数据治理&口径归一
 - 用本体做资产**&BOM**&设备静态建模
 - 用本体做语义约束&治幻觉
 - 集成商蹭概念范式→把旧东西换皮改名当本体卖
- 本体是个硬骨头，涉及到语义、操作等，已经远不是那个学术的状态，但做好，很不容易，持久战。



感谢观看